

Threshold ECDSA in Three Rounds

Jack Doerner



Yashvanth Kondi



Eysa Lee



BROWN

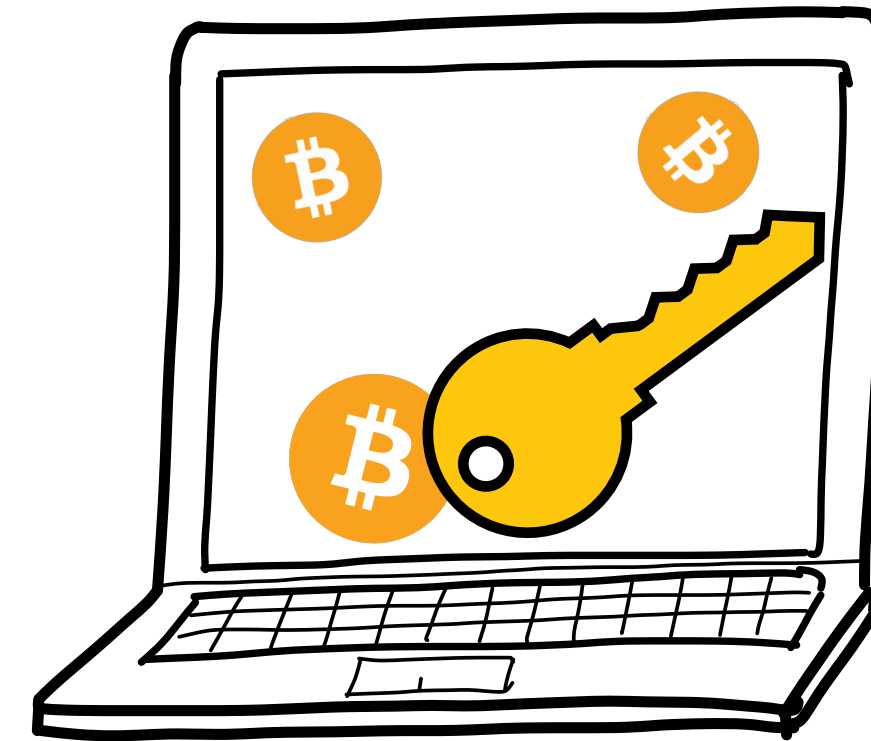
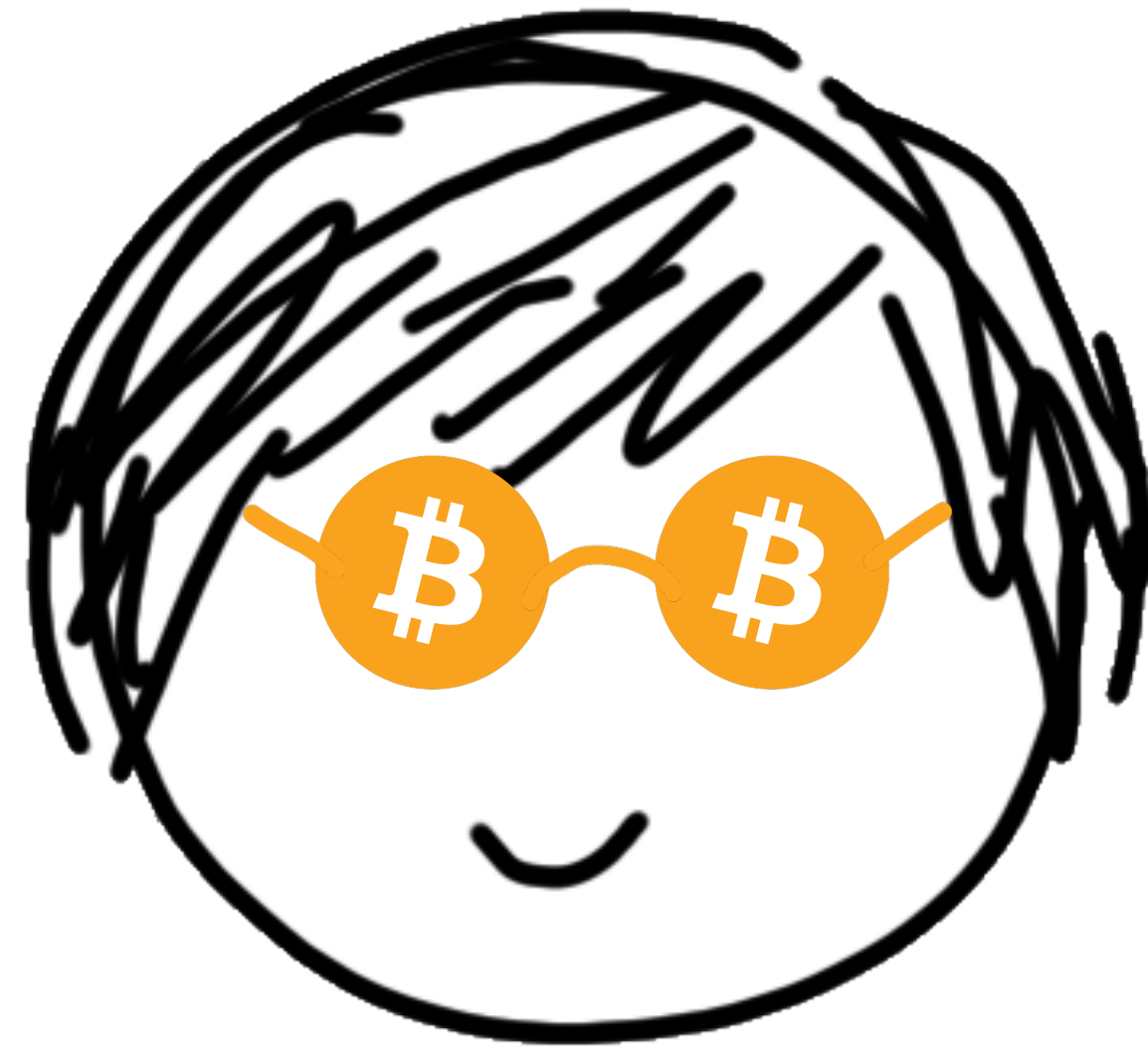
abhi shelat



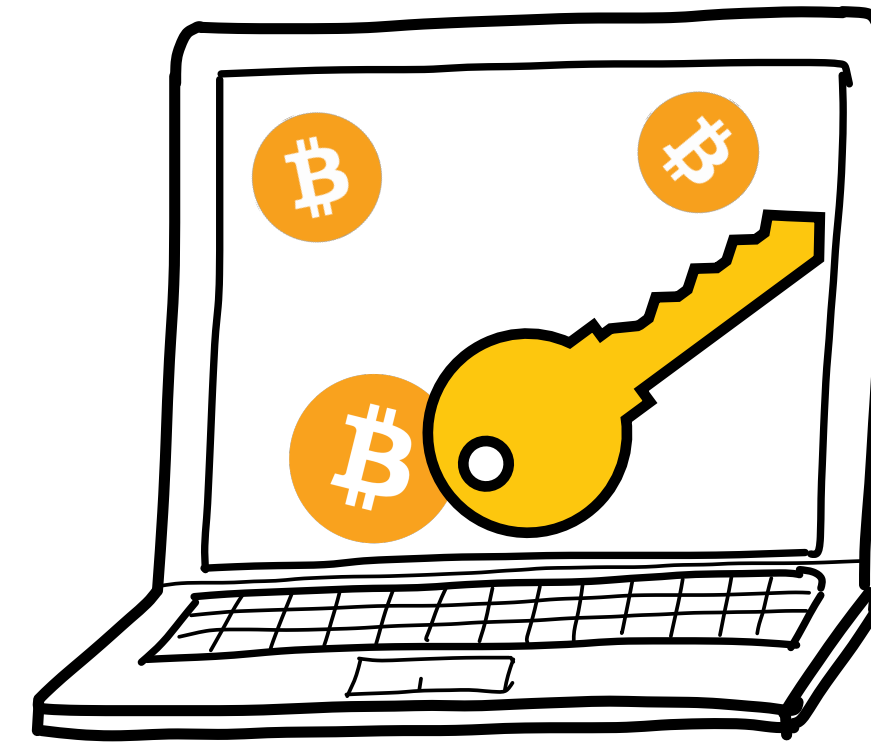
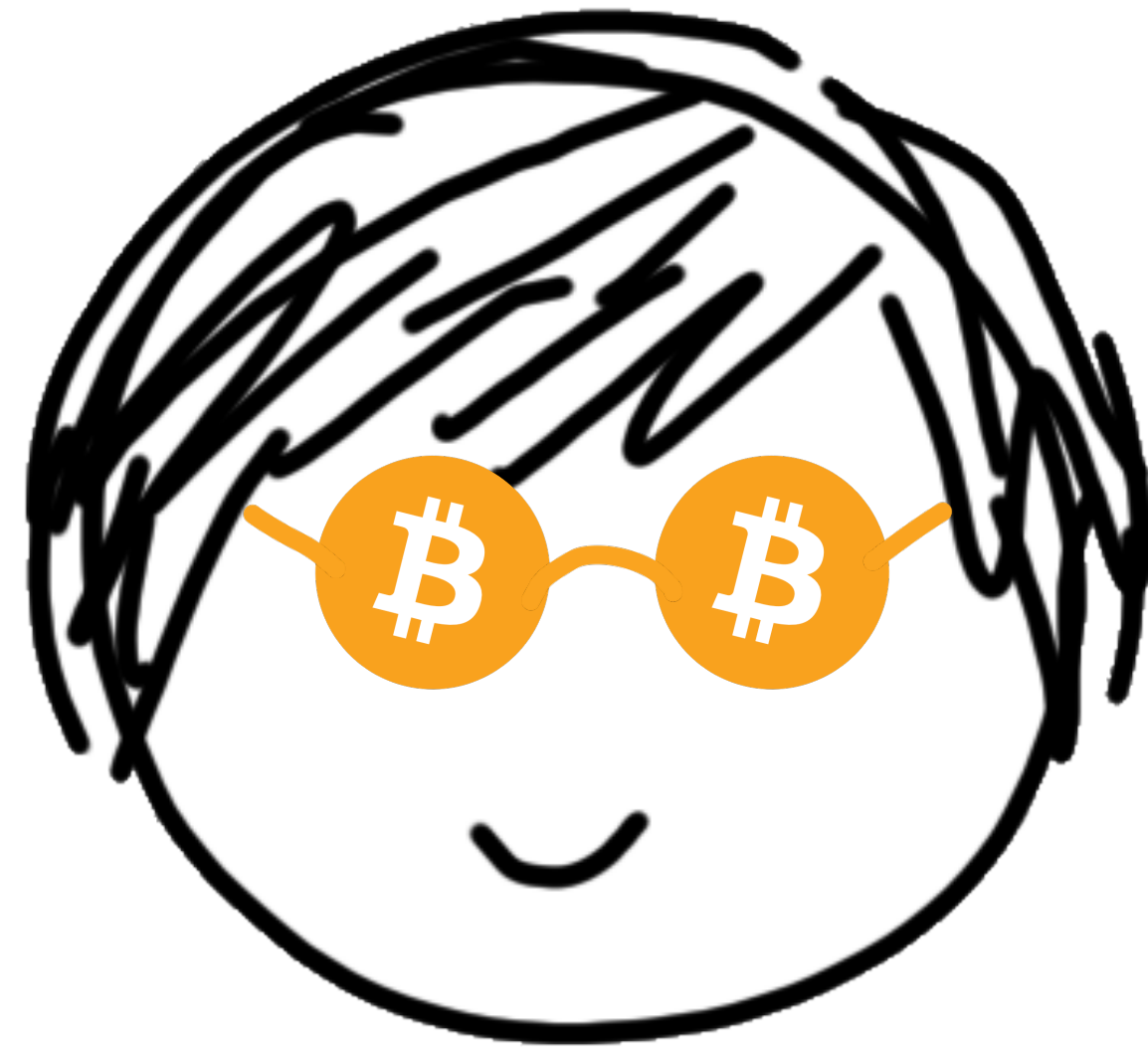
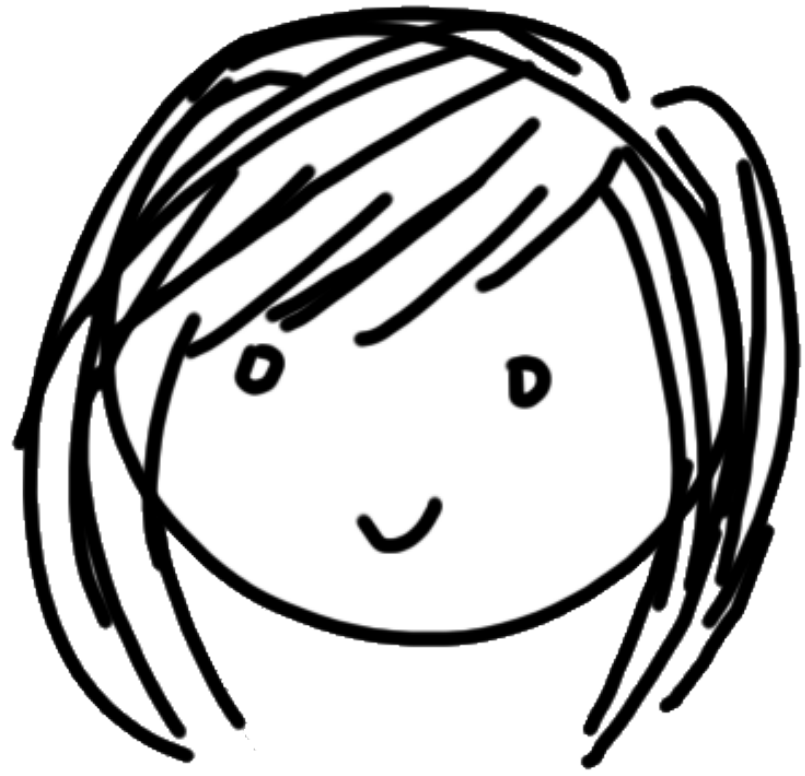
Northeastern
University

dkls.info

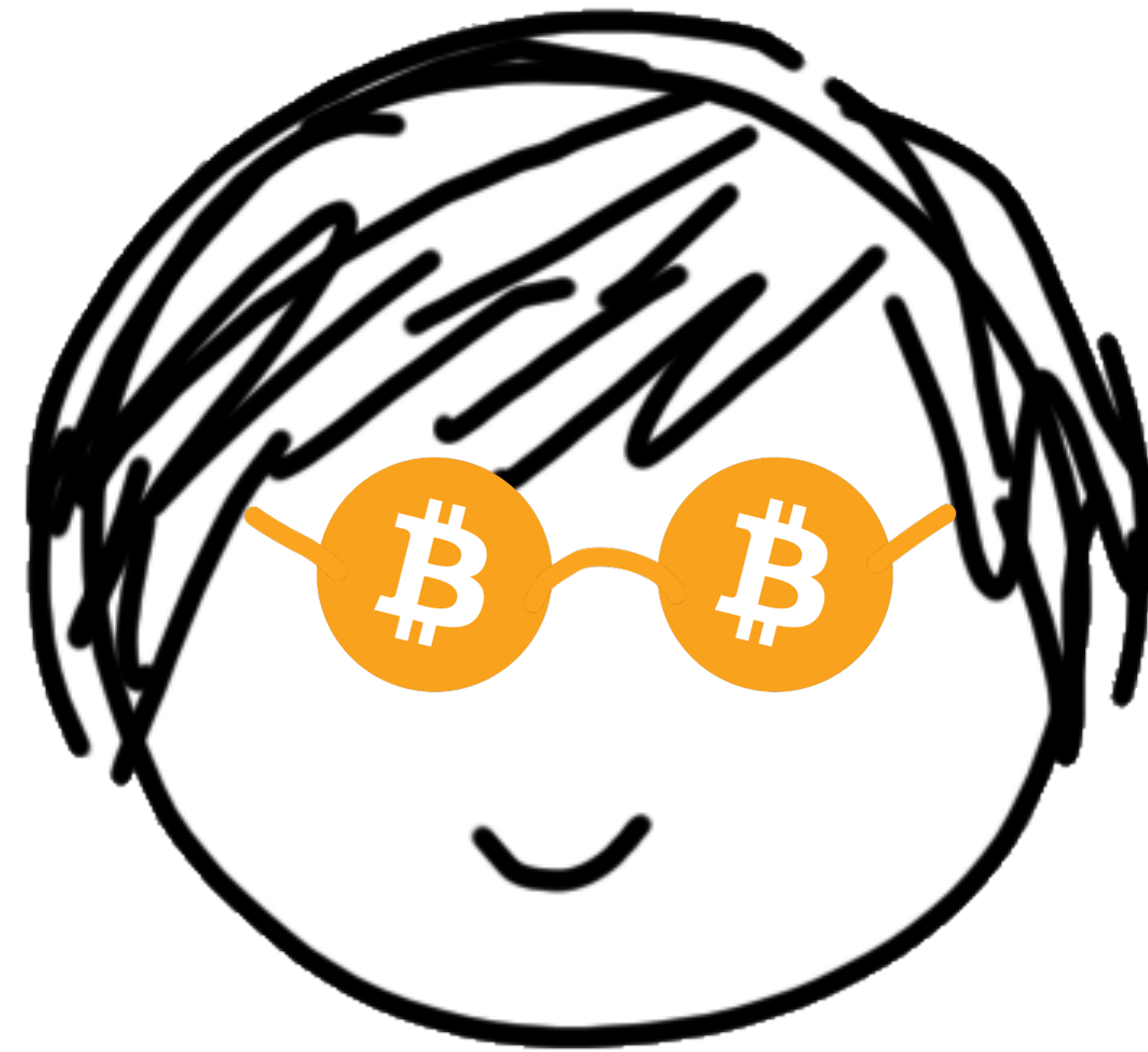
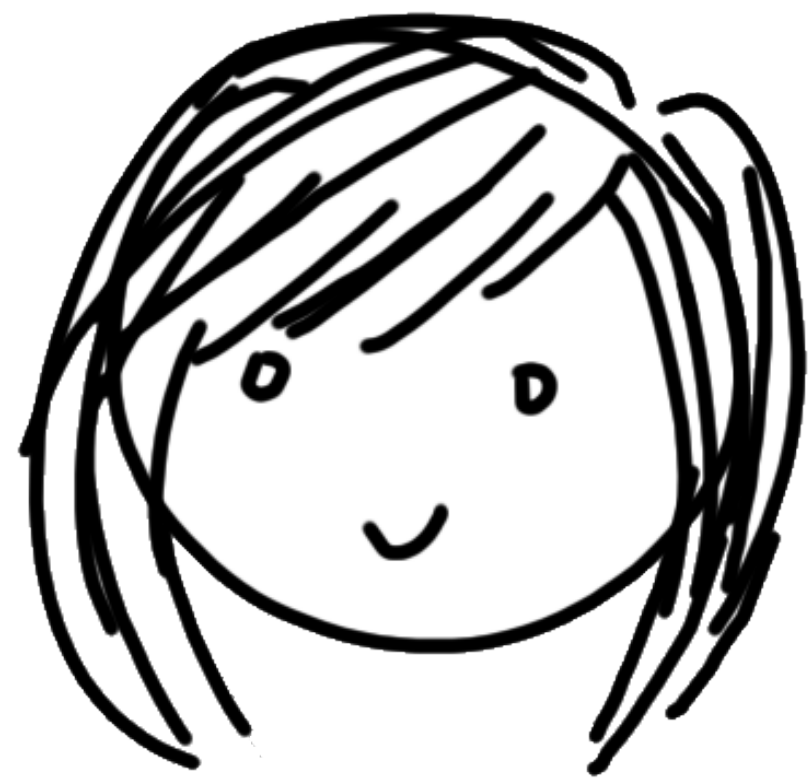
Ballad of *Bitcoin* Bob



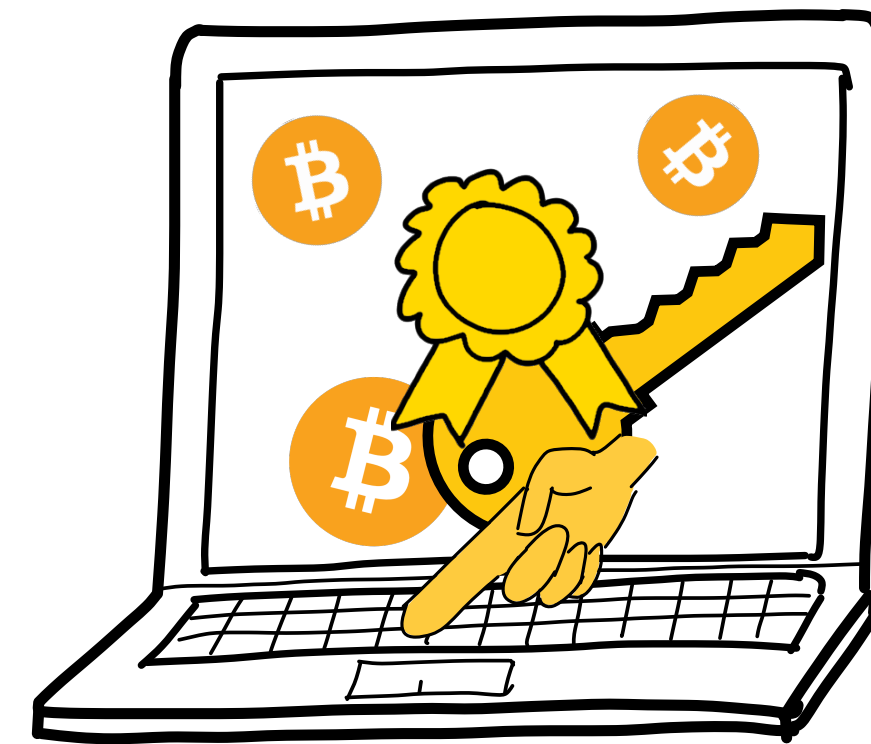
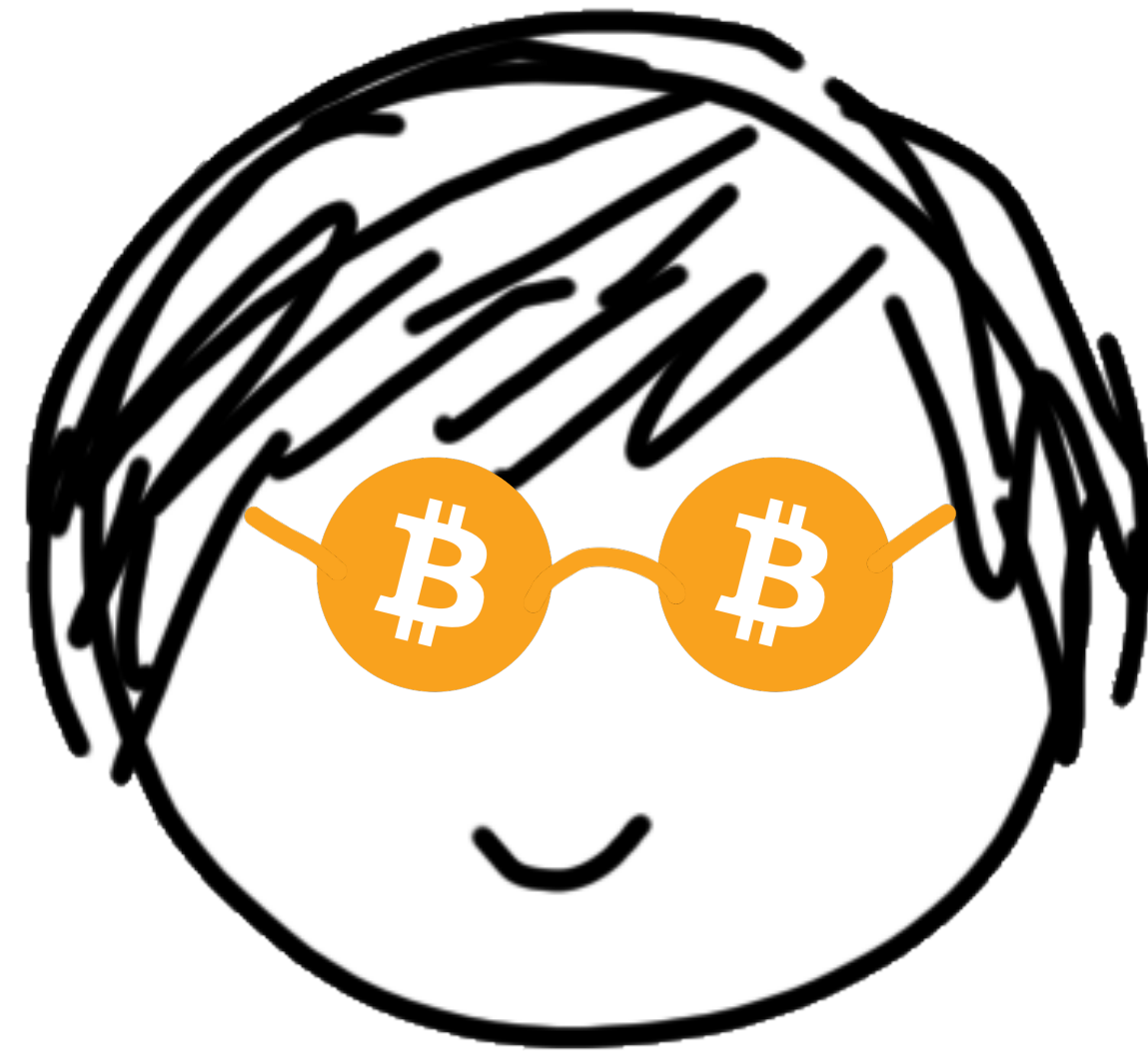
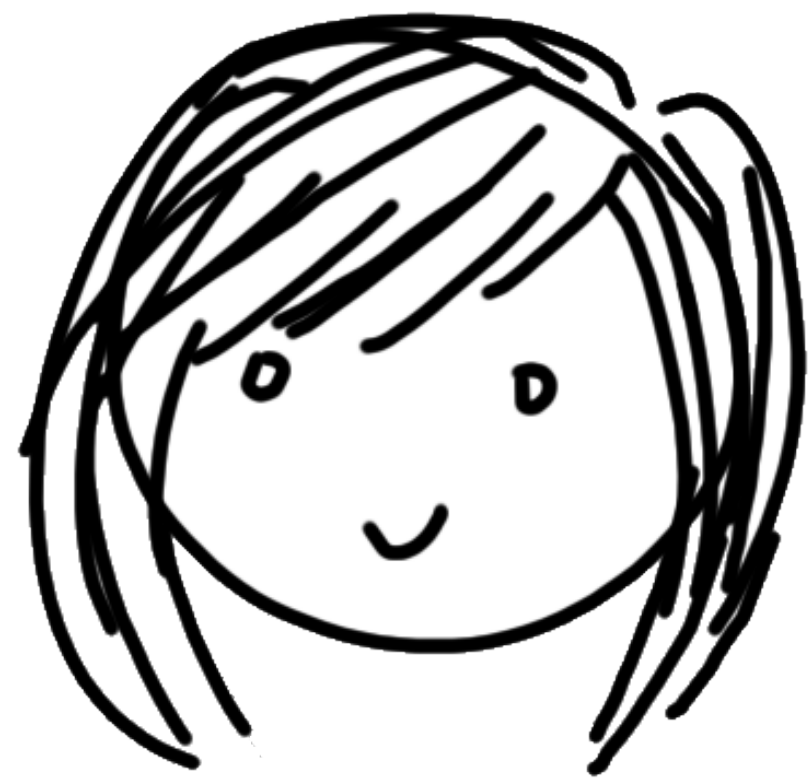
Ballad of Bitcoin Bob



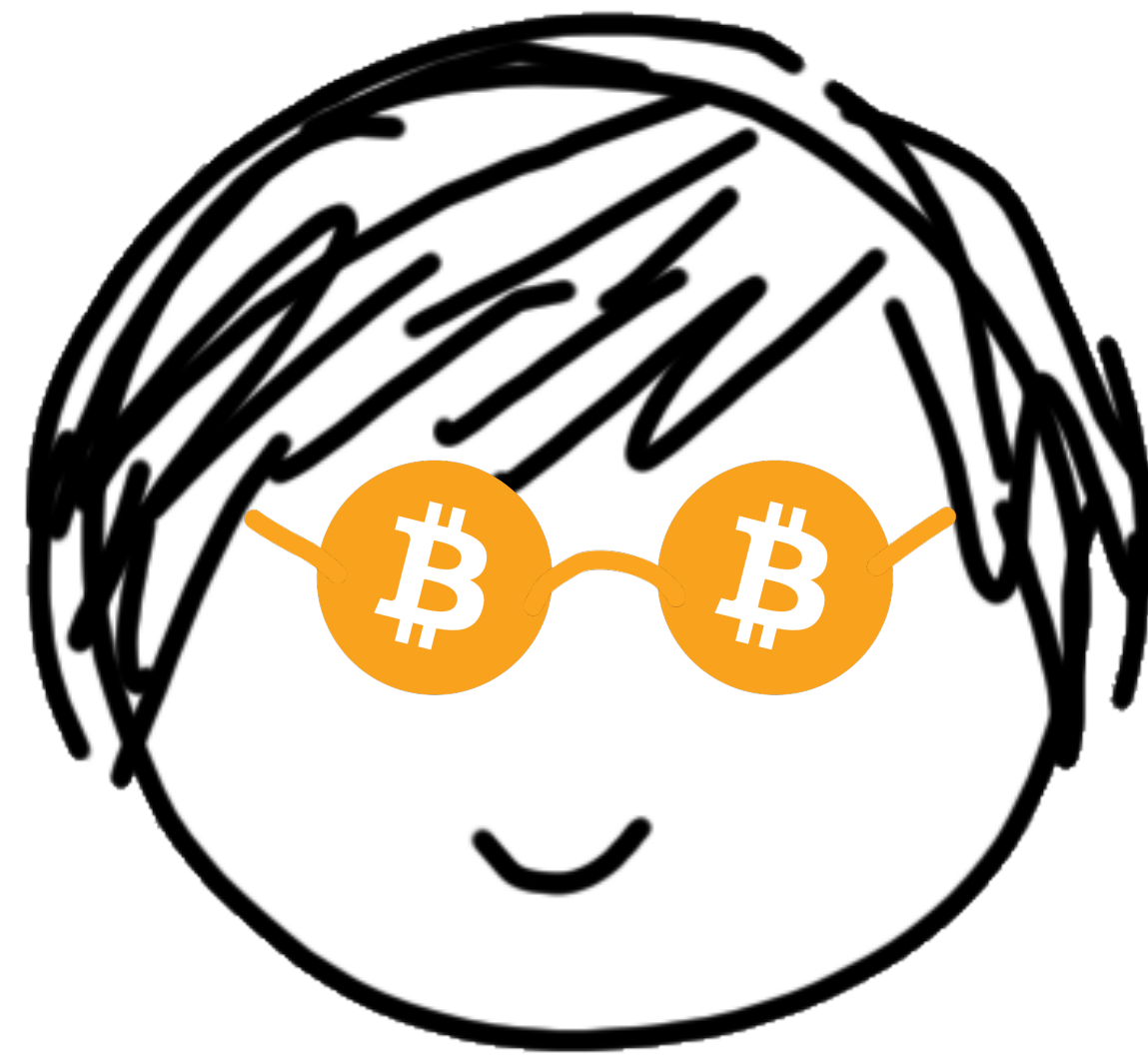
Ballad of Bitcoin Bob



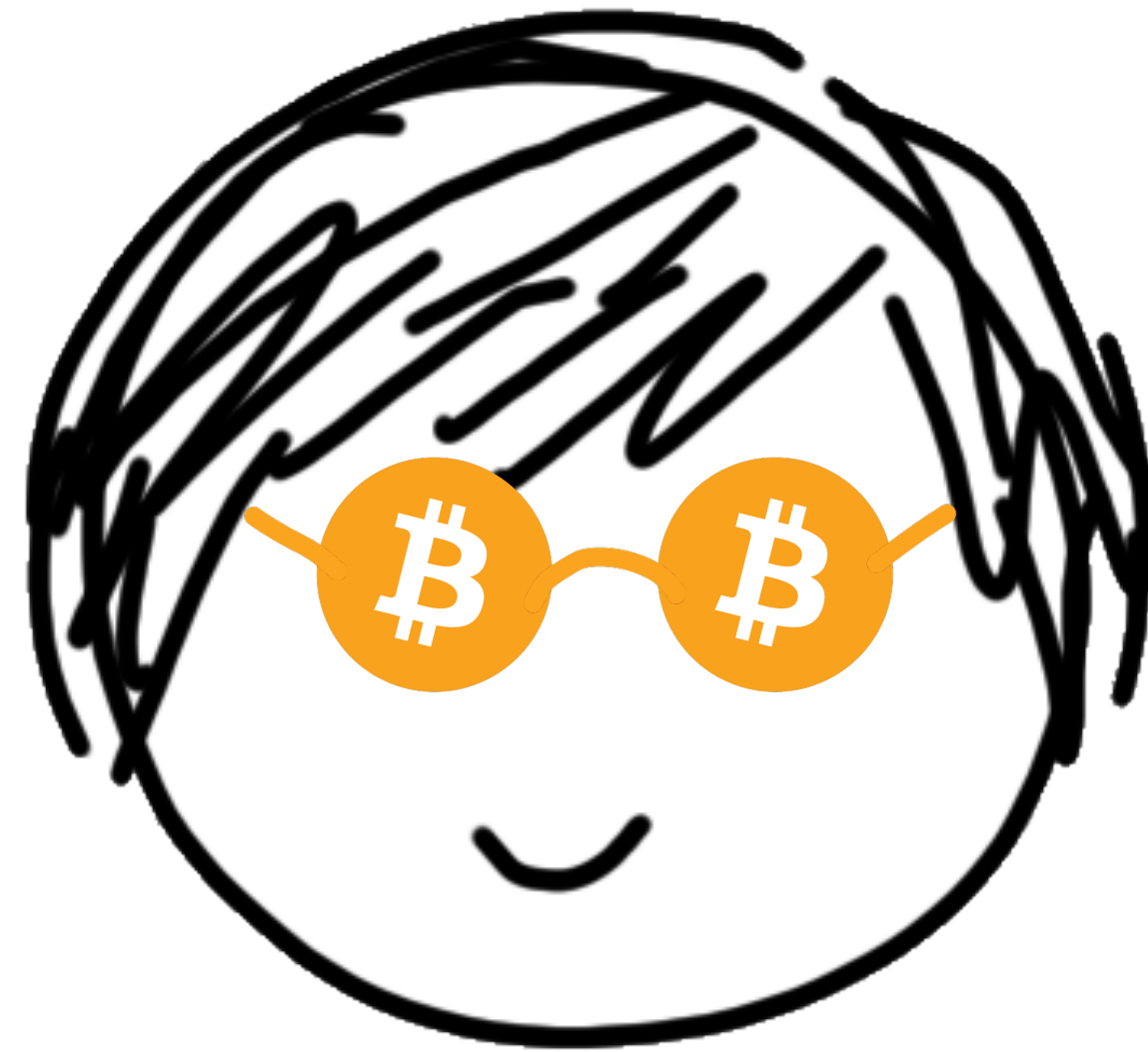
Ballad of Bitcoin Bob



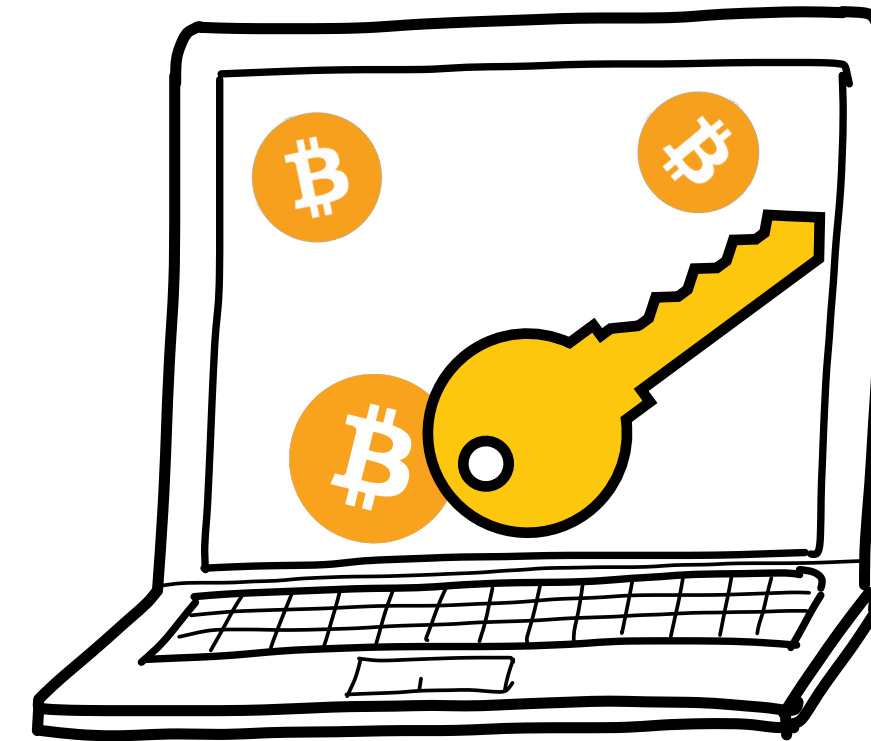
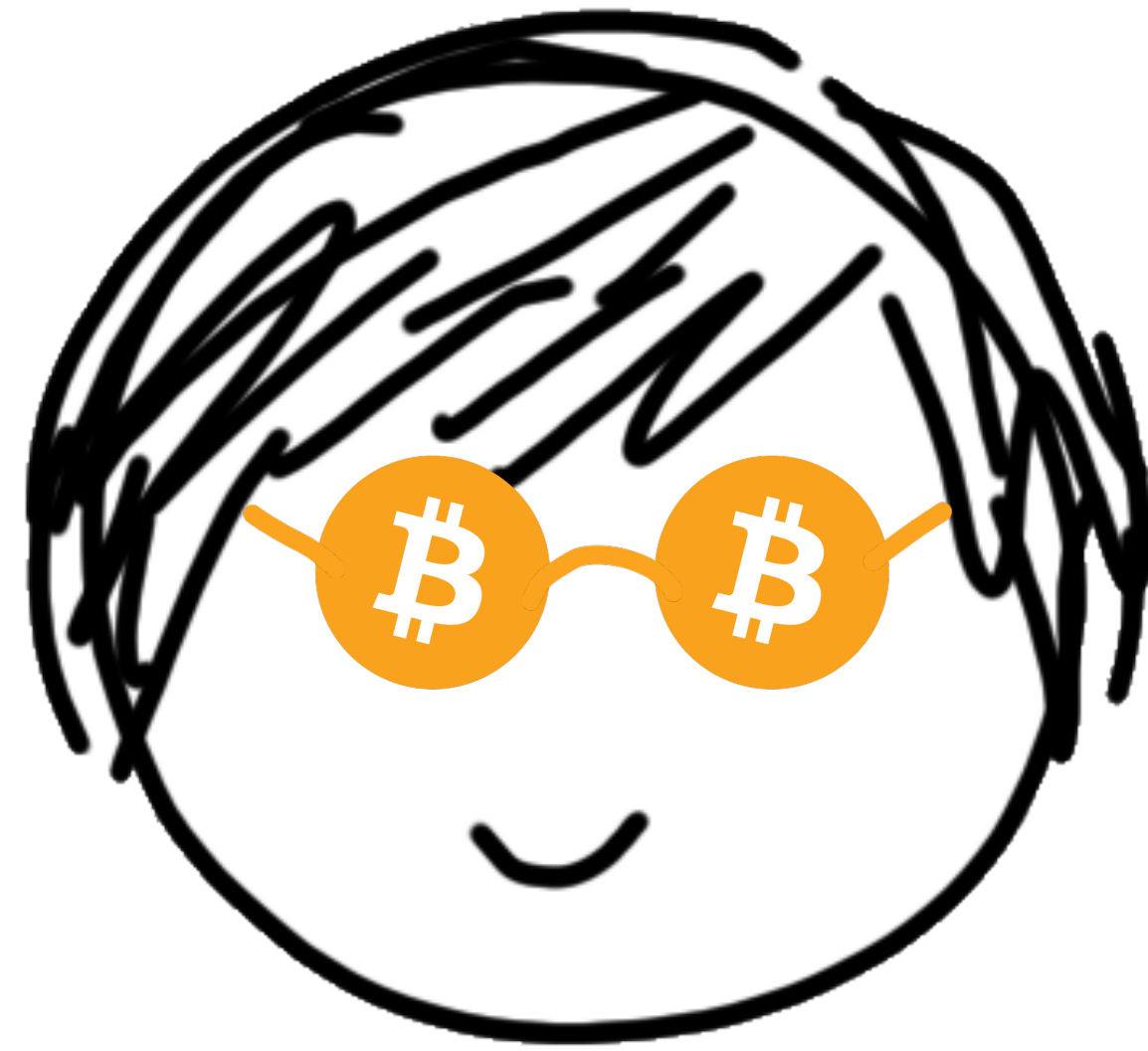
Ballad of Bitcoin Bob



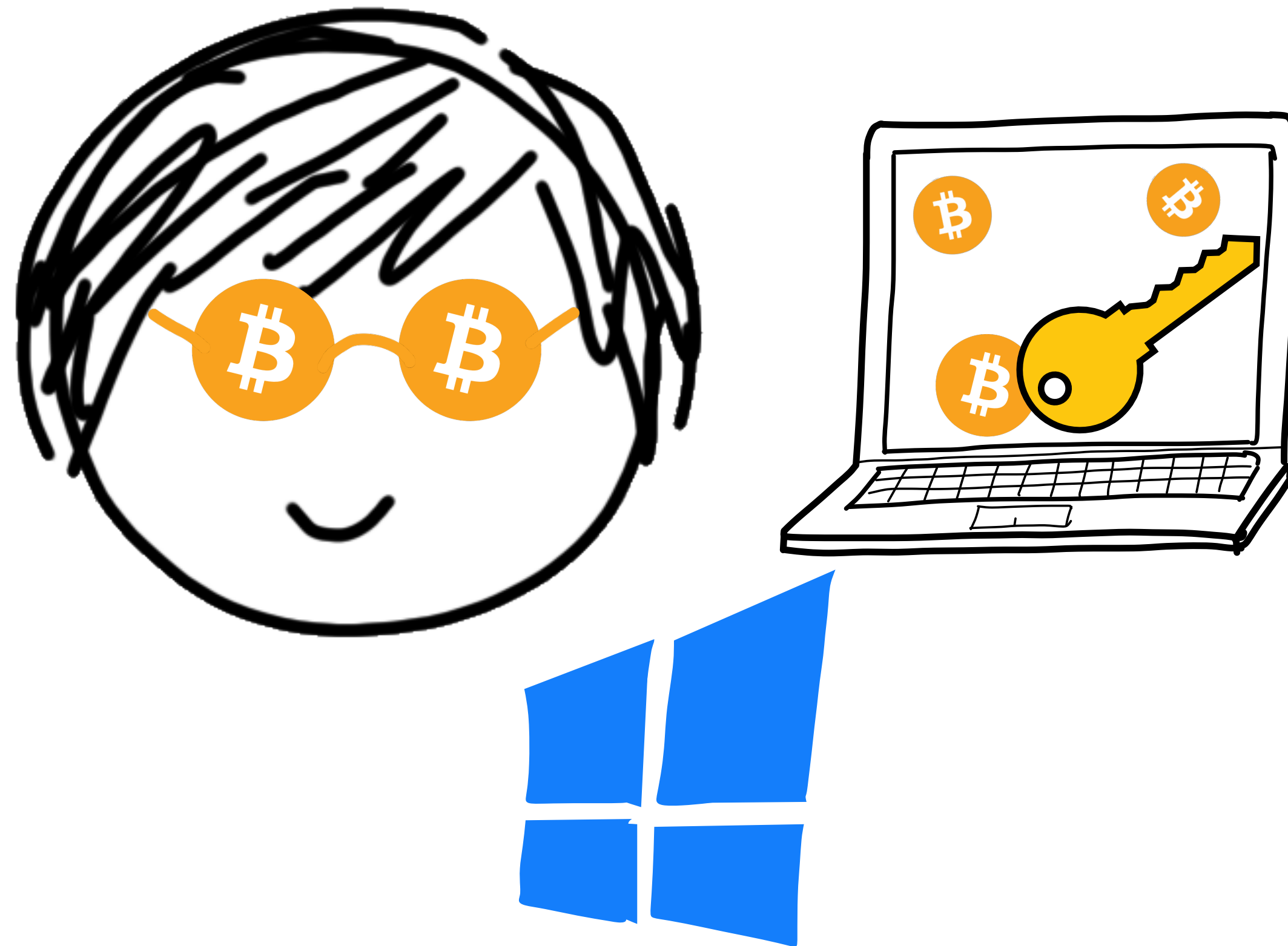
Ballad of *Bitcoin* Bob



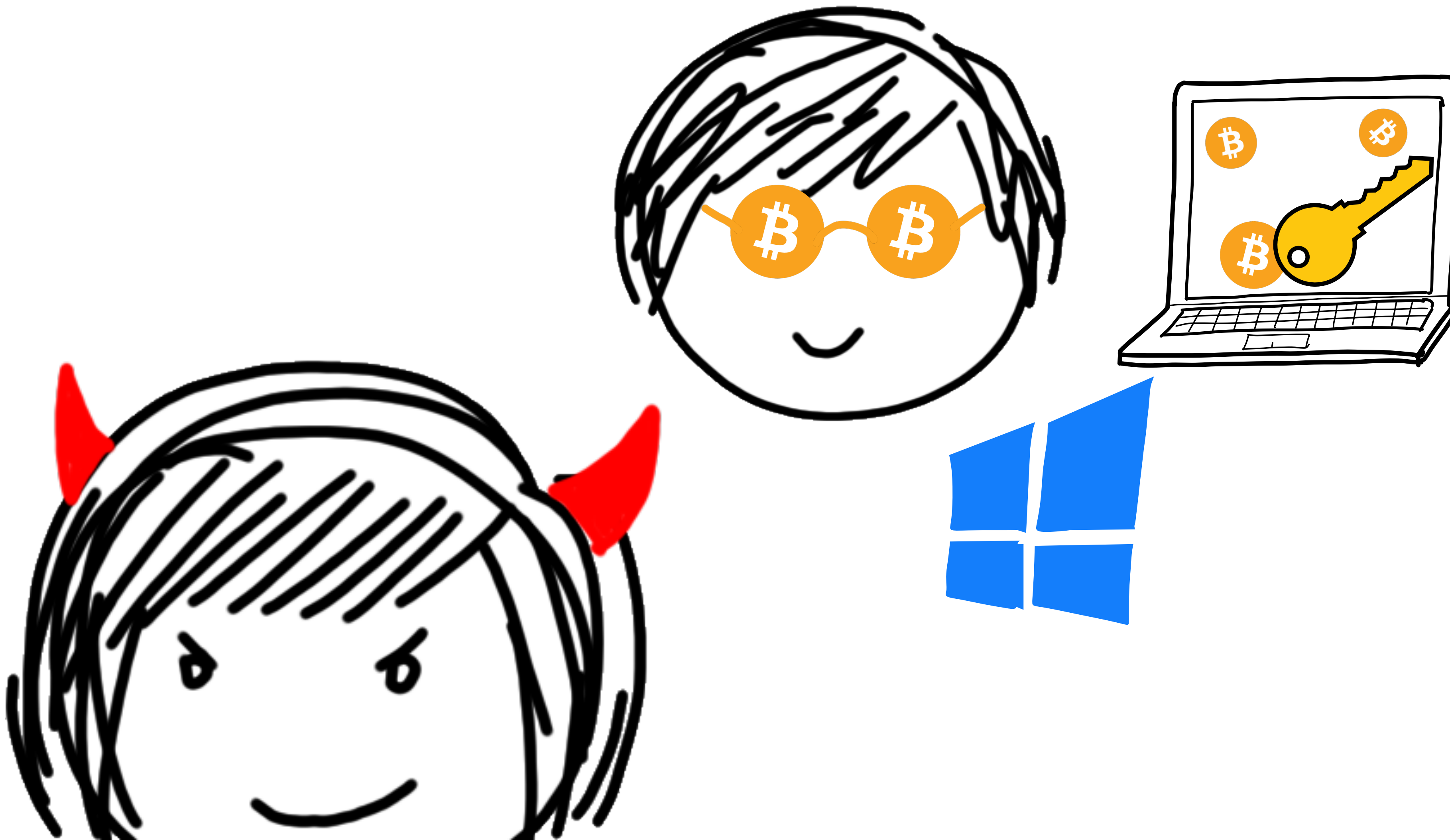
Ballad of *Bitcoin* Bob



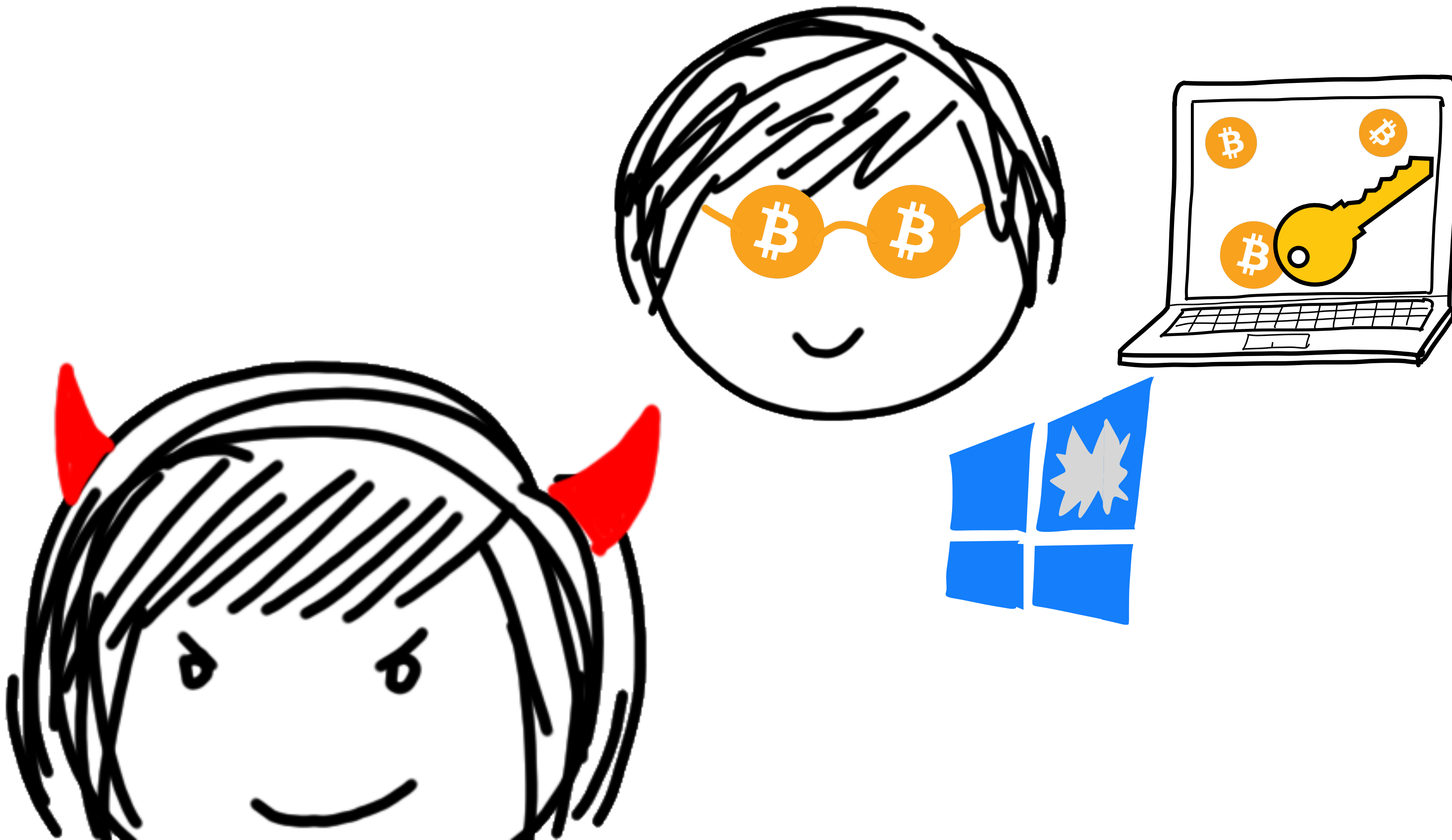
Ballad of Bitcoin Bob



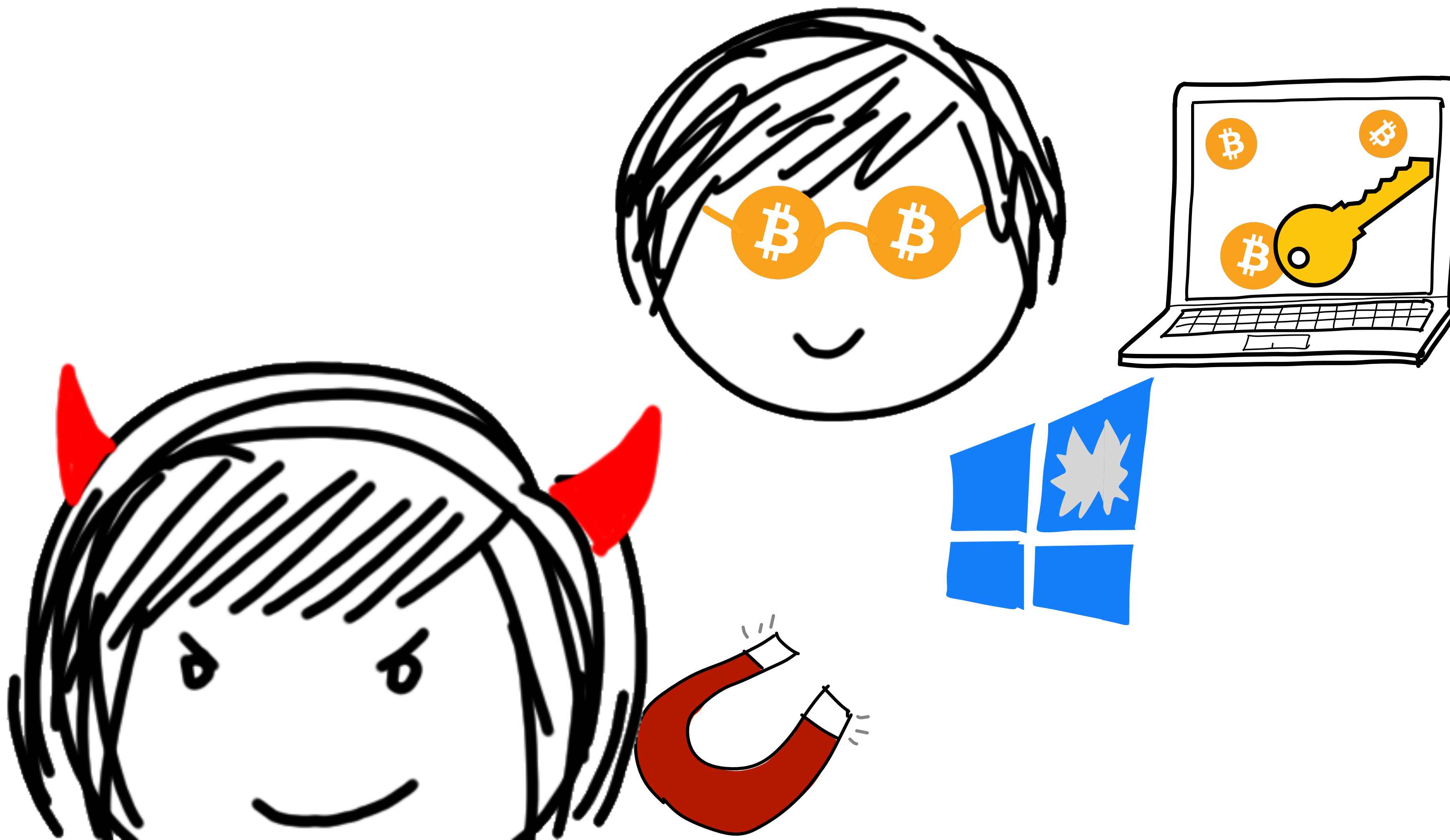
Ballad of Bitcoin Bob



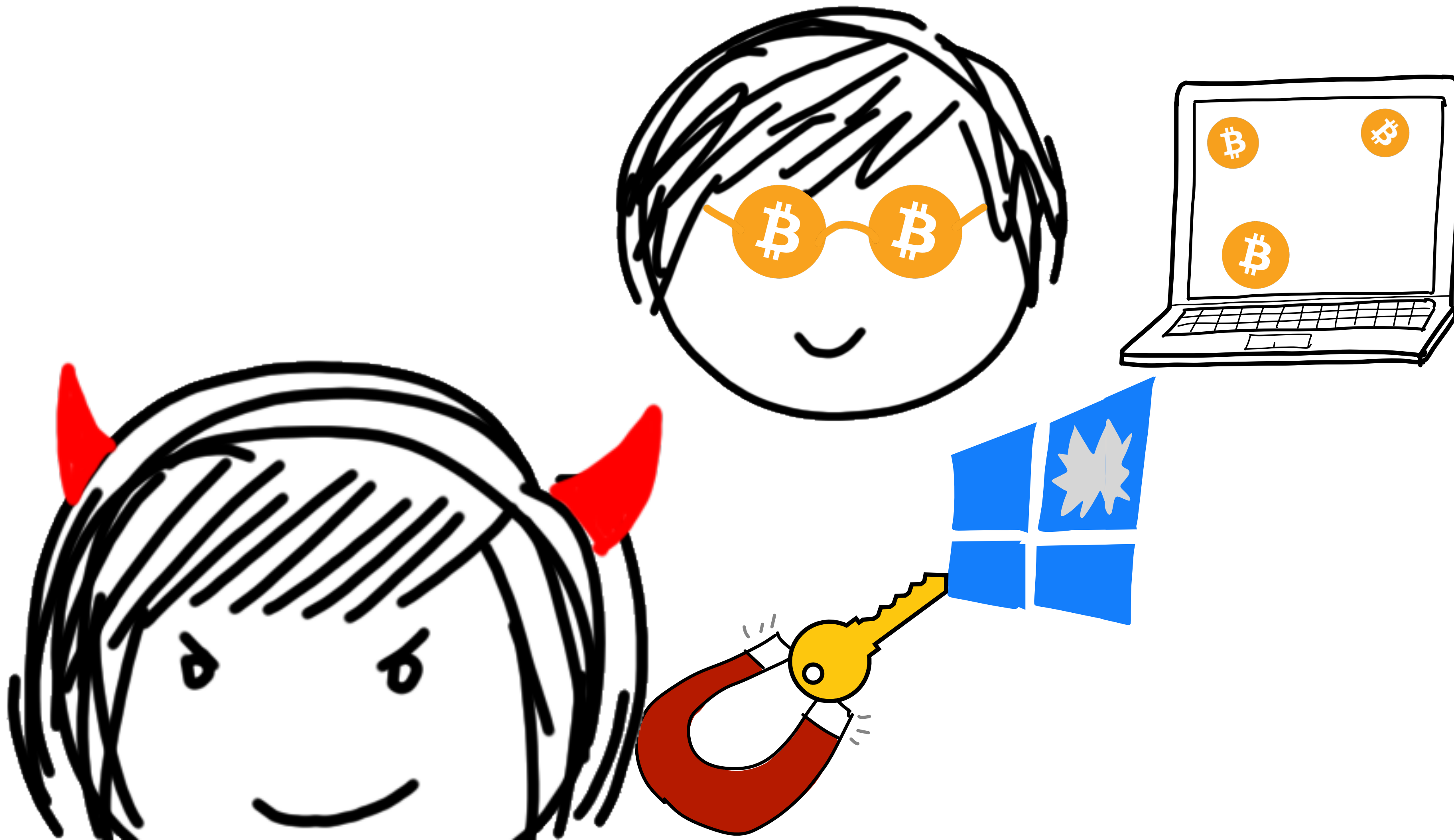
Ballad of Bitcoin Bob



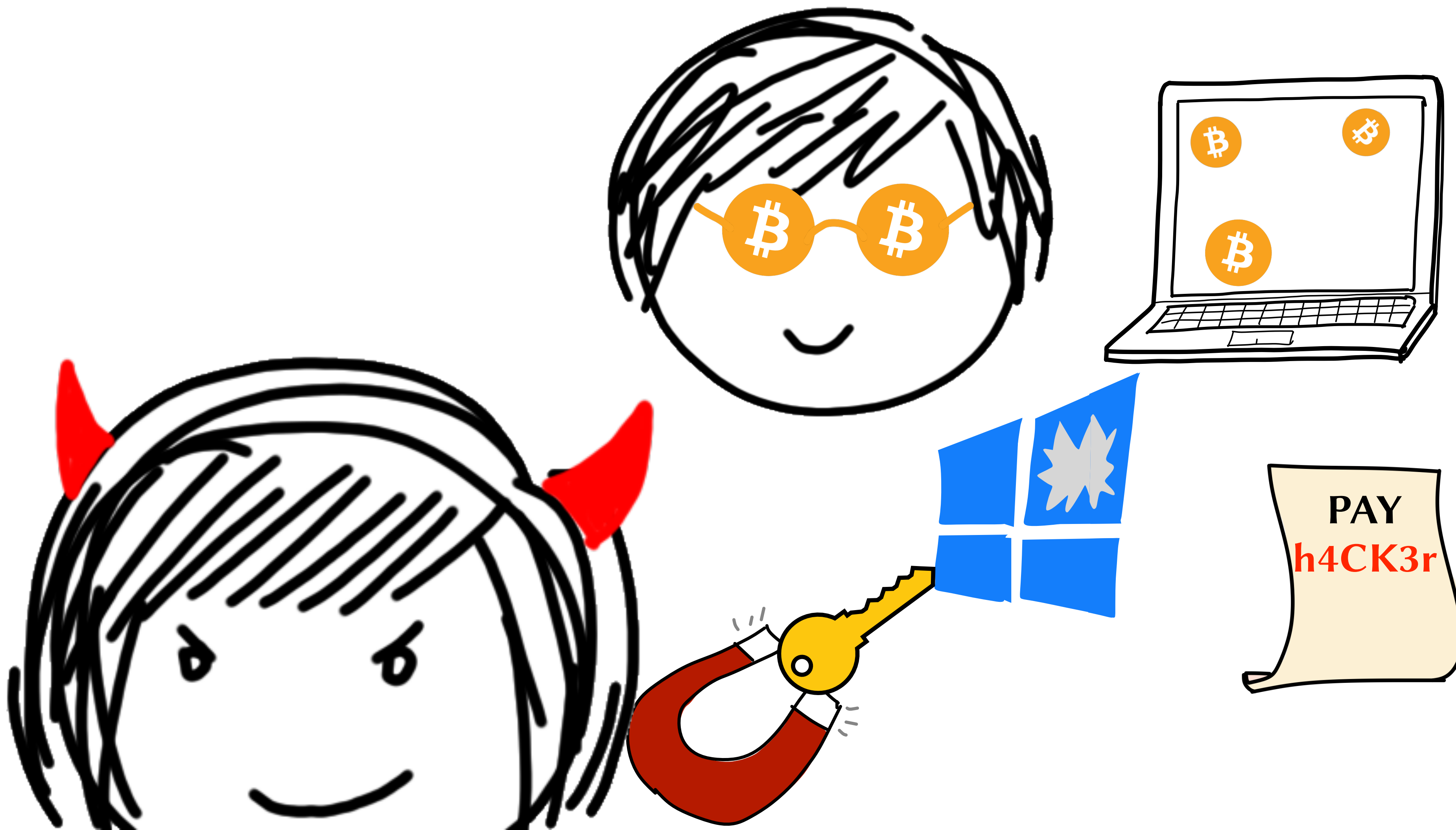
Ballad of Bitcoin Bob



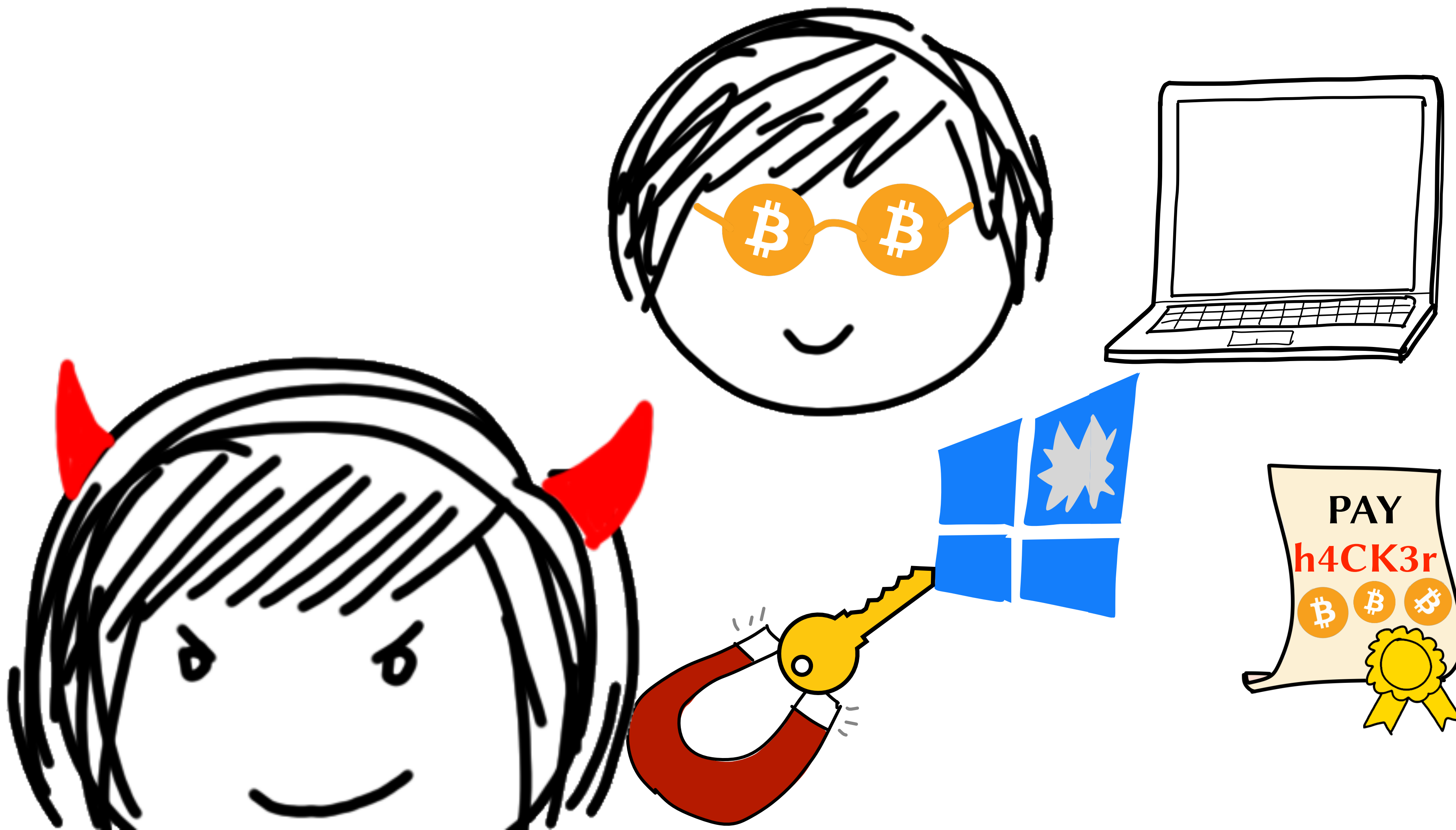
Ballad of Bitcoin Bob



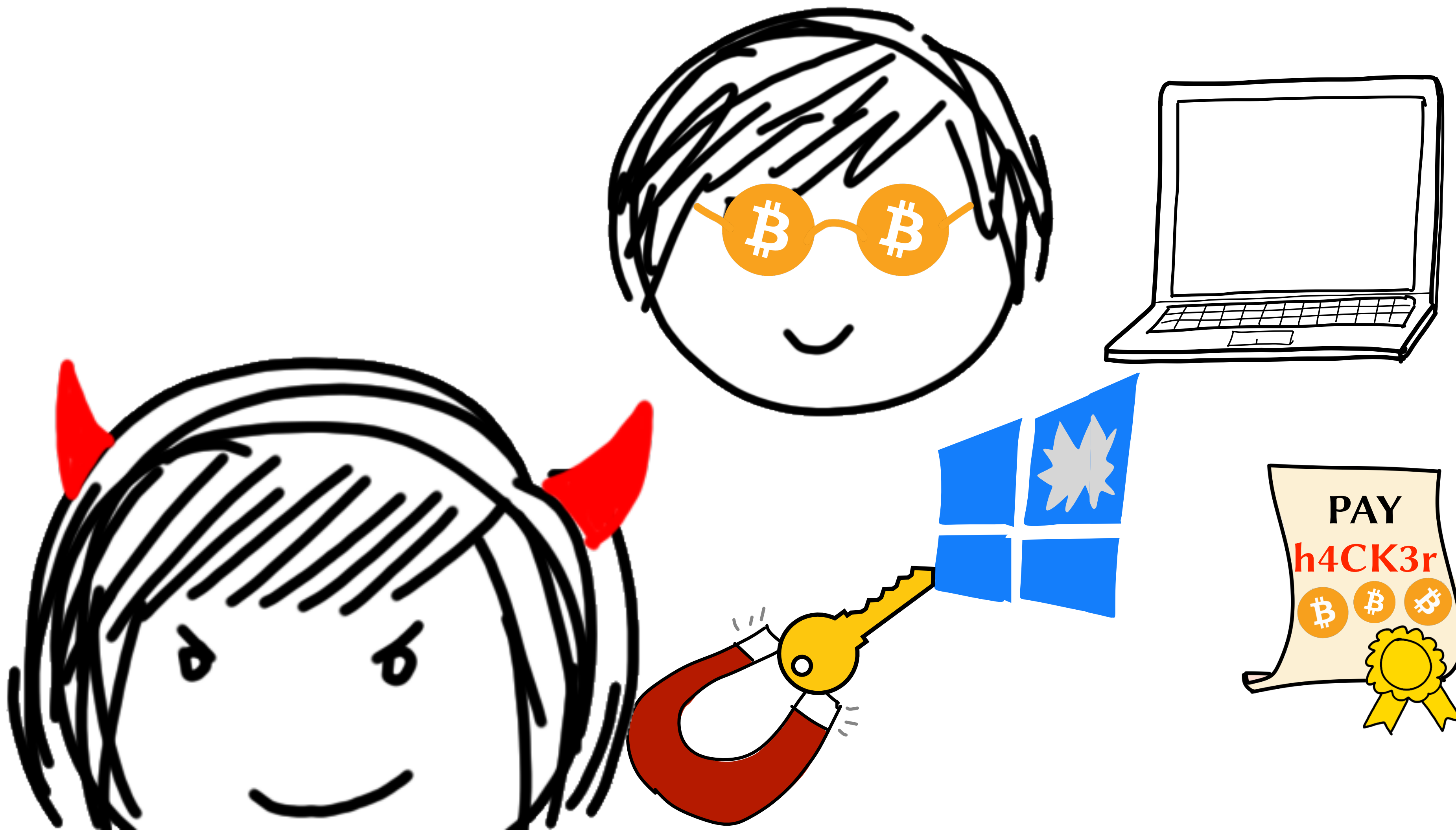
Ballad of Bitcoin Bob



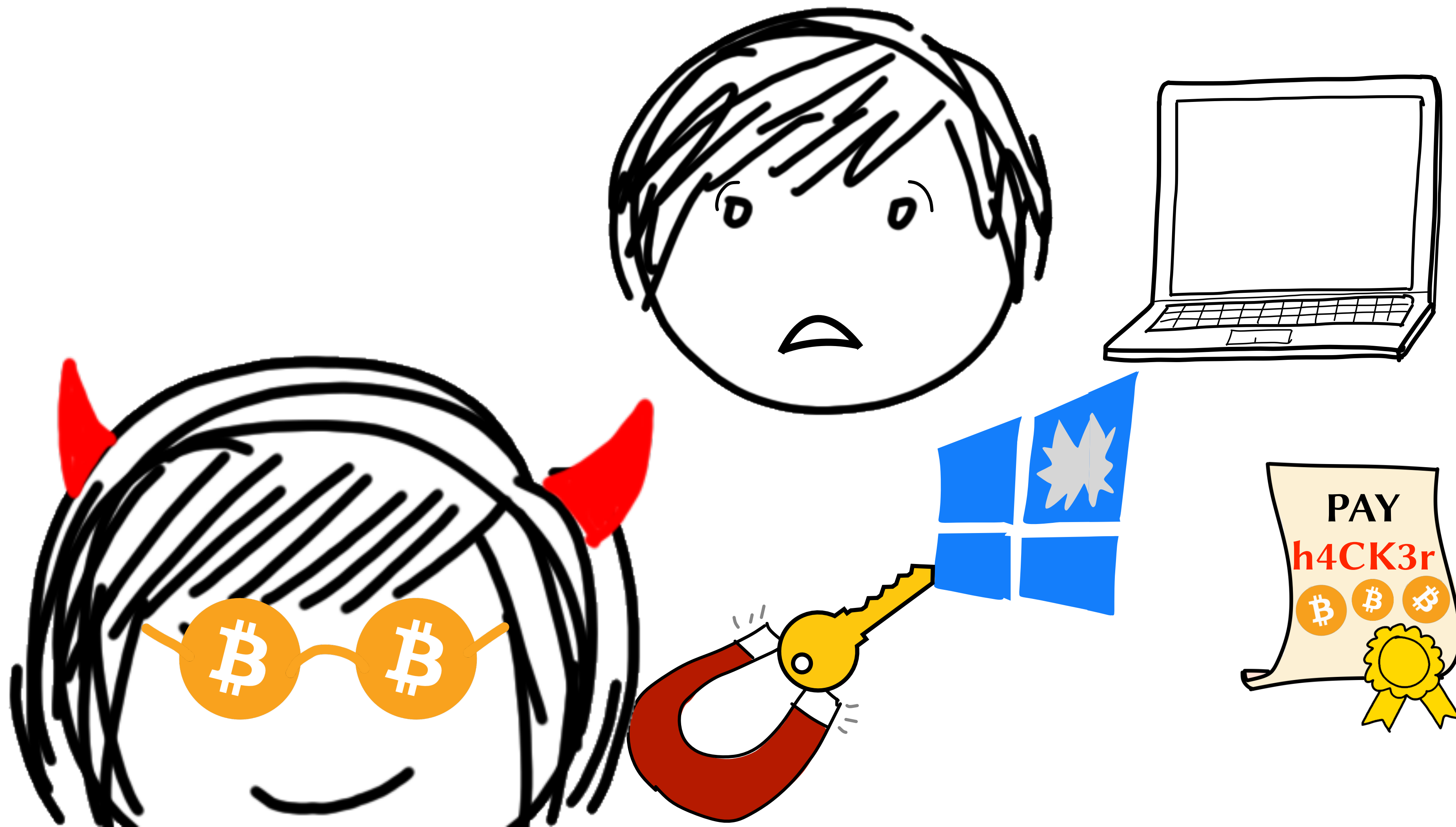
Ballad of Bitcoin Bob



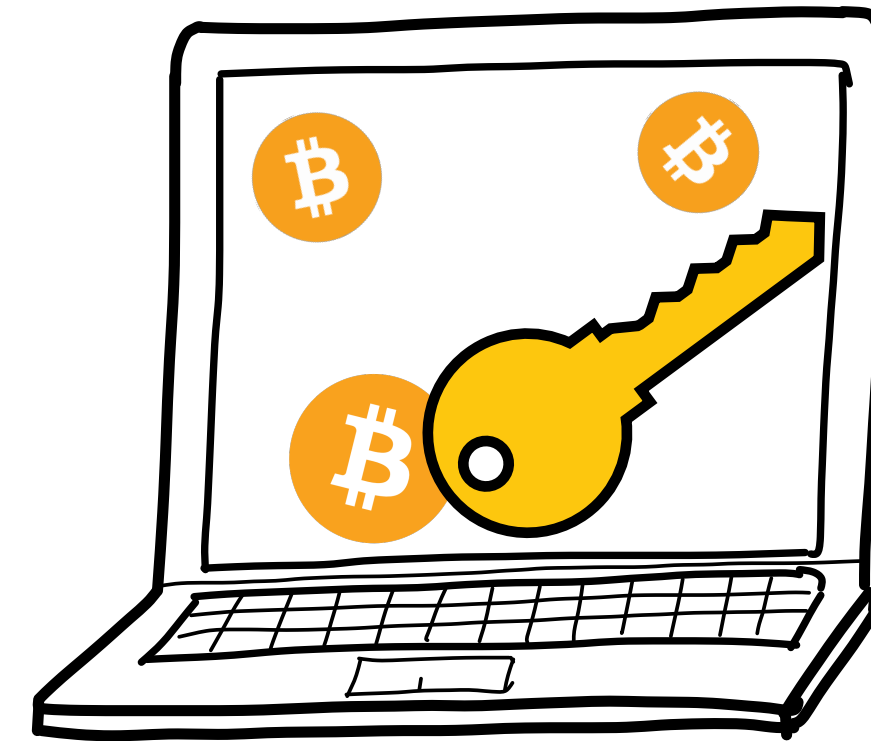
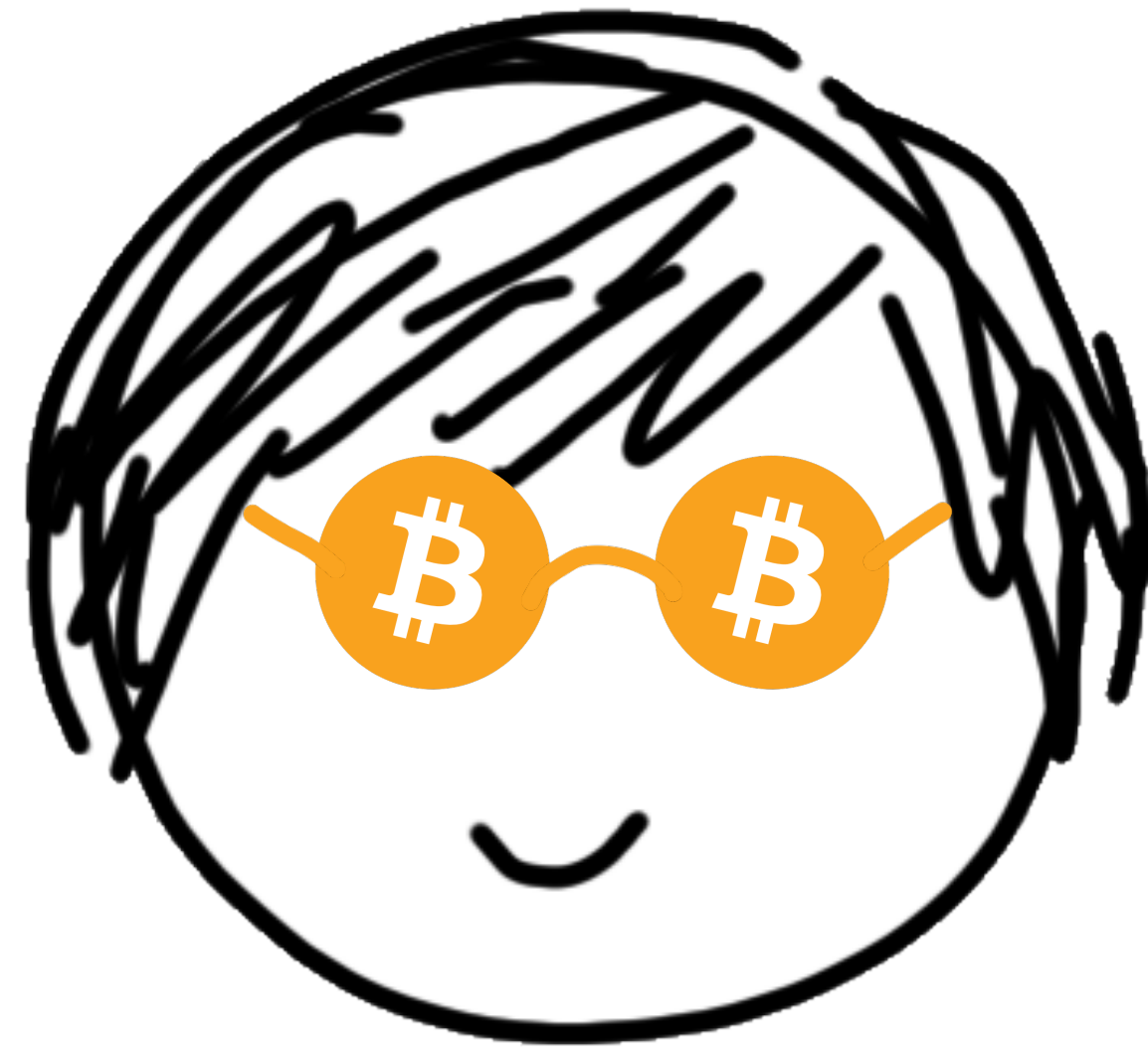
Ballad of Bitcoin Bob



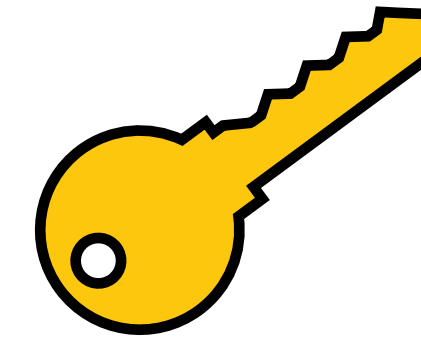
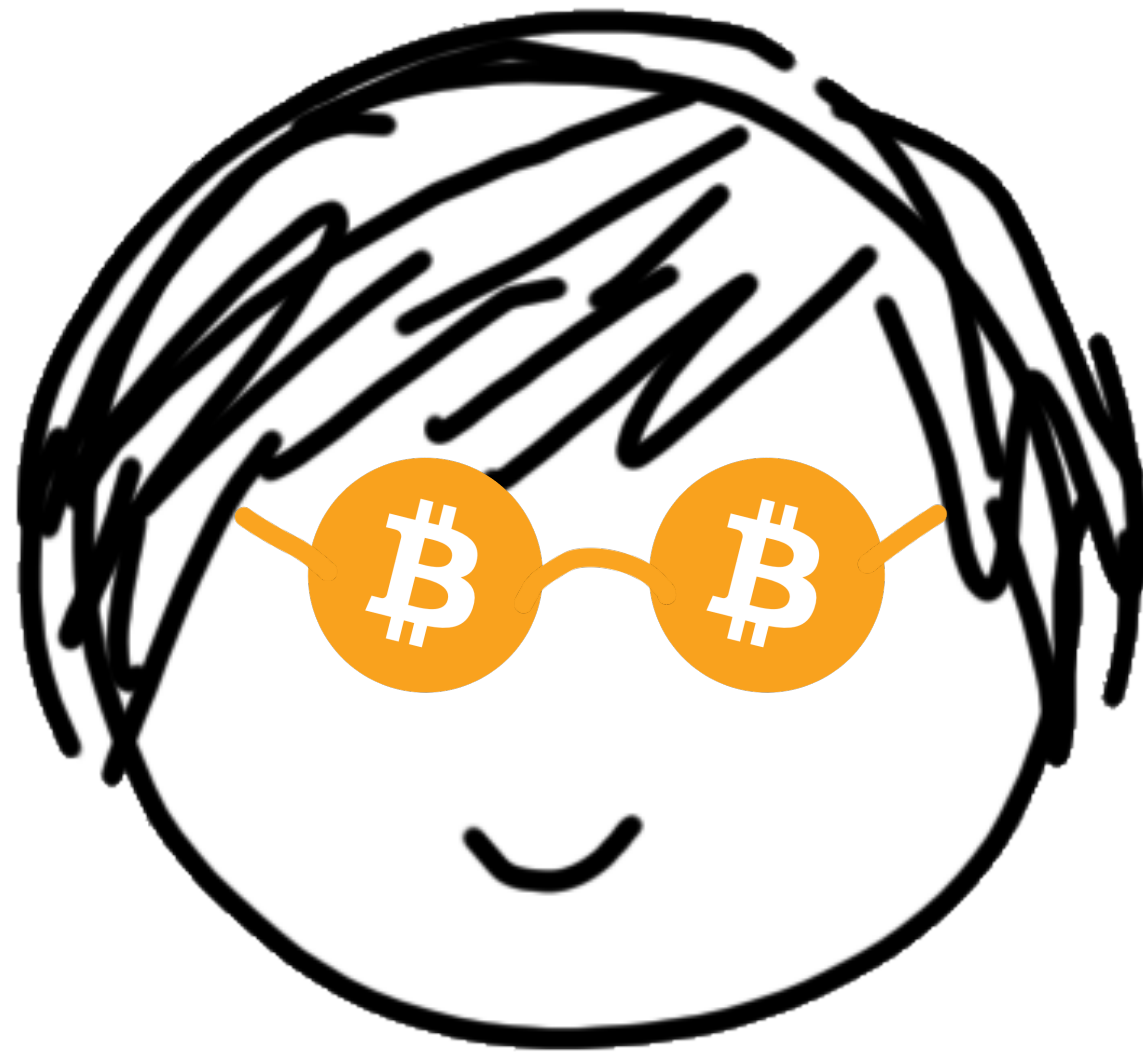
Ballad of Bitcoin Bob



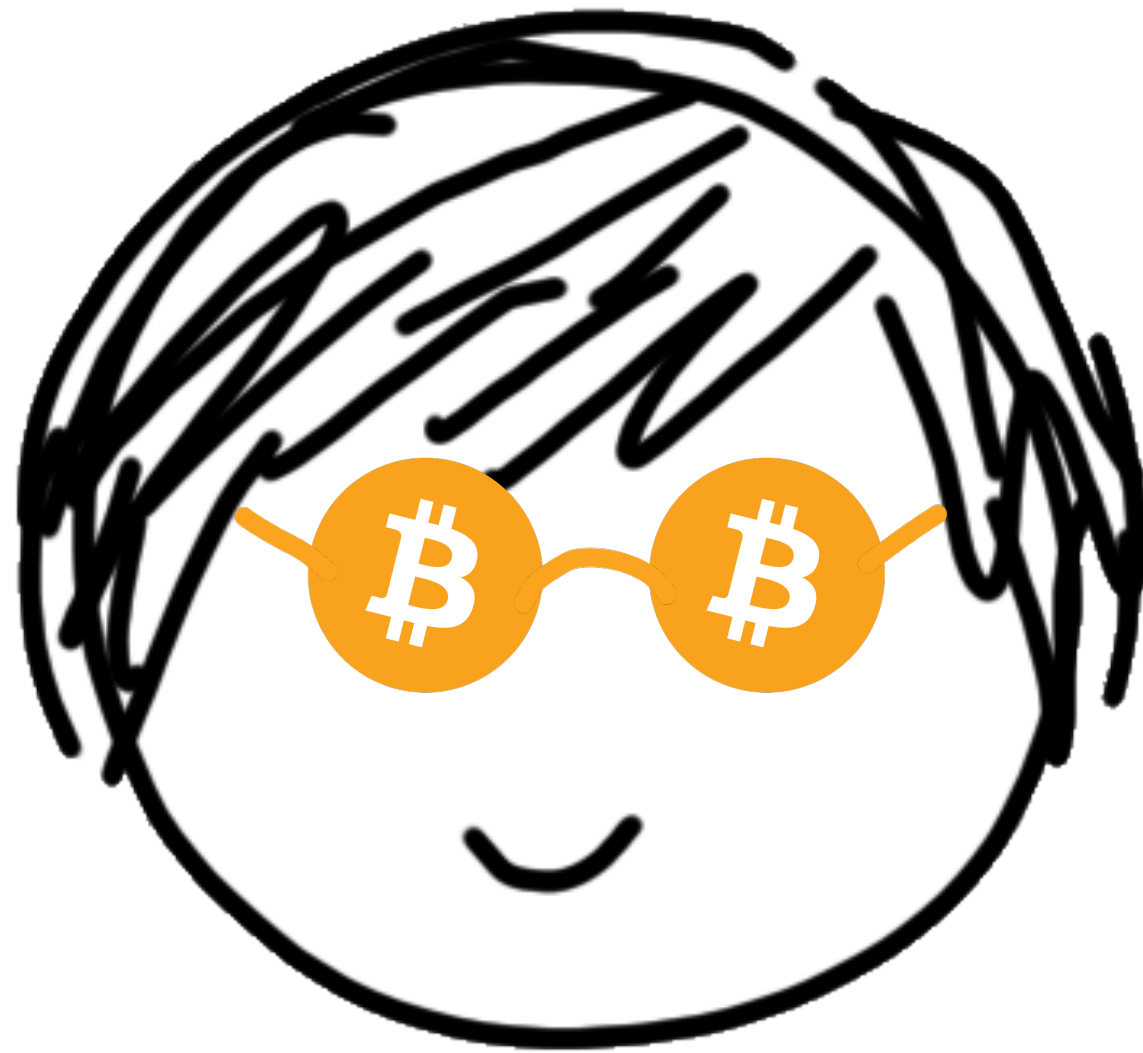
Ballad of Bitcoin Bob



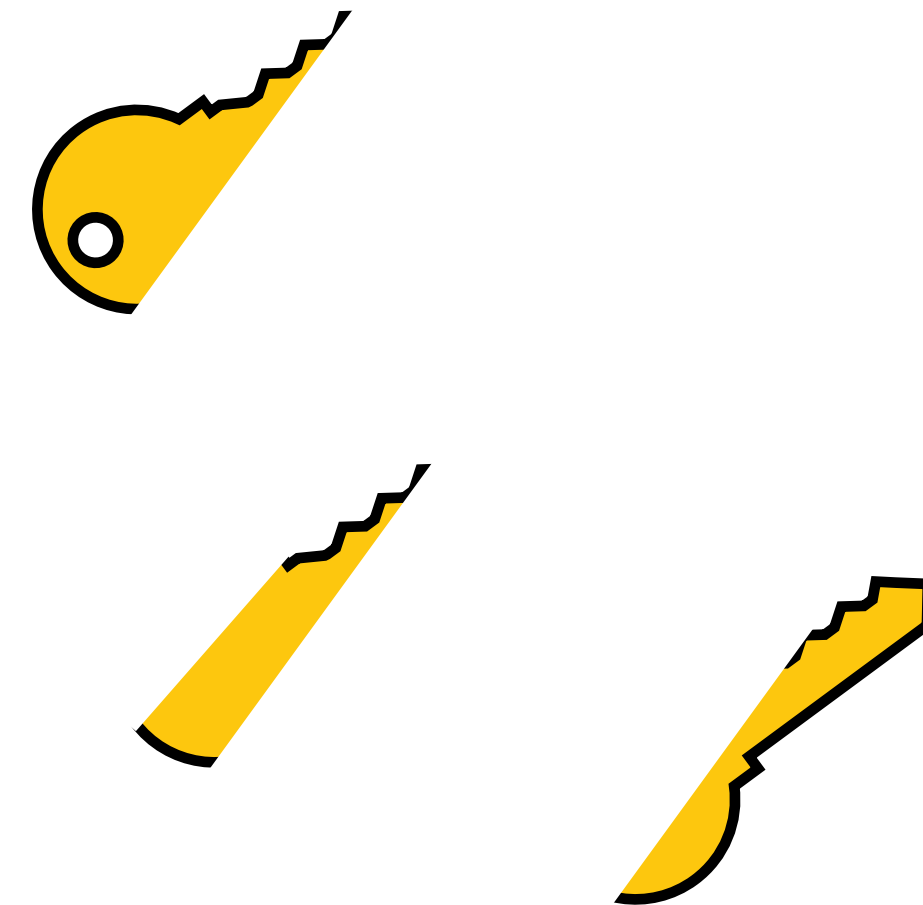
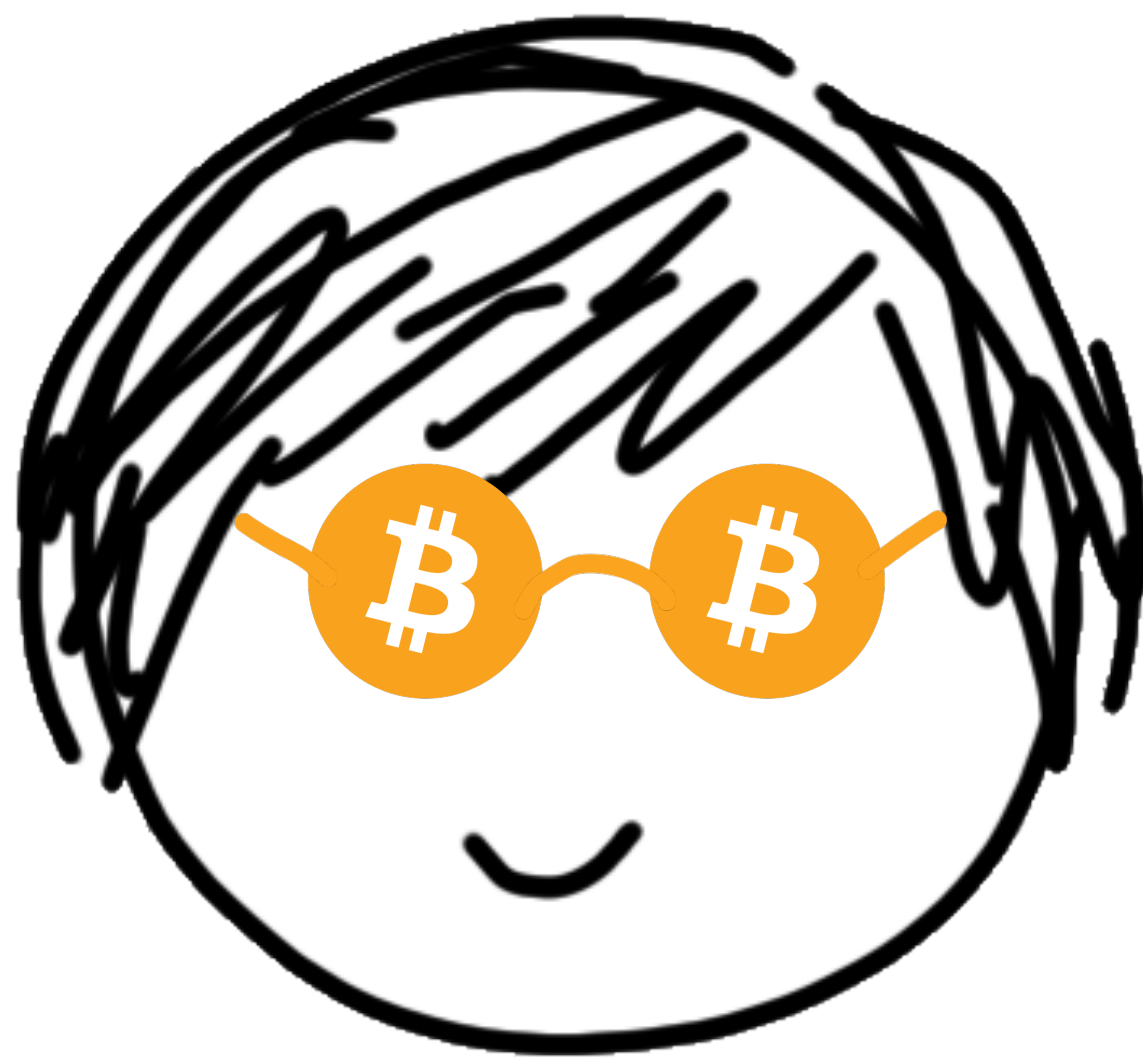
Threshold Signing



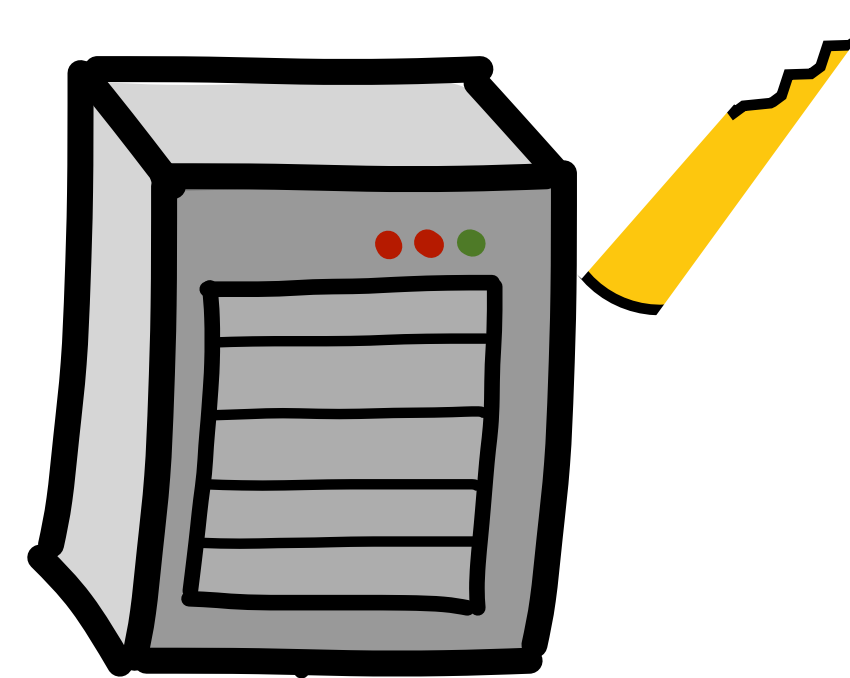
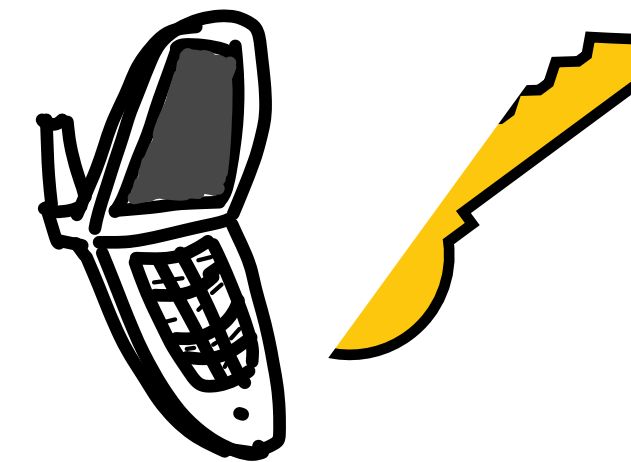
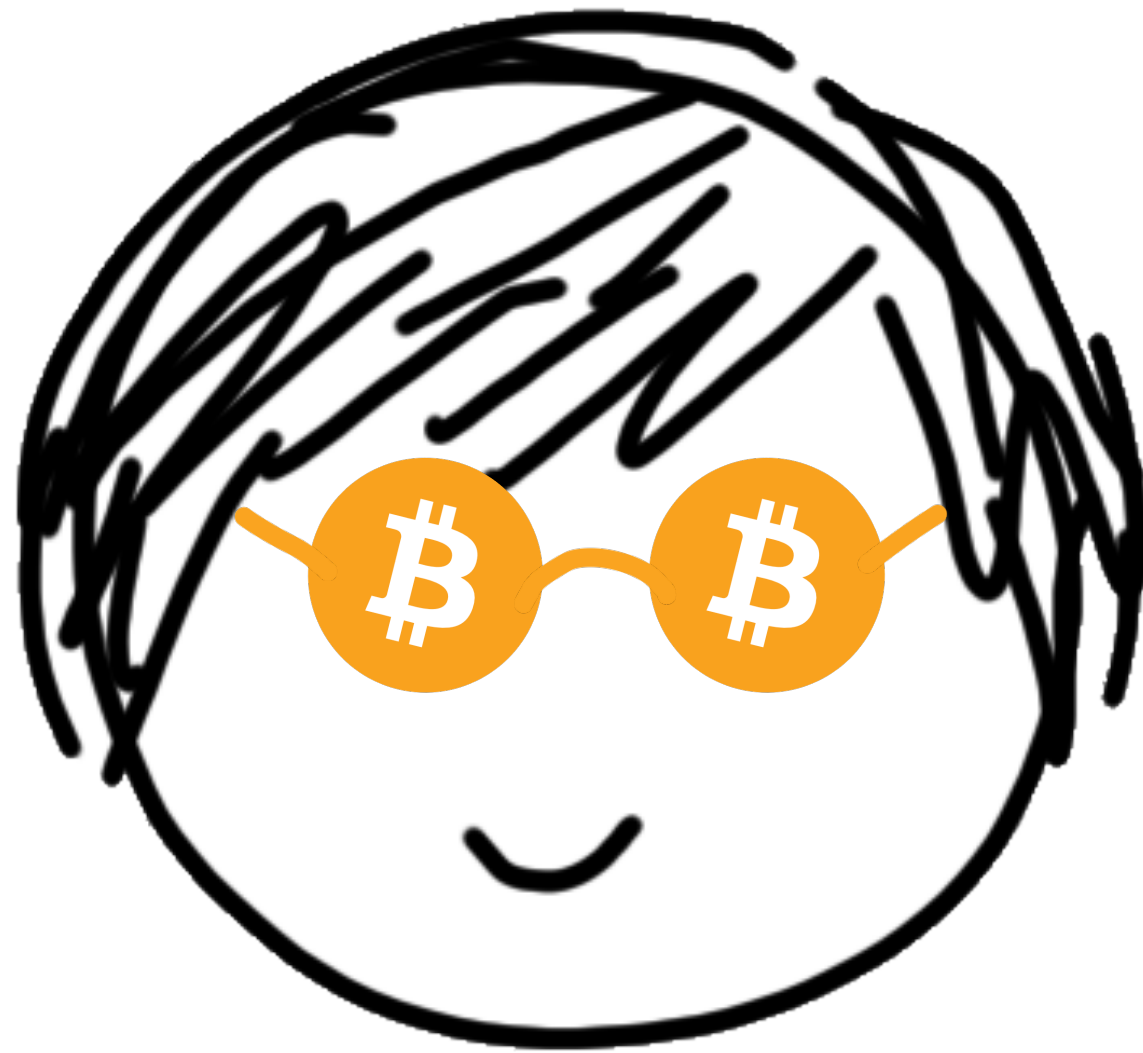
Threshold Signing



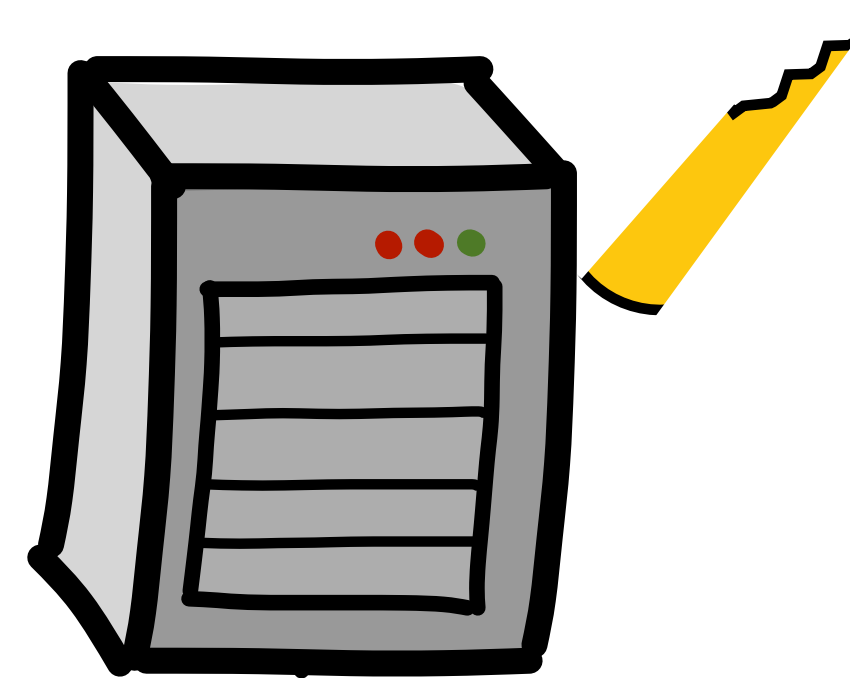
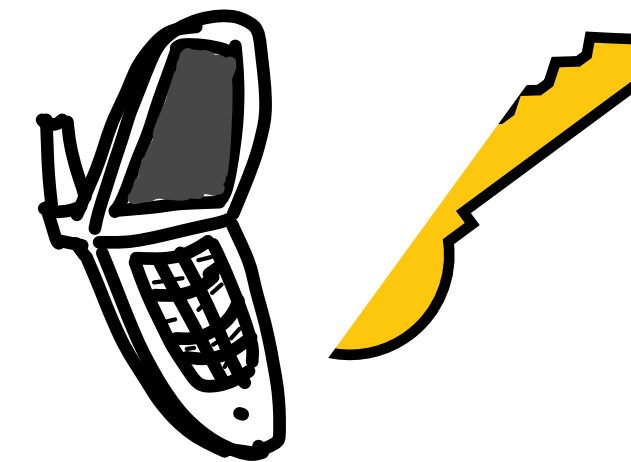
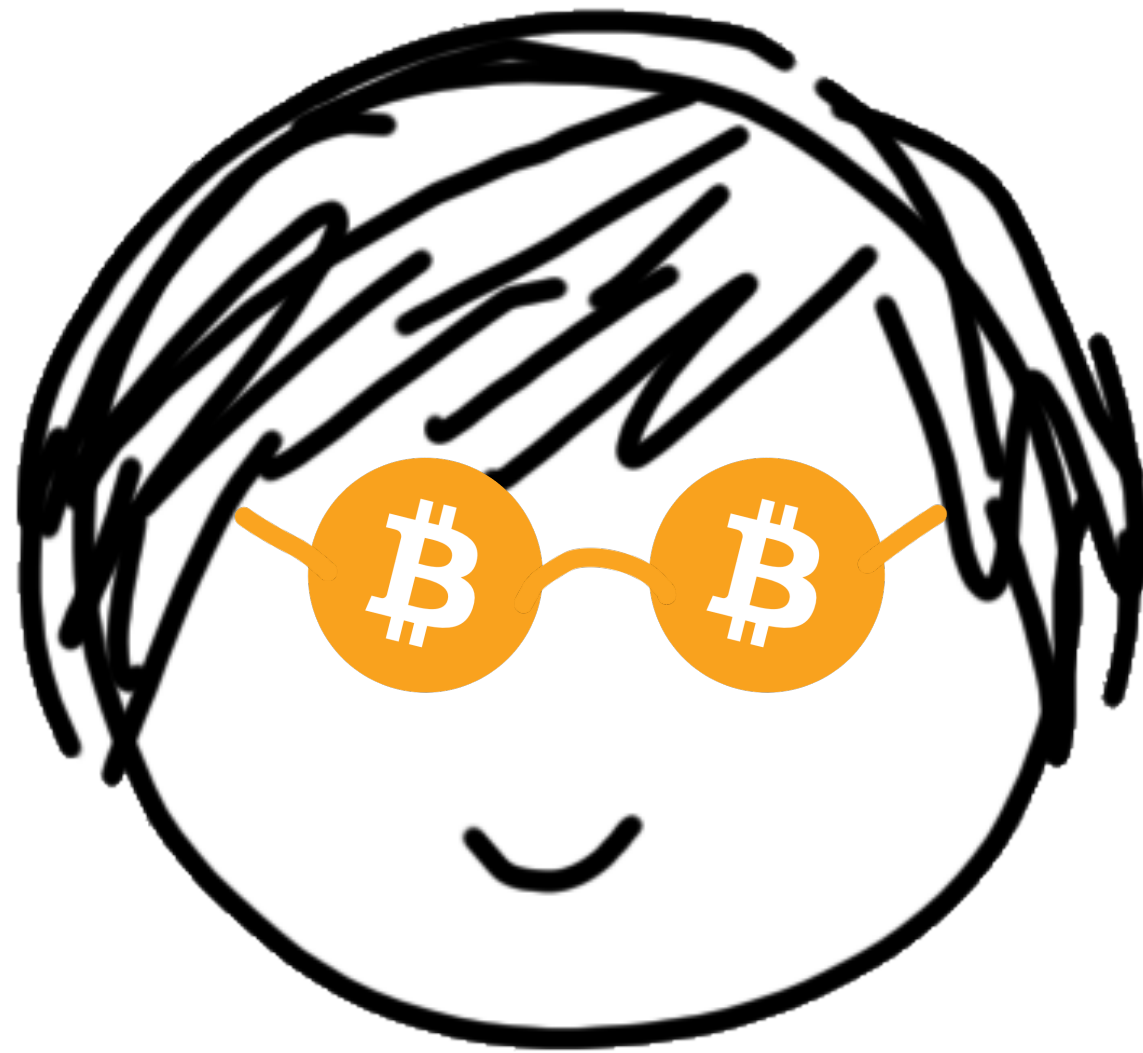
Threshold Signing



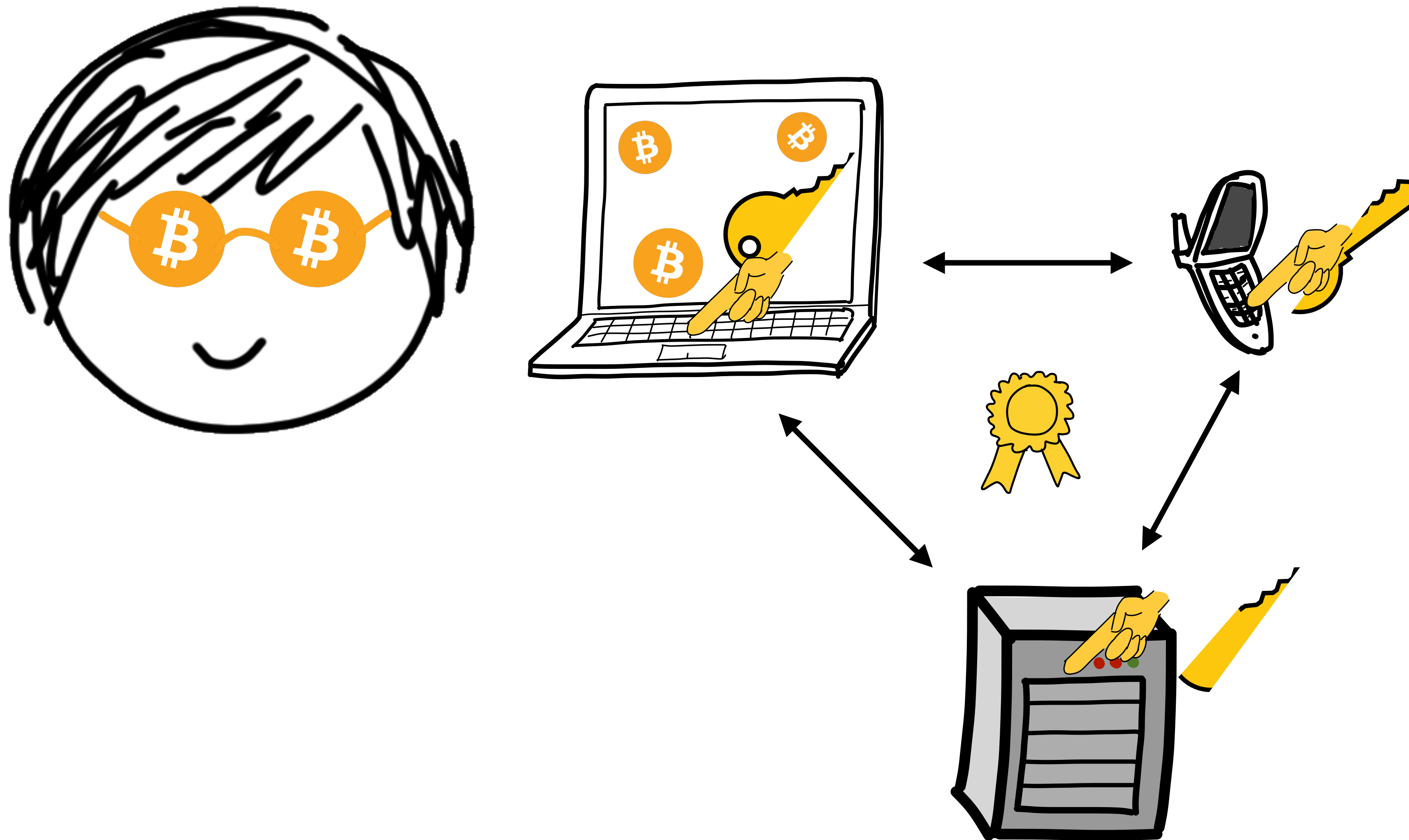
Threshold Signing



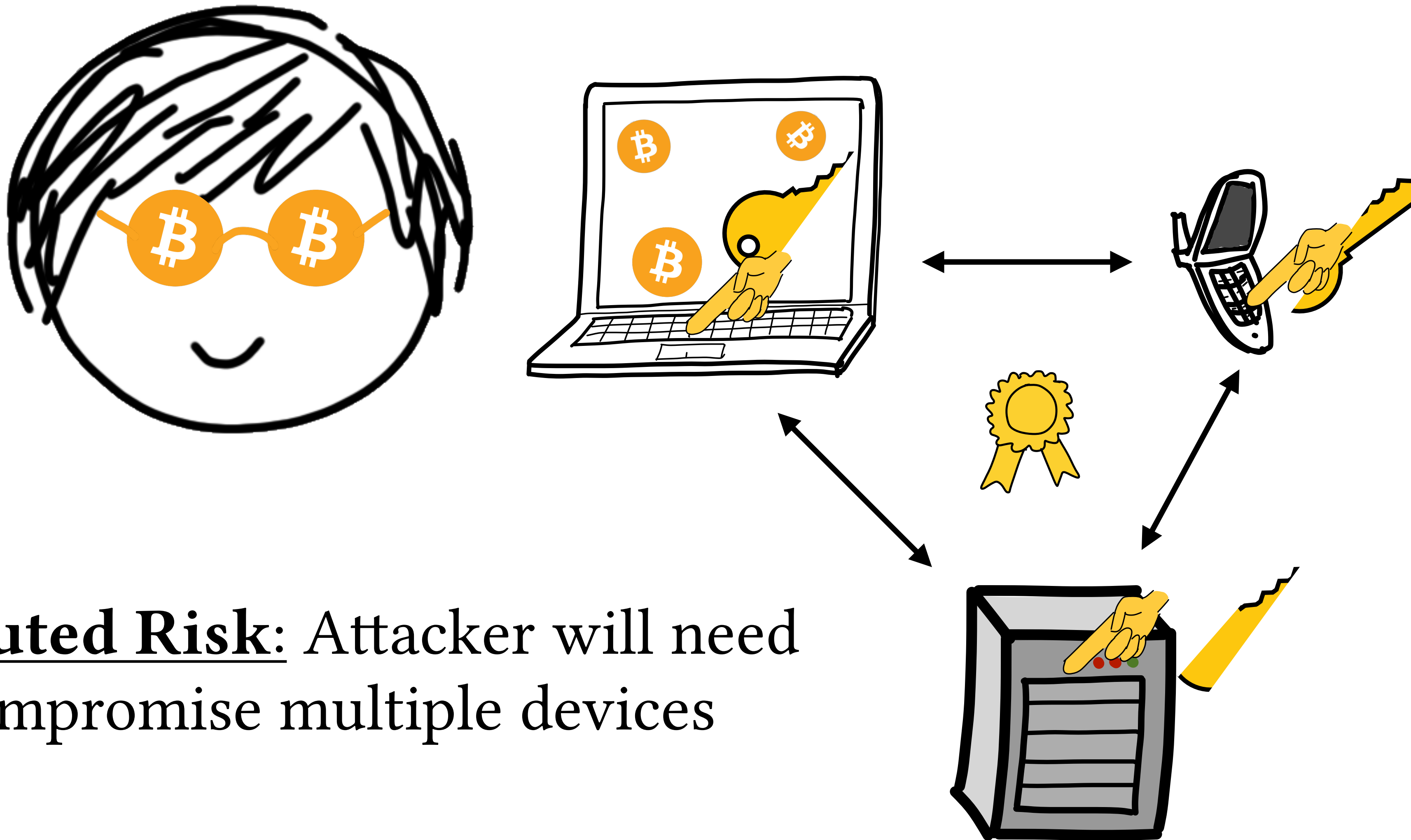
Threshold Signing



Threshold Signing



Threshold Signing



Distributed Risk: Attacker will need to compromise multiple devices

Intro

How to distribute ECDSA

Tradeoffs

MP-Schnorr
is easy

but not
ECDSA

Evolution of
Techniques

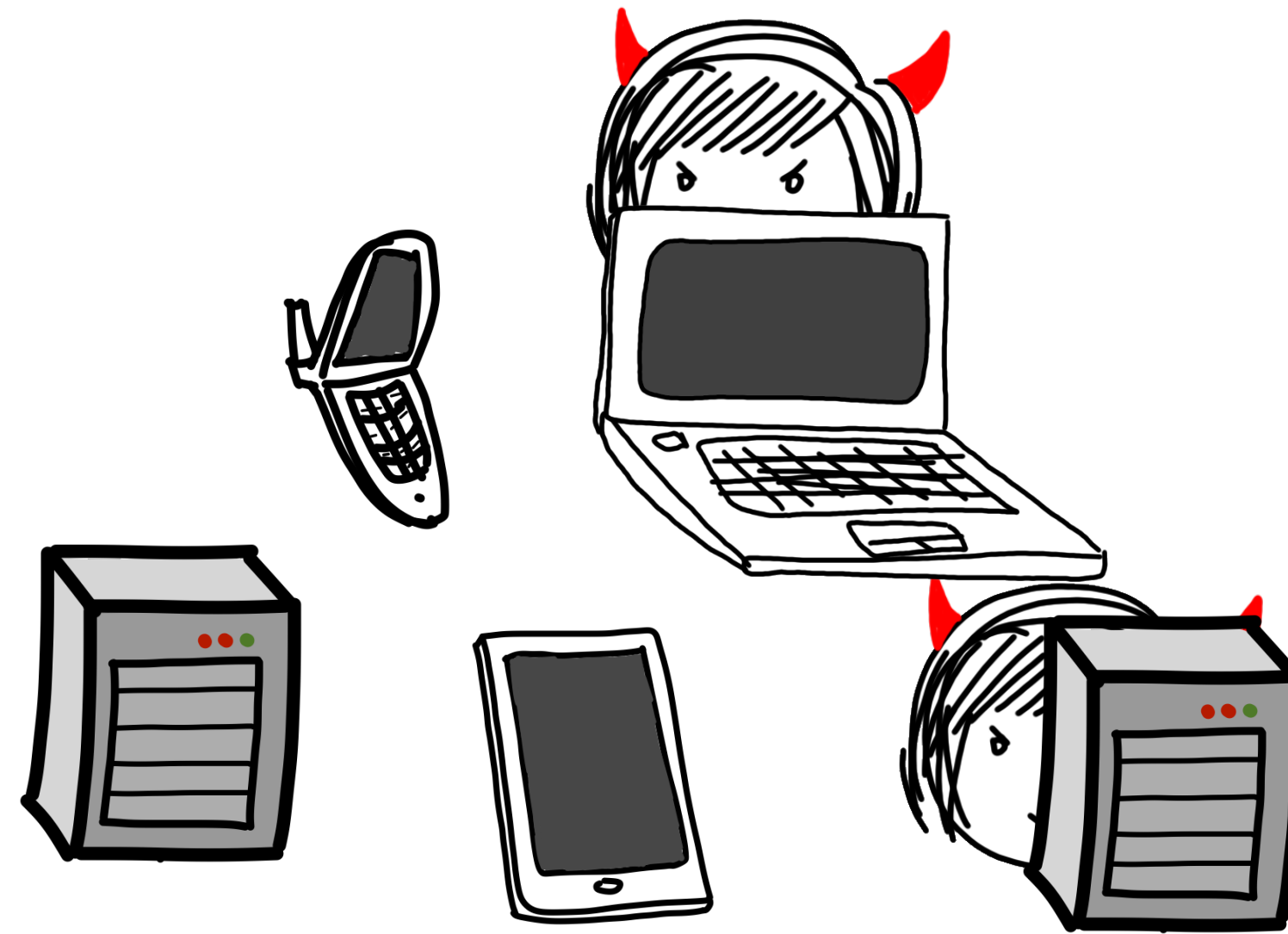
ECDSA
Tuples

New protocol:
Simple consistency
check

OT vs
AHE

Adversary Model

- Corruption threshold



Dishonest majority
(only one device uncompromised)

Adversary Model

- Corruption threshold



Dishonest majority
(only one device uncompromised)

Adversary Model

- Corruption threshold



Dishonest majority
(only one device uncompromised)

-
- Adversarial behaviour

Adversary Model

- Corruption threshold



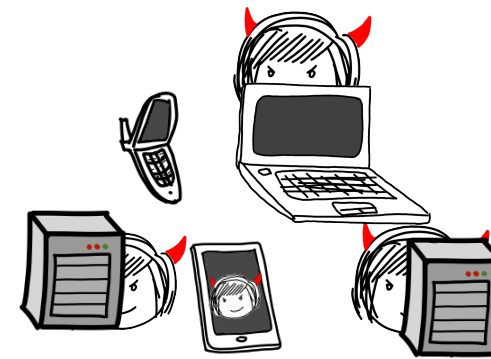
Dishonest majority
(only one device uncompromised)

-
- Adversarial behaviour



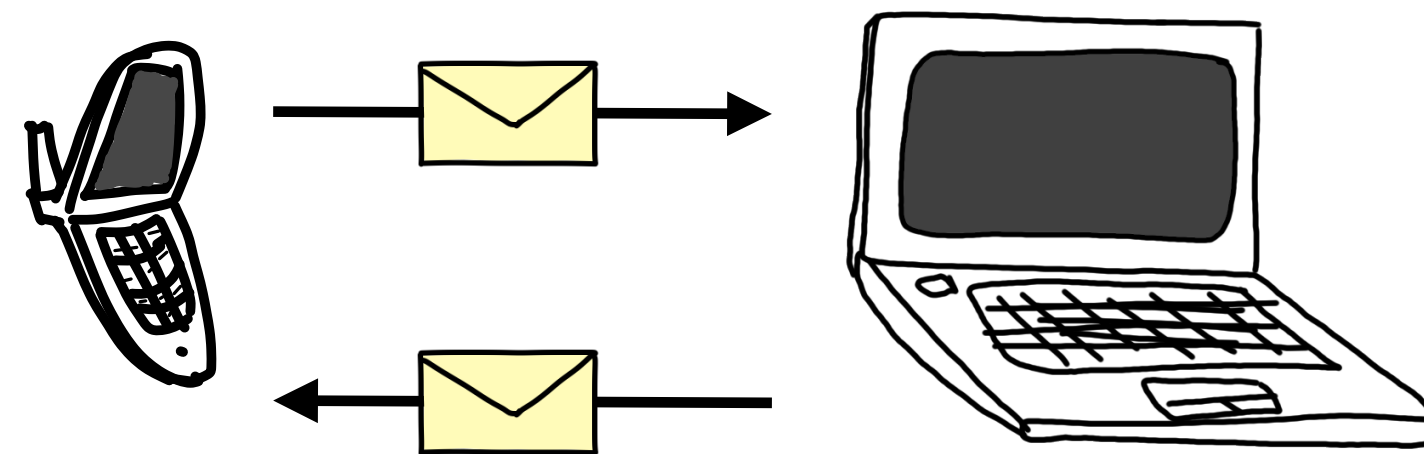
Adversary Model

- Corruption threshold



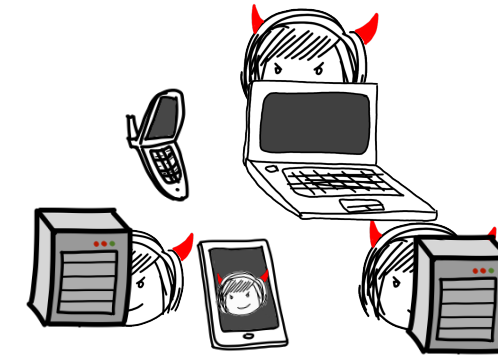
Dishonest majority
(only one device uncompromised)

-
- Adversarial behaviour



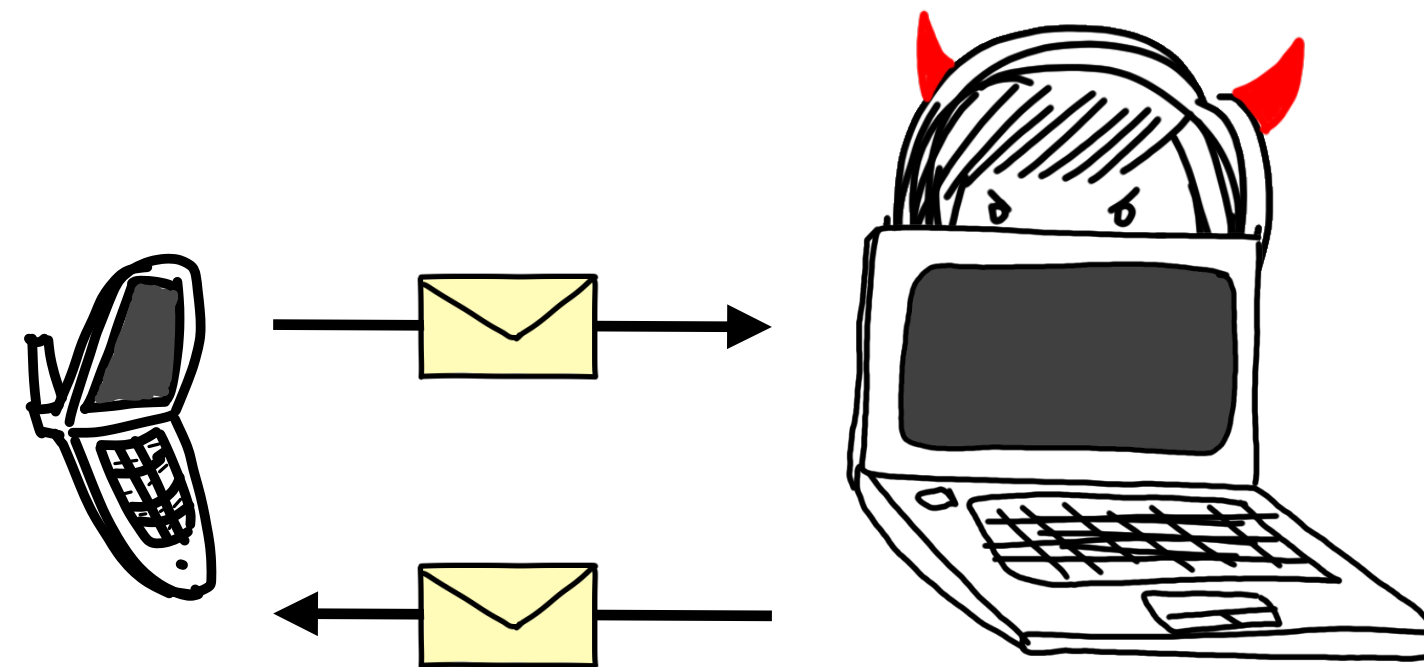
Adversary Model

- Corruption threshold



Dishonest majority
(only one device uncompromised)

-
- Adversarial behaviour



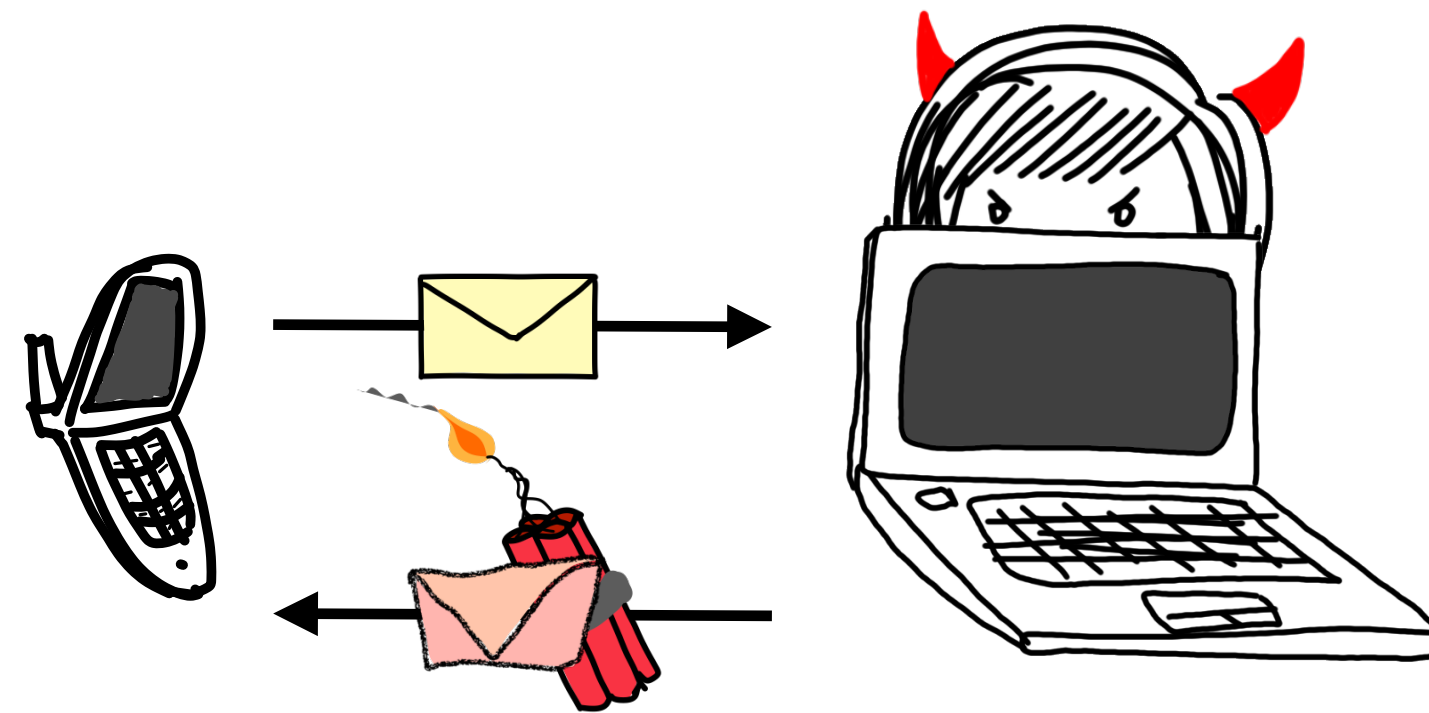
Adversary Model

- Corruption threshold



Dishonest majority
(only one device uncompromised)

-
- Adversarial behaviour



Malicious
(arbitrary deviations from protocol)

Concrete Example: Schnorr Signatures

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]
- Tools:

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]
- Tools:
 - Hash function - $H : \{0,1\}^* \rightarrow \mathbb{Z}_q$

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]
- Tools:
 - Hash function - $H : \{0,1\}^* \rightarrow \mathbb{Z}_q$
 - Group - $(\mathbb{G}, G, q, +)$

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]
- Tools:
 - Hash function - $H : \{0,1\}^* \rightarrow \mathbb{Z}_q$
 - Group - $(\mathbb{G}, G, q, +)$
Group elements

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]
- Tools:
 - Hash function - $H : \{0,1\}^* \rightarrow \mathbb{Z}_q$
 - Group - $(\mathbb{G}, G, q, +)$

Group elements

Points on an
Elliptic Curve

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]
- Tools:
 - Hash function - $H : \{0,1\}^* \rightarrow \mathbb{Z}_q$
 - Group - $(\mathbb{G}, G, q, +)$

Group elements

Points on an
Elliptic Curve

Generator

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]
- Tools:

- Hash function - $H : \{0,1\}^* \rightarrow \mathbb{Z}_q$

- Group - $(\mathbb{G}, G, q, +)$

Group elements

Points on an
Elliptic Curve

Generator

(Large prime) order

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]
- Tools:

- Hash function - $H : \{0,1\}^* \rightarrow \mathbb{Z}_q$

- Group - $(\mathbb{G}, G, q, +)$

Group elements

Points on an
Elliptic Curve

Generator

(Large prime) order

$\approx 2^{256}$

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]
- Tools:

- Hash function - $H : \{0,1\}^* \rightarrow \mathbb{Z}_q$

- Group - $(\mathbb{G}, G, q, +)$

Group elements

Addition law

Points on an
Elliptic Curve

Generator

(Large prime) order

$\approx 2^{256}$

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]

- Tools:

- Hash function - $H : \{0,1\}^* \rightarrow \mathbb{Z}_q$

- Group - $(\mathbb{G}, G, q, +)$

Group elements

Addition law

Points on an
Elliptic Curve

Generator

(Large prime) order

$\approx 2^{256}$

Sixty Seconds on Cyclic Groups

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]

- Tools:

- Hash function - $H : \{0,1\}^* \rightarrow \mathbb{Z}_q$

- Group - $(\mathbb{G}, G, q, +)$

Group elements

Addition law

Points on an
Elliptic Curve

Generator

(Large prime) order

$\approx 2^{256}$

Sixty Seconds on Cyclic Groups

If $X, Y \in \mathbb{G}$ then $X + Y = Z \in \mathbb{G}$

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]

- Tools:

- Hash function - $H : \{0,1\}^* \rightarrow \mathbb{Z}_q$

- Group - $(\mathbb{G}, G, q, +)$

Group elements

Addition law

Points on an
Elliptic Curve

Generator

(Large prime) order

$\approx 2^{256}$

Sixty Seconds on Cyclic Groups

If $X, Y \in \mathbb{G}$ then $X + Y = Z \in \mathbb{G}$

Any $X \in \mathbb{G}$ can be written as $x \cdot G$
 $x \in \mathbb{Z}_q$ is the *discrete logarithm* of X

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]

- Tools:

- Hash function - $H : \{0,1\}^* \rightarrow \mathbb{Z}_q$

- Group - $(\mathbb{G}, G, q, +)$

Group elements

Addition law

Points on an
Elliptic Curve

Generator

(Large prime) order
 $\approx 2^{256}$

Sixty Seconds on Cyclic Groups

If $X, Y \in \mathbb{G}$ then $X + Y = Z \in \mathbb{G}$

Any $X \in \mathbb{G}$ can be written as $x \cdot G$
 $x \in \mathbb{Z}_q$ is the *discrete logarithm* of X

$$(x + y) \cdot G = x \cdot G + y \cdot G$$

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]

- Tools:

- Hash function - $H : \{0,1\}^* \rightarrow \mathbb{Z}_q$

- Group - $(\mathbb{G}, G, q, +)$

Group elements

Addition law

Points on an
Elliptic Curve

Generator

(Large prime) order
 $\approx 2^{256}$

Sixty Seconds on Cyclic Groups

If $X, Y \in \mathbb{G}$ then $X + Y = Z \in \mathbb{G}$

Any $X \in \mathbb{G}$ can be written as $x \cdot G$
 $x \in \mathbb{Z}_q$ is the *discrete logarithm* of X

Integer addition mod q

Group addition

$$(x \boxed{+} y) \cdot G = x \cdot G \boxed{+} y \cdot G$$

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]

- Tools:

- Hash function - $H : \{0,1\}^* \rightarrow \mathbb{Z}_q$

- Group - $(\mathbb{G}, G, q, +)$

Group elements

Addition law

Points on an
Elliptic Curve

Generator

(Large prime) order
 $\approx 2^{256}$

Sixty Seconds on Cyclic Groups

If $X, Y \in \mathbb{G}$ then $X + Y = Z \in \mathbb{G}$

Any $X \in \mathbb{G}$ can be written as $x \cdot G$
 $x \in \mathbb{Z}_q$ is the *discrete logarithm* of X

Integer addition mod q

Group addition

$$(x \boxed{+} y) \cdot G = x \cdot G \boxed{+} y \cdot G$$

Discrete Logarithm Problem: Given random $X \in \mathbb{G}$, find its discrete logarithm

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]

- Tools:

- Hash function - $H : \{0,1\}^* \rightarrow \mathbb{Z}_q$

- Group - $(\mathbb{G}, G, q, +)$

Group elements

Addition law

Points on an
Elliptic Curve

Generator

(Large prime) order
 $\approx 2^{256}$

Sixty Seconds on Cyclic Groups

If $X, Y \in \mathbb{G}$ then $X + Y = Z \in \mathbb{G}$

Any $X \in \mathbb{G}$ can be written as $x \cdot G$
 $x \in \mathbb{Z}_q$ is the *discrete logarithm* of X

Integer addition mod q Group addition
 $(x \boxed{+} y) \cdot G = x \cdot G \boxed{+} y \cdot G$

Discrete Logarithm Problem: Given random $X \in \mathbb{G}$, find its discrete logarithm

For certain elliptic curves, best known algorithms for DLP run in time $\Theta(\sqrt{q})$

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]

- Tools:

- Hash function - $H : \{0,1\}^* \rightarrow \mathbb{Z}_q$

- Group - $(\mathbb{G}, G, q, +)$

Group elements

Addition law

Points on an
Elliptic Curve

Generator

(Large prime) order

$\approx 2^{256}$

Sixty Seconds on Cyclic Groups

If $X, Y \in \mathbb{G}$ then $X + Y = Z \in \mathbb{G}$

Any $X \in \mathbb{G}$ can be written as $x \cdot G$
 $x \in \mathbb{Z}_q$ is the *discrete logarithm* of X

Integer addition mod q

Group addition

$$(x \boxed{+} y) \cdot G = x \cdot G \boxed{+} y \cdot G$$

Very informally:

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]

- Tools:

- Hash function - $H : \{0,1\}^* \rightarrow \mathbb{Z}_q$

- Group - $(\mathbb{G}, G, q, +)$

Group elements

Addition law

Points on an
Elliptic Curve

Generator

(Large prime) order

$\approx 2^{256}$

Sixty Seconds on Cyclic Groups

If $X, Y \in \mathbb{G}$ then $X + Y = Z \in \mathbb{G}$

Any $X \in \mathbb{G}$ can be written as $x \cdot G$
 $x \in \mathbb{Z}_q$ is the *discrete logarithm* of X

Integer addition mod q

Group addition

$$(x \boxed{+} y) \cdot G = x \cdot G \boxed{+} y \cdot G$$

Very informally:

$$x \rightarrow X \text{ EASY}$$

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]

- Tools:

- Hash function - $H : \{0,1\}^* \rightarrow \mathbb{Z}_q$

- Group - $(\mathbb{G}, G, q, +)$

Group elements

Addition law

Points on an
Elliptic Curve

Generator

(Large prime) order
 $\approx 2^{256}$

Sixty Seconds on Cyclic Groups

If $X, Y \in \mathbb{G}$ then $X + Y = Z \in \mathbb{G}$

Any $X \in \mathbb{G}$ can be written as $x \cdot G$
 $x \in \mathbb{Z}_q$ is the *discrete logarithm* of X

Integer addition mod q

Group addition

$$(x \boxed{+} y) \cdot G = x \cdot G \boxed{+} y \cdot G$$

Very informally:

$$x \rightarrow X \text{ EASY}$$

$$X \rightarrow x \text{ HARD}$$

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]

- Tools:

- Hash function - $H : \{0,1\}^* \rightarrow \mathbb{Z}_q$

- Group - $(\mathbb{G}, G, q, +)$

Group elements

Addition law

Points on an
Elliptic Curve

Generator

(Large prime) order
 $\approx 2^{256}$

Sixty Seconds on Cyclic Groups

If $X, Y \in \mathbb{G}$ then $X + Y = Z \in \mathbb{G}$

Any $X \in \mathbb{G}$ can be written as $x \cdot G$
 $x \in \mathbb{Z}_q$ is the *discrete logarithm* of X

Integer addition mod q

Group addition

$$(x \boxed{+} y) \cdot G = x \cdot G \boxed{+} y \cdot G$$

Very informally:

$$x \rightarrow X \text{ EASY}$$

$30\mu s$

$$X \rightarrow x \text{ HARD}$$

Concrete Example: Schnorr Signatures

- Elegant signature scheme based on the Discrete Logarithm problem [Schnorr 89]

- Tools:

- Hash function - $H : \{0,1\}^* \rightarrow \mathbb{Z}_q$

- Group - $(\mathbb{G}, G, q, +)$

Group elements

Addition law

Points on an
Elliptic Curve

Generator

(Large prime) order
 $\approx 2^{256}$

Sixty Seconds on Cyclic Groups

If $X, Y \in \mathbb{G}$ then $X + Y = Z \in \mathbb{G}$

Any $X \in \mathbb{G}$ can be written as $x \cdot G$
 $x \in \mathbb{Z}_q$ is the *discrete logarithm* of X

Integer addition mod q

Group addition

$$(x \boxed{+} y) \cdot G = x \cdot G \boxed{+} y \cdot G$$

Very informally:

$$x \rightarrow X \text{ EASY}$$

$30\mu s$

$$X \rightarrow x \text{ HARD}$$

Many billion billions of years

Schnorr Key Generation

SchnorrKeyGen($\mathbb{G}, G, q$) :

$$sk \leftarrow \mathbb{Z}_q$$

$$PK = sk \cdot G$$

output (sk, PK)

secret key: kept private

Public Key: exposed to the outside world

Schnorr Signing

SchnorrKeyGen($\mathbb{G}, G, q$) :

$$\textcolor{green}{sk} \leftarrow \mathbb{Z}_q$$

$$\textcolor{green}{PK} = \textcolor{green}{sk} \cdot G$$

output ($\textcolor{green}{sk}, \textcolor{green}{PK}$)

⋮

SchnorrSign($\textcolor{green}{sk}, m$) :

Schnorr Signing

SchnorrKeyGen($\mathbb{G}, G, q$) :

$$\textcolor{green}{sk} \leftarrow \mathbb{Z}_q$$

$$\textcolor{green}{PK} = \textcolor{green}{sk} \cdot G$$

output ($\textcolor{green}{sk}, \textcolor{green}{PK}$)

⋮

SchnorrSign($\textcolor{green}{sk}, m$) :

$$\textcolor{green}{k} \leftarrow \mathbb{Z}_q$$

Schnorr Signing

SchnorrKeyGen($\mathbb{G}, G, q$) :

$$\text{sk} \leftarrow \mathbb{Z}_q$$

$$\text{PK} = \text{sk} \cdot G$$

output (sk, PK)

⋮

SchnorrSign(sk, m) :

$$k \leftarrow \mathbb{Z}_q$$

$$R = k \cdot G$$

Schnorr Signing

SchnorrKeyGen($\mathbb{G}, G, q$) :

$$\text{sk} \leftarrow \mathbb{Z}_q$$

$$\text{PK} = \text{sk} \cdot G$$

output (sk, PK)

SchnorrSign(sk, m) :

$$k \leftarrow \mathbb{Z}_q$$

$$R = k \cdot G$$

NONCE
One-time use
value

Schnorr Signing

SchnorrKeyGen($\mathbb{G}, G, q$) :

$$\text{sk} \leftarrow \mathbb{Z}_q$$

$$\text{PK} = \text{sk} \cdot G$$

output (sk, PK)

SchnorrSign(sk, m) :

$$k \leftarrow \mathbb{Z}_q$$

$$R = k \cdot G$$

$$e = H(R || m)$$

NONCE
One-time use
value

Schnorr Signing

SchnorrKeyGen($\mathbb{G}, G, q$) :

$$\text{sk} \leftarrow \mathbb{Z}_q$$

$$\text{PK} = \text{sk} \cdot G$$

output (sk, PK)

SchnorrSign(sk, m) :

$$k \leftarrow \mathbb{Z}_q$$

$$R = k \cdot G$$

$$e = H(R || m)$$

$$s = k - \text{sk} \cdot e \pmod{q}$$

NONCE
One-time use
value

Schnorr Signing

SchnorrKeyGen($\mathbb{G}, G, q$) :

$$\text{sk} \leftarrow \mathbb{Z}_q$$

$$\text{PK} = \text{sk} \cdot G$$

output (sk, PK)

SchnorrSign(sk, m) :

$$k \leftarrow \mathbb{Z}_q$$

$$R = k \cdot G$$

$$e = H(R || m)$$

$$s = k - \text{sk} \cdot e \pmod{q}$$

$$\sigma = (s, R)$$

output σ

NONCE
One-time use
value

Schnorr Signing

SchnorrKeyGen($\mathbb{G}, G, q$) :

$$\text{sk} \leftarrow \mathbb{Z}_q$$

$$\text{PK} = \text{sk} \cdot G$$

output (sk, PK)

SchnorrSign(sk, m) :

$$k \leftarrow \mathbb{Z}_q$$

$$R = k \cdot G$$

$$e = H(R || m)$$

$$s = k - \text{sk} \cdot e \pmod{q}$$

$$\sigma = (s, R)$$

output σ

NONCE
One-time use
value

Verifying a signature: $s \cdot G \stackrel{?}{=} R - e \cdot \text{PK}$

Schnorr Signing

SchnorrKeyGen($\mathbb{G}, G, q$) :

$$\text{sk} \leftarrow \mathbb{Z}_q$$

$$\text{PK} = \text{sk} \cdot G$$

output (sk, PK)

SchnorrSign(sk, m) :

$$k \leftarrow \mathbb{Z}_q$$

$$R = k \cdot G$$

$$e = H(R || m)$$

$$s = k - \text{sk} \cdot e \pmod{q}$$

$$\sigma = (s, R)$$

output σ

NONCE
One-time use
value

Verifying a signature:

$$s \cdot G \stackrel{?}{=} \underbrace{R}_{k \cdot G} - e \cdot \underbrace{\text{PK}}_{\text{sk} \cdot G}$$

Schnorr Signing

SchnorrKeyGen($\mathbb{G}, G, q$) :

$$\text{sk} \leftarrow \mathbb{Z}_q$$

$$\text{PK} = \text{sk} \cdot G$$

output (sk, PK)

SchnorrSign(sk, m) :

$$k \leftarrow \mathbb{Z}_q$$

$$R = k \cdot G$$

$$e = H(R || m)$$

$$s = k - \text{sk} \cdot e \pmod{q}$$

$$\sigma = (s, R)$$

output σ

NONCE
One-time use
value

Verifying a signature:

$$s \cdot G \stackrel{?}{=} R - e \cdot \text{PK}$$

$$k \cdot G$$

$$\text{sk} \cdot G$$

$$s \cdot G \stackrel{?}{=} (k - e \cdot \text{sk}) \cdot G$$

Secret Sharing

- $[x]$ denotes that a value $x \in \mathbb{Z}_q$ is “secret-shared” across devices
- We will only use “linear” secret sharing schemes
 $a[x] + b[y] = [ax + by]$

Additive Secret Sharing



Additive Secret Sharing

$$x \in \mathbb{Z}_q$$



Additive Secret Sharing



$$x \in \mathbb{Z}_q$$
$$x_A + x_B = x$$



Additive Secret Sharing



x_A

$$x \in \mathbb{Z}_q$$
$$x_A + x_B = x$$



x_B

Additive Secret Sharing



x_A

$$x \in \mathbb{Z}_q$$
$$x_A + x_B = x$$

$[x]$



x_B

Additive Secret Sharing



x_A

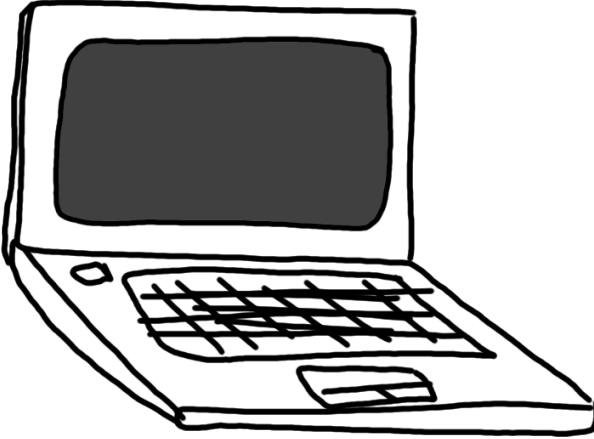
$$x \in \mathbb{Z}_q$$
$$x_A + x_B = x$$

$[x]$



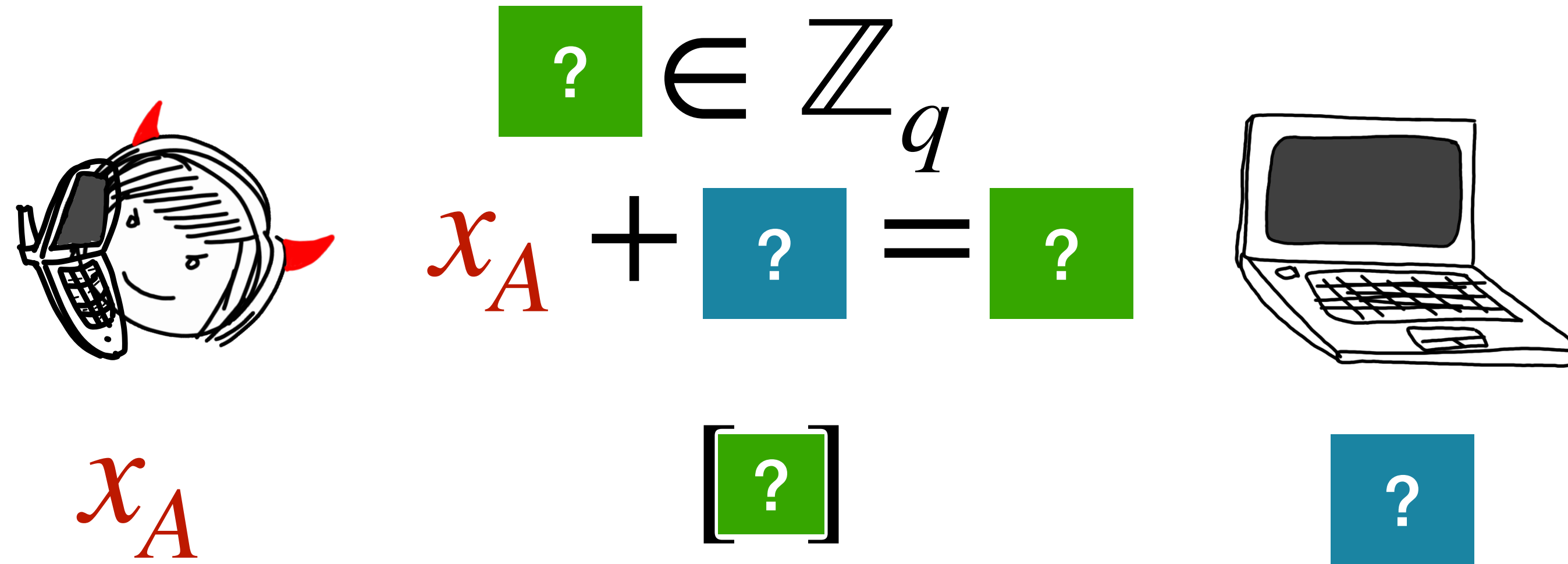
x_B

Additive Secret Sharing


$$x \in \mathbb{Z}_q$$
$$x_A + x_B = x$$

$$[x]$$

x_A x_B

Additive Secret Sharing



Additive Secret Sharing



x_A

$$x \in \mathbb{Z}_q$$
$$x_A + x_B = x$$



x_B

$[x]$

$[y]$

$$y_A + y_B = y$$

Additive Secret Sharing



x_A

y_A

$$x \in \mathbb{Z}_q$$
$$x_A + x_B = x$$

$[x]$

$[y]$



x_B

y_B

Additive Secret Sharing

 x_A y_A

$$x \in \mathbb{Z}_q$$

$$x_A + x_B = x$$

 $[x]$ $[y]$

$$[z = cx + y]$$

 x_B y_B

Additive Secret Sharing

$$x \in \mathbb{Z}_q$$



$$x_A + x_B = x$$



$$x_A$$

$$[x]$$

$$x_B$$

$$y_A$$

$$[y]$$

$$y_B$$

$$z_A = cx_A + y_A$$

$$[z = cx + y]$$

$$z_B = cx_B + y_B$$

Distributing Schnorr w. Additive Secret Sharing



x_A

y_A

$$z_A = cx_A + y_A$$

$$x \in \mathbb{Z}_q$$

$$x_A + x_B = x$$

$[x]$

$[y]$

$$[z = cx + y]$$



x_B

y_B

$$z_B = cx_B + y_B$$

Distributing Schnorr w. Additive Secret Sharing


$$sk \in \mathbb{Z}_q$$
$$sk_A + sk_B = sk$$


sk_A

$[sk]$

sk_B

k_A

$[k]$

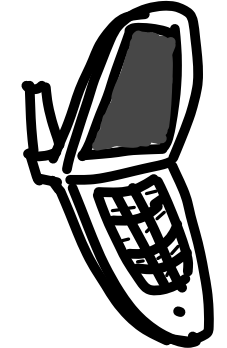
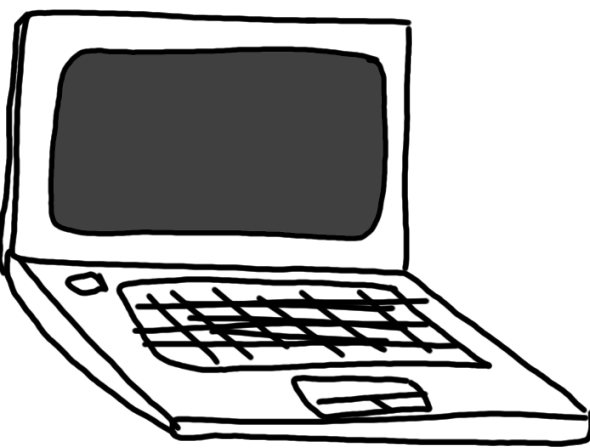
k_B

$$s_A = esk_A + k_A$$

$$[s = esk + k]$$

$$s_B = esk_B + k_B$$

Distributing Schnorr w. Additive Secret Sharing


$$sk \in \mathbb{Z}_q$$
$$sk_A + sk_B = sk$$


sk_A

$[sk]$

sk_B

k_A

$[k]$

k_B

$$R = k \cdot G$$

$$e = H(m, R)$$

$$s_A = e sk_A + k_A$$

$$[s = e sk + k]$$

$$s_B = e sk_B + k_B$$

3 Round Schnorr Signing

Folklore, [Lindell 22]

Input : $\text{pk} = [\text{sk}] \cdot G$, $[\text{sk}]$, $[k]$

Round 1

Establish $R = [k] \cdot G$

Exchange Commit ($R_i = [k]_i \cdot G$)

Round 2

Release R_i , set $R = \sum_i R_i$

Round 3

Reveal $s = [\text{sk}] \cdot H(m, R) + [k]$

Output (R, s)

(Threshold) Schnorr in Practice?

- Schnorr signatures are **old** (well-studied), **compact**, **fast**, and **easy to distribute with MPC (i.e. thresholdize)**
- However it was patented—major barrier for internet adoption
- Patent expired recently; adoption is increasing but much of the **internet infrastructure does not support Schnorr**

ECDSA

- Elliptic Curve Digital Signature Algorithm
 - Devised by Scott Vanstone in 1992, standardized by NIST
 - Differs from Schnorr enough so that **patent doesn't apply**
 - **Widespread adoption** across the internet
- ... but **MPC-unfriendly**

Threshold ECDSA: Challenges

SchnorrSign(**sk**, m) :

$$k \leftarrow \mathbb{Z}_q$$

$$R = k \cdot G$$

$$e = H(R||m)$$

$$s = k - \text{sk} \cdot e$$

$$\sigma = (s, R)$$

output σ

⋮

ECDSASign(**sk**, m) :

$$k \leftarrow \mathbb{Z}_q$$

$$R = k \cdot G$$

$$e = H(m)$$

Threshold ECDSA: Challenges

SchnorrSign(**sk**, m) :

$$k \leftarrow \mathbb{Z}_q$$

$$R = k \cdot G$$

$$e = H(R||m)$$

$$s = k - \text{sk} \cdot e$$

$$\sigma = (s, R)$$

output σ

⋮

ECDSASign(**sk**, m) :

$$k \leftarrow \mathbb{Z}_q$$

$$R = k \cdot G$$

$$e = H(m)$$

Standard 2
round sampling

Threshold ECDSA: Challenges

SchnorrSign(**sk**, m) :

$$k \leftarrow \mathbb{Z}_q$$

$$R = k \cdot G$$

$$e = H(R||m)$$

$$s = k - \mathbf{sk} \cdot e$$

$$\sigma = (s, R)$$

output σ

ECDSASign(**sk**, m) :

$$k \leftarrow \mathbb{Z}_q$$

$$R = k \cdot G$$

$$e = H(m)$$

$$s = \frac{e + \mathbf{sk} \cdot r_x}{k}$$

output $\sigma = (s, R)$

Standard 2
round sampling

Threshold ECDSA: Challenges

ECDSASign($\mathbf{sk}, m$) :

$$k \leftarrow \mathbb{Z}_q$$

$$R = k \cdot G$$

$$e = H(m)$$

$$s = \frac{e + \mathbf{sk} \cdot r_x}{k}$$

output $\sigma = (s, R)$

Threshold ECDSA: Challenges

ECDSASign(sk, m) :

$$k \leftarrow \mathbb{Z}_q$$

$$R = k \cdot G$$

$$e = H(m)$$

$$s = \frac{e + \text{sk} \cdot r_x}{k}$$

output $\sigma = (s, R)$

Multiplication of
secret values

Threshold ECDSA: Challenges

ECDSASign(sk, m) :

$$k \leftarrow \mathbb{Z}_q$$

$$R = k \cdot G$$

$$e = H(m)$$

Multiplication of
secret values

$$s = \frac{e + \text{sk} \cdot r_x}{k}$$

output $\sigma = (s, R)$

Division (Modular inverse)

Threshold ECDSA: Challenges

ECDSASign($\mathbf{sk}, m$) :

$$k \leftarrow \mathbb{Z}_q$$

$$R = k \cdot G$$

$$e = H(m)$$

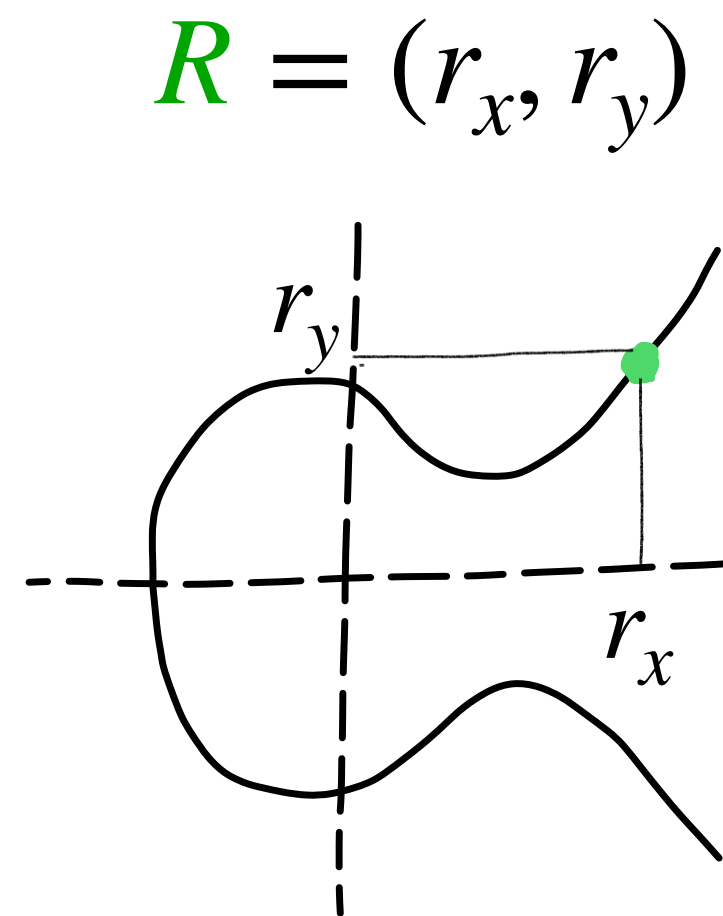
Multiplication of
secret values

$$s = \frac{e + \mathbf{sk} \cdot r_x}{k}$$

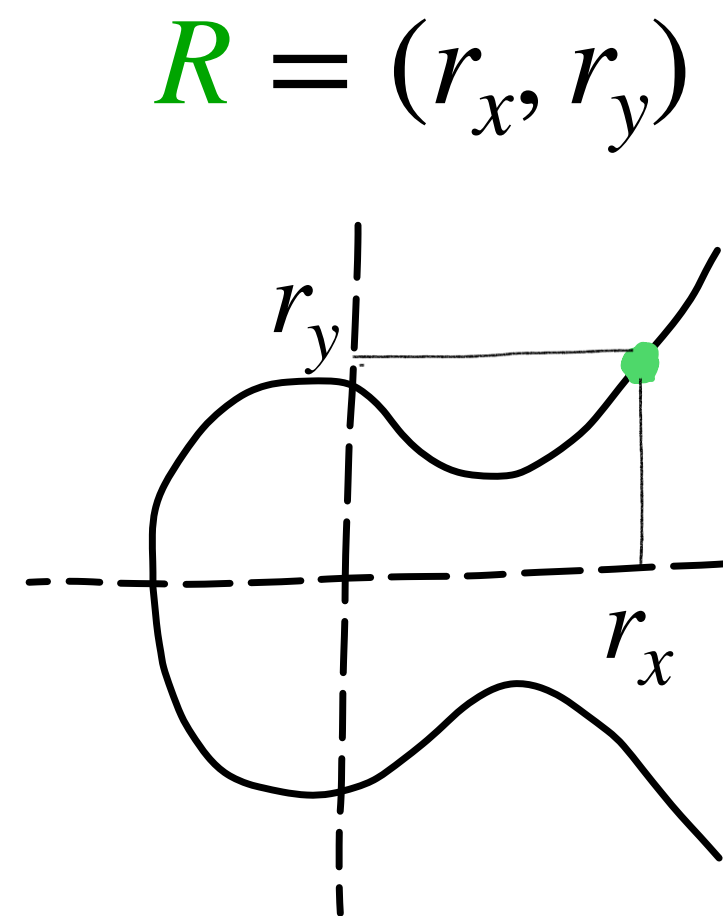
x-coordinate of R (not secret)

output $\sigma = (s, R)$

Division (Modular inverse)



Threshold ECDSA: Challenges



ECDSASign(sk, m) :

$$k \leftarrow \mathbb{Z}_q$$

$$R = k \cdot G$$

$$e = H(m)$$

$$s = \frac{e + \boxed{sk} \cdot r_x}{\boxed{k}}$$

x-coordinate of R (not secret)

output $\sigma = (s, R)$

There is no one-size-fits-all solution

Multiplication of secret values

Division (Modular inverse)

Secure Two-Party Multiplication

a.k.a. OLE, Mult2Add

Underlies many dishonest
majority MPC protocols



α

2P-MUL

β



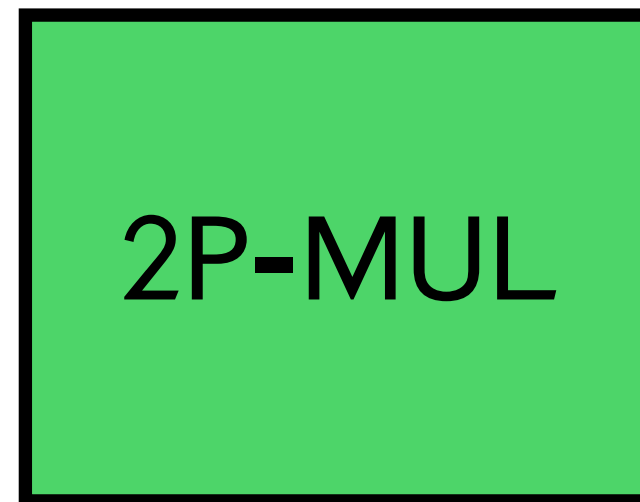
Secure Two-Party Multiplication

a.k.a. OLE, Mult2Add

Underlies many dishonest
majority MPC protocols



α



β



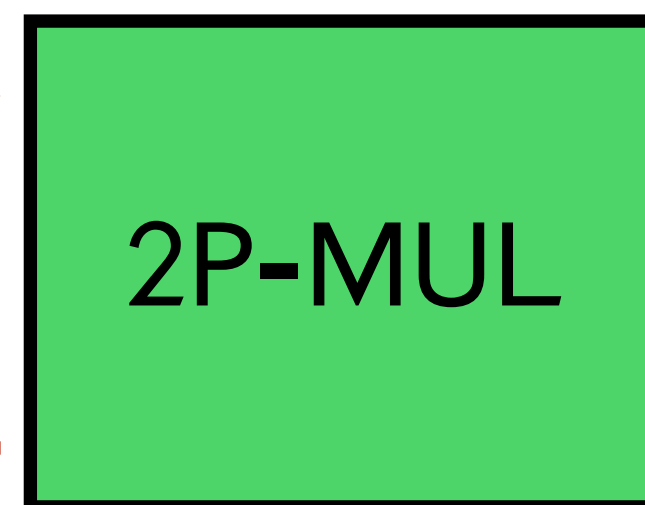
Secure Two-Party Multiplication

a.k.a. OLE, Mult2Add

Underlies many dishonest majority MPC protocols



α



β



c

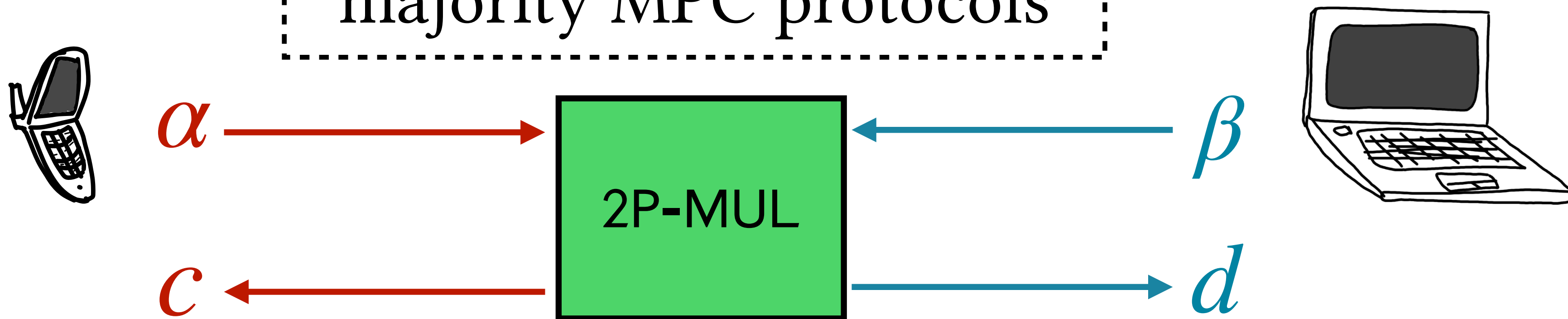


d

Secure Two-Party Multiplication

a.k.a. OLE, Mult2Add

Underlies many dishonest majority MPC protocols

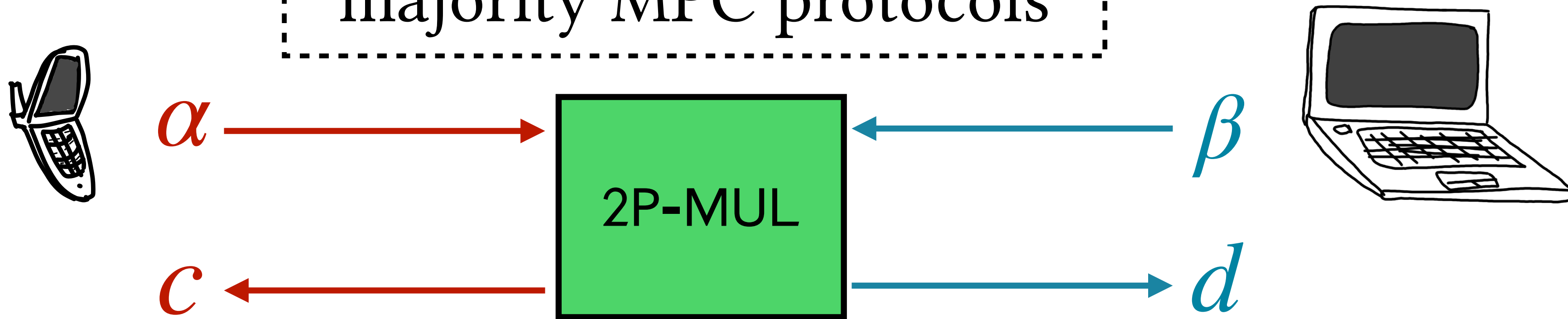


$$c + d = \alpha \cdot \beta$$

Secure Two-Party Multiplication

a.k.a. OLE, Mult2Add

Underlies many dishonest majority MPC protocols



$$c + d = \alpha \cdot \beta$$

Tool to split a product of secret inputs $\alpha\beta$ into additive secrets c, d

Instantiable efficiently from:
OT, Paillier, Class Groups

Threshold ECDSA: State of the Art

- Rough costs with 256-bit curve, for each additional party
(computation aggregated across [Gavenda 21, XAXYC 21, BMP 22]):

Protocol	2P-MUL	Rounds	Bandwidth (KB)	Computation (ms)
[DKLs 19]	OT	$\log(t) + 6$	90	<10
[HLNR 18/23]	OT+	5	40	50—100
[CGGMP 20]	Paillier	4	15	Hundreds
[GG 18]		8	7	Hundreds
[CCLST20, YCX21]	Class Groups	4	4	> 1000

Threshold ECDSA:

Goal

- Rough costs with 256-bit curve, for each additional party
(computation aggregated across [Gavenda 21, XAXYC 21, BMP 22]):

	2P-MUL	Bandwidth (KB)	Computation (ms)
Simple, unified protocol	OT	90	<10
	Paillier	15	Hundreds
	Class Groups	4	> 1000

Threshold ECDSA:

Goal

- Rough costs with 256-bit curve, for each additional party (computation aggregated across [Gavenda 21, XAXYC 21, BMP 22]):

	2P-MUL	Bandwidth (KB)	Computation (ms)
Simple, unified protocol	OT	90	<10
	Paillier	15	Hundreds
	Class Groups	4	> 1000

This work:
3 Round Signing
from
2 round 2P-MUL

Threshold ECDSA:

Goal

- Rough costs with 256-bit curve, for each additional party
(computation aggregated across [Gavenda 21, XAXYC 21, BMP 22]):

	2P-MUL	Bandwidth (KB)	Computation (ms)
Simple, unified protocol	OT	90	<10
	Paillier	15	Hundreds
	Class Groups	4	> 1000

This work:
3 Round Signing
from
2 round 2P-MUL

mild/no overhead

Threshold ECDSA:

Goal

- Rough costs with 256-bit curve, for each additional party
(computation aggregated across [Gavenda 21, XAXYC 21, BMP 22]):

	2P-MUL	Bandwidth (KB)	Computation (ms)
Simple, unified protocol	OT	90	<10
	Paillier	15	Hundreds
	Class Groups	4	> 1000

This work:
3 Round Signing
from
2 round 2P-MUL

mild/no overhead

Insight:
well-chosen
rewriting of ECDSA
+simple consistency
check

Threshold ECDSA:

Goal

- Rough costs with 256-bit curve, for each additional party
(computation aggregated across [Gavenda 21, XAXYC 21, BMP 22]):

	2P-MUL	Bandwidth (KB)	Computation (ms)
Simple, unified protocol	OT	60	<10
	Paillier	15	Hundreds
	Class Groups	4	> 1000

This work:
3 Round Signing
from
2 round 2P-MUL

mild/no overhead

Insight:
well-chosen
rewriting of ECDSA
+simple consistency
check



Intro

How to distribute ECDSA

Tradeoffs

MP-Schnorr
is easy

but not
ECDSA

Evolution of
Techniques

ECDSA
Tuples

Our protocol:
Simple consistency
check

A Brief History of Threshold ECDSA

- “End result” protocols are typically compared by security models, assumptions, concrete efficiency (bandwidth, rounds), and benchmarks.
- This doesn’t tell the full story of *techniques*
⇒ necessary context for “simplicity”
- Qualitative comparison: trace how Threshold ECDSA protocol structure has evolved over time

MPC for ECDSA

- Computing $[k^{-1}]$ given $[k]$ (as used in ECDSA signing) naively as an arithmetic circuit is prohibitively expensive—warrants custom protocols
- Standard recipe in the literature:
 - Rewrite ECDSA signing equation to an “MPC-friendly” equivalent i.e. only additions and multiplications of secret values
 - Cryptographic Machinery for secure multiplication
 - Verify that all operations were performed honestly

Inverted Nonce Rewriting

[Langford 95][Gennaro Jarecki Krawczyk Rabin 96]

ECDSASign($\textcolor{green}{sk}, m$) :

$$\textcolor{green}{[k]} \leftarrow \mathbb{Z}_q$$

$$\textcolor{green}{R} = \textcolor{green}{[k]} \cdot G$$

$$e = H(m)$$

$$\textcolor{green}{s} = \frac{e + \textcolor{green}{[sk]} \cdot r_x}{\textcolor{green}{[k]}}$$

output $\sigma = (\textcolor{green}{s}, \textcolor{green}{R})$

Inverted Nonce Rewriting

[Langford 95][Gennaro Jarecki Krawczyk Rabin 96]

ECDSASign($\textcolor{green}{sk}, m$) :

$$\textcolor{green}{[k]} \leftarrow \mathbb{Z}_q$$

$$\textcolor{green}{R} = \textcolor{green}{[k]} \cdot G$$

$$e = H(m)$$

$$\textcolor{green}{s} = (e + [\textcolor{green}{sk}] \cdot r_x) \textcolor{green}{[k]}$$

$$\text{output } \sigma = (\textcolor{green}{s}, \textcolor{green}{R})$$

Inverted Nonce Rewriting

[Langford 95][Gennaro Jarecki Krawczyk Rabin 96]

ECDSASign($\textcolor{green}{sk}, m$) :

$$\textcolor{green}{[k]} \leftarrow \mathbb{Z}_q$$

$$\textcolor{green}{R} = \textcolor{green}{[k]} \cdot G$$

$$e = H(m)$$

$$\textcolor{green}{s} = (e + [\textcolor{green}{sk}] \cdot r_x) \textcolor{green}{[k]}$$

output $\sigma = (\textcolor{green}{s}, \textcolor{green}{R})$

Inverted Nonce Rewriting

[Langford 95][Gennaro Jarecki Krawczyk Rabin 96]

ECDSA $\text{Sign}(\textcolor{green}{sk}, m) :$

$$[\textcolor{green}{k}] \leftarrow \mathbb{Z}_q$$

$$\textcolor{green}{R} = [\textcolor{green}{k}^{-1}] \cdot G$$

$$e = H(m)$$

Equivalent to ECDSA

But how to securely compute $k^{-1}G$?

$$\textcolor{green}{s} = (e + [\textcolor{green}{sk}] \cdot r_x) [\textcolor{green}{k}]$$

$$\text{output } \sigma = (\textcolor{green}{s}, \textcolor{green}{R})$$

Inverted Nonce Rewriting

[Langford 95][Gennaro Jarecki Krawczyk Rabin 96]

ECDSASign(sk, m) :

$$[k] \leftarrow \mathbb{Z}_q$$

$$[\phi] \leftarrow \mathbb{Z}_q$$

reveal $[\phi] \cdot [k]$

reveal $\Phi = \phi \cdot G$

$$R = (\phi k)^{-1} \cdot \Phi = [k^{-1}] \cdot G$$

$$e = H(m)$$

$$s = (e + [\text{sk}] \cdot r_x) [k]$$

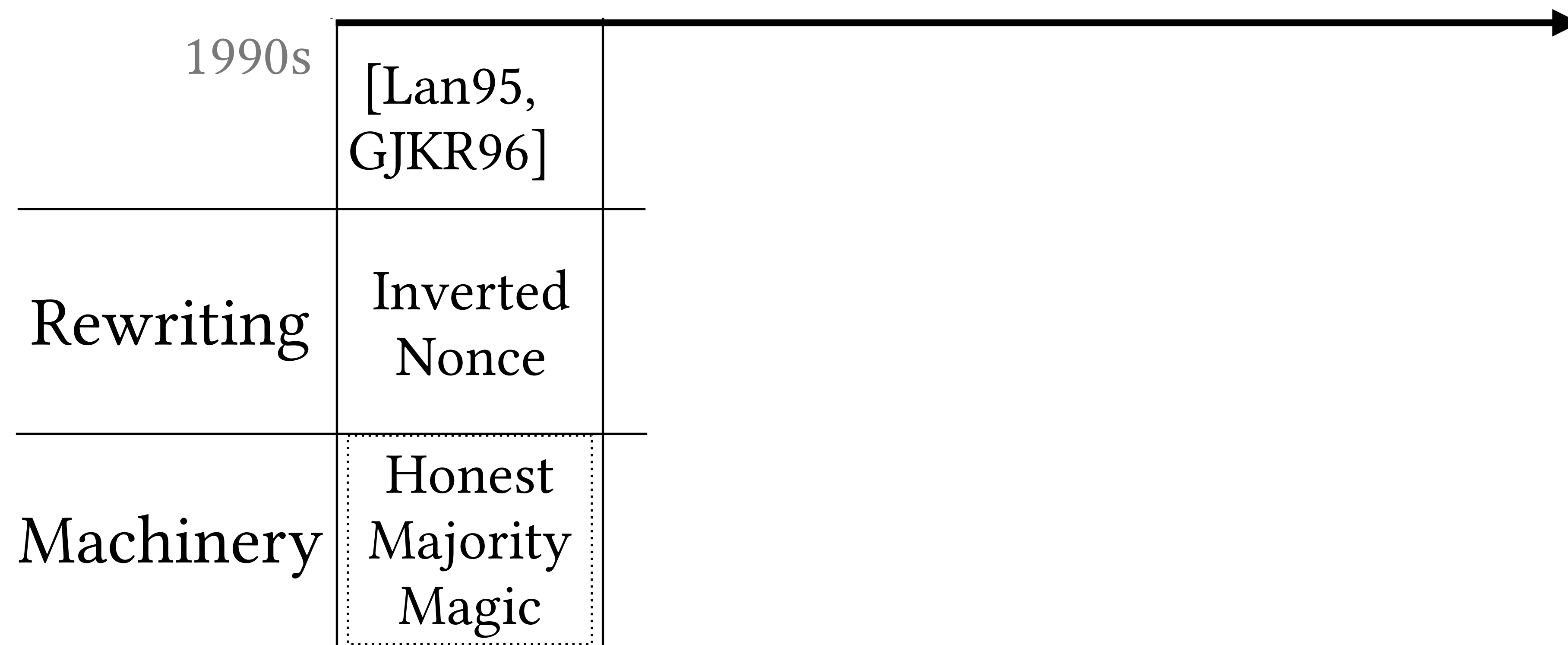
output $\sigma = (s, R)$

First appears in
[Bar-Ilan Beaver 89]

Equivalent to ECDSA

But how to securely compute $k^{-1}G$?

A Brief History of Threshold ECDSA



A Brief History of Threshold ECDSA

1990s	[Lan95, GJKR96]	[MR01]	[GGN16, BGG17]	[Lin17]	[DKLs 18, 19]	2018
Rewriting	Inverted Nonce	Multiplicative				
Machinery	Honest Majority Magic	(Threshold) Paillier		Paillier	OT	

A Brief History of Threshold ECDSA

Now showing

1990s	[Lan95, GJKR96]	[MR01]	[GGN16, BGG17]	[Lin17]	[DKLs 18, 19]	2018
Rewriting	Inverted Nonce	Multiplicative				
Machinery	Honest Majority Magic	(Threshold) Paillier		Paillier	OT	

A Brief History of Threshold ECDSA

Now showing

MacKenzie Reiter 01

1990s	[Lan95, GJKR96]	[MR01]	[GGN16, BGG17]	[Lin17]	[DKLs 18, 19]	2018
Rewriting	Inverted Nonce	Multiplicative: $s = \left(\frac{a}{k}\right) + \left(\frac{bsk}{k}\right)$				
Machinery	Honest Majority Magic	(Threshold) Paillier	Paillier	OT		

A Brief History of Threshold ECDSA

Now showing Gennaro Goldfeder Narayanan 16, Boneh, Gennaro, Goldfeder 17

1990s	[Lan95, GJKR96]	[MR01]	[GGN16, BGG17]	[Lin17]	[DKLs 18, 19]	2018
Rewriting	Inverted Nonce	Multiplicative: $s = \left(\frac{a}{k}\right) + \left(\frac{bsk}{k}\right)$				
Machinery	Honest Majority Magic	(Threshold) Paillier	Paillier	OT		

A Brief History of Threshold ECDSA

Now showing		Lindell 17				
1990s	[Lan95, GJKR96]	[MR01]	[GGN16, BGG17]	[Lin17]	[DKLs 18, 19]	2018
Rewriting	Inverted Nonce	Multiplicative: $s = \left(\frac{a}{k}\right) + \left(\frac{bsk}{k}\right)$				
Machinery	Honest Majority Magic	(Threshold) Paillier		Paillier	OT	

A Brief History of Threshold ECDSA

Now showing

Doerner, **K**, Lee, shelat 18 & 19

1990s	[Lan95, GJKR96]	[MR01]	[GGN16, BGG17]	[Lin17]	[D K Ls 18, 19]	2018
Rewriting	Inverted Nonce	Multiplicative $s = \left(\frac{a}{k}\right) + \left(\frac{b\textcolor{green}{s}k}{k}\right)$				
Machinery	Honest Majority Magic	(Threshold) Paillier	Paillier	OT		

A Brief History of Threshold ECDSA

	2018						now-ish
	[Lin17]	[DKLs 18, 19]	[GG 18, 20]	[CGGMP 20]	[HLNR 18, 23]	[ANOSS 22]	[ST19, DOKSS20]
Rewriting	Multiplicative		Inverted Nonce		ECDSA tuple		<flexible>
Machinery	Paillier	OT		Paillier	OT++	PCG	<flexible>

A Brief History of Threshold ECDSA

Now showing		Gennaro Goldfeder 18 & 20				now-ish	
	[Lin17]	[DKLs 18, 19]	[GG 18, 20]	[CGGMP 20]	[HLNR 18, 23]	[ANOSS 22]	[ST19, DOKSS20]
Rewriting	Multiplicative	Inverted Nonce		ECDSA tuple		<flexible>	
Machinery	Paillier	OT	Paillier		OT++	PCG	<flexible>

A Brief History of Threshold ECDSA

Now showing		Canetti, Gennaro, Goldfeder, Makriyannis, Peled 20					now-ish
	[Lin17]	[DKLs 18, 19]	[GG 18, 20]	[CGGMP 20]	[HLNR 18, 23]	[ANOSS 22]	[ST19, DOKSS20]
Rewriting	Multiplicative		Inverted Nonce		ECDSA tuple		<flexible>
Machinery	Paillier	OT	Paillier		OT++	PCG	<flexible>

A Brief History of Threshold ECDSA

Now showing		Lindell Nof 18, Haitner, Lindell, Nof, Ranellucci 23					now-ish
	[Lin17]	[DKLs 18, 19]	[GG 18, 20]	[CGGMP 20]	[HLNR 18, 23]	[ANOSS 22]	[ST19, DOKSS20]
Rewriting	Multiplicative		Inverted Nonce		ECDSA tuple		<flexible>
Machinery	Paillier	OT	Paillier		OT++	PCG	<flexible>

A Brief History of Threshold ECDSA

Now showing		Abram Nof Orlandi Scholl Shlomovits 22					now-ish
	[Lin17]	[DKLs 18, 19]	[GG 18, 20]	[CGGMP 20]	[HLNR 18, 23]	[ANOSS 22]	[ST19, DOKSS20]
Rewriting	Multiplicative		Inverted Nonce		ECDSA tuple		<flexible>
Machinery	Paillier	OT	Paillier		OT++	PCG	<flexible>

A Brief History of Threshold ECDSA

Now showing

Smart Talibi 19, Dalskov, Orlandi, Keller, Shrishak, Shulman 20

	[Lin17]	[DKLs 18, 19]	[GG 18, 20]	[CGGMP 20]	[HLNR 18, 23]	[ANOSS 22]	[ST19, DOKSS20]
Rewriting	Multiplicative		Inverted Nonce		ECDSA tuple		<flexible>
Machinery	Paillier	OT	Paillier		OT++	PCG	<flexible>

A Brief History of Threshold ECDSA

now-ish

	[Lin17]	[DKLs 18, 19]	[GG 18, 20]	[CGGMP 20]	[HLNR 18, 23]	[ANOSS 22]	[ST19, DOKSS20]
Rewriting	Multiplicative		Inverted Nonce		ECDSA tuple		<flexible>
Machinery	Paillier	OT	Paillier		OT++	PCG	<flexible>
Verification	2P magic	Check relations in exponent	ZK in $\mathbb{Z}_N$ + Masked sig verification	GMW- style ZK in $\mathbb{Z}_N$	Replay in committed form ZK in $\mathbb{Z}_q$	BDOZ MAC	Any $\mathbb{Z}_q$ MAC

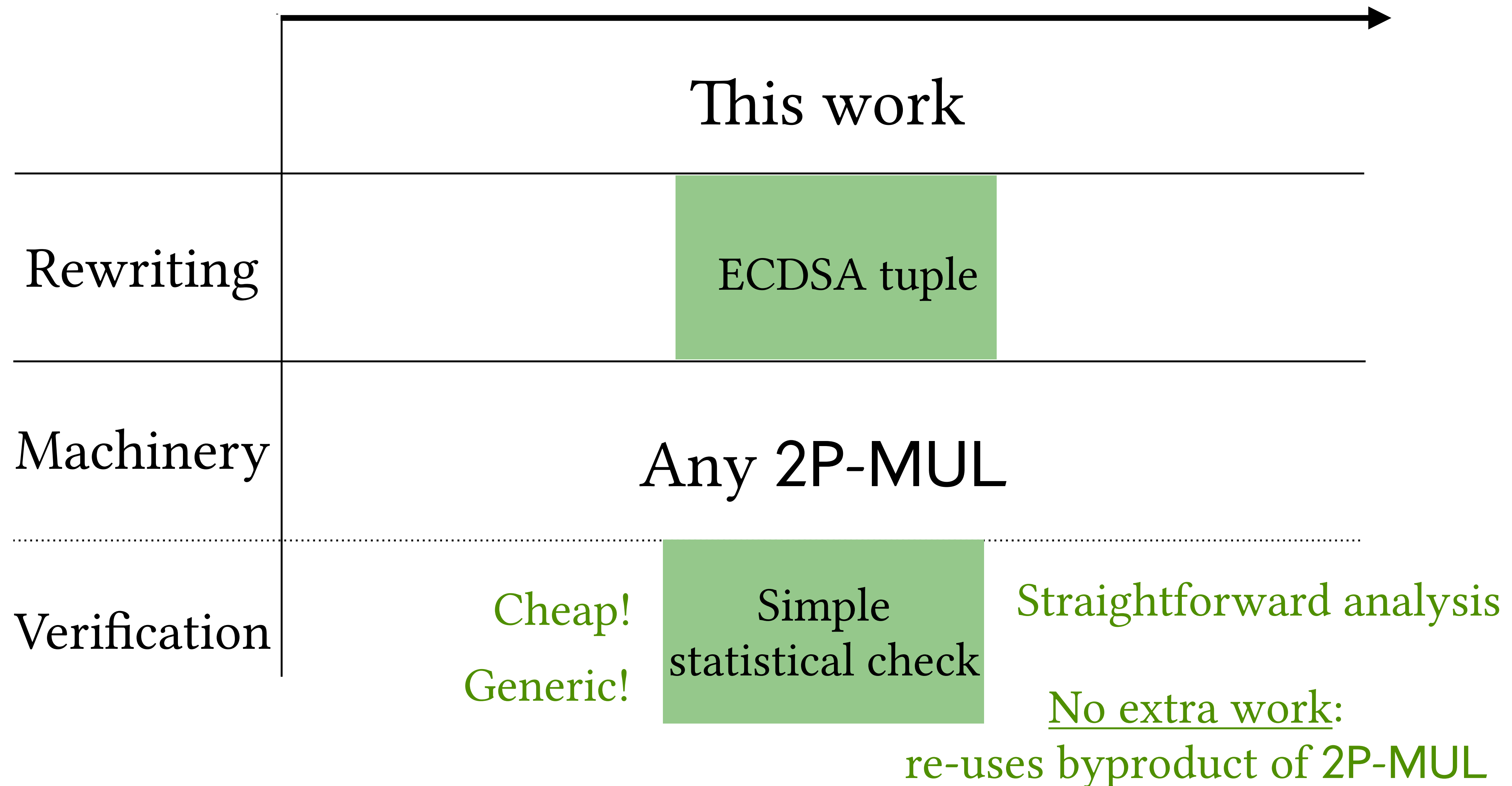
A Brief History of Threshold ECDSA

now-ish

	[Lin17]	[DKLs 18, 19]	[GG 18, 20]	[CGGMP 20]	[HLNR 18, 23]	[ANOSS 22]	[ST19, DOKSS20]
Rewriting	Multiplicative		Inverted Nonce		ECDSA tuple		<flexible>
Machinery	Paillier	OT	Paillier		OT++	PCG	<flexible>
Verification	2P magic	Check relations in exponent	ZK in $\mathbb{Z}_N$ + Masked sig verification	GMW- style ZK in $\mathbb{Z}_N$	Replay in committed form	BDOZ MAC	Any $\mathbb{Z}_q$ MAC
	2P-only	Involved analysis Interactive	Machinery- specific	Expensive proofs	ZK in $\mathbb{Z}_q$ UC ZK	Creating MACs is extra work	

A Brief History of Threshold ECDSA

now-ish



Rewriting ECDSA with ECDSA Tuples

[Lindell Nof Ranellucci 18] [Abram Nof Orlandi Scholl Shlomovits 22]

ECDSASign($\textcolor{green}{sk}, m$) :

$$\textcolor{green}{[k]} \leftarrow \mathbb{Z}_q$$

$$\textcolor{green}{R} = \textcolor{green}{[k]} \cdot G$$

$$e = H(m)$$

$$\textcolor{green}{s} = \frac{e + \textcolor{green}{[sk]} \cdot r_x}{\textcolor{green}{[k]}}$$

output $\sigma = (\textcolor{green}{s}, \textcolor{green}{R})$

Rewriting ECDSA with ECDSA Tuples

[Lindell Nof Ranellucci 18] [Abram Nof Orlandi Scholl Shlomovits 22]

ECDSASign($\textcolor{green}{sk}, m$) :

$$\textcolor{green}{[k]} \leftarrow \mathbb{Z}_q$$

$$\textcolor{green}{R} = \textcolor{green}{[k]} \cdot G$$

$$e = H(m)$$

$$\textcolor{green}{s} = \frac{e + \textcolor{green}{[sk]} \cdot r_x}{\textcolor{green}{[k]}} \cdot \frac{\textcolor{red}{[\phi]}}{\textcolor{red}{[\phi]}}$$

output $\sigma = (\textcolor{green}{s}, \textcolor{green}{R})$

Rewriting ECDSA with ECDSA Tuples

[Lindell Nof Ranellucci 18] [Abram Nof Orlandi Scholl Shlomovits 22]

ECDSASign($\textcolor{green}{sk}, m$) :

$$[\textcolor{green}{k}] \leftarrow \mathbb{Z}_q$$

$$\textcolor{green}{R} = [\textcolor{green}{k}] \cdot G$$

$$e = H(m)$$

$$\alpha = (e + [\textcolor{green}{sk}] \cdot r_x) [\textcolor{red}{\phi}]$$

$$\beta = [\textcolor{green}{k}][\textcolor{red}{\phi}]$$

$$\textcolor{green}{s} =$$

$$\text{output } \sigma = (\textcolor{green}{s}, \textcolor{green}{R})$$

Rewriting ECDSA with ECDSA Tuples

[Lindell Nof Ranellucci 18] [Abram Nof Orlandi Scholl Shlomovits 22]

ECDSASign(**sk**, m) :

$$[k] \leftarrow \mathbb{Z}_q$$

$$R = [k] \cdot G$$

$$e = H(m)$$

$$\alpha = (e + [\mathbf{sk}] \cdot r_x) [\phi]$$

$$\beta = [k][\phi]$$

$$s = \frac{\alpha}{\beta}$$

output $\sigma = (s, R)$

Rewriting ECDSA with ECDSA Tuples

[Lindell Nof Ranellucci 18] [Abram Nof Orlandi Scholl Shlomovits 22]

ECDSASign($\textcolor{green}{sk}, m$) :

$$[\textcolor{green}{k}] \leftarrow \mathbb{Z}_q$$

$$\textcolor{green}{R} = [\textcolor{green}{k}] \cdot G$$

$$e = H(m)$$

$$[\textcolor{red}{\phi}] \leftarrow \mathbb{Z}_q$$

$$\alpha = (e + [\textcolor{green}{sk}] \cdot r_x) [\textcolor{red}{\phi}]$$

$$\beta = [\textcolor{green}{k}][\textcolor{red}{\phi}]$$

$$\textcolor{green}{s} = \frac{\alpha}{\beta}$$

$$\text{output } \sigma = (\textcolor{green}{s}, \textcolor{green}{R})$$

Rewriting ECDSA with ECDSA Tuples

[Lindell Nof Ranellucci 18] [Abram Nof Orlandi Scholl Shlomovits 22]

ECDSASign($\textcolor{green}{sk}, m$) :

$$[\textcolor{green}{k}] \leftarrow \mathbb{Z}_q$$

$$\textcolor{green}{R} = [\textcolor{green}{k}] \cdot G$$

$$e = H(m)$$

$$[\textcolor{red}{\phi}] \leftarrow \mathbb{Z}_q$$

$$\alpha = (e + [\textcolor{green}{sk}] \cdot r_x) [\textcolor{red}{\phi}]$$

Public values

$$\beta = [\textcolor{green}{k}][\textcolor{red}{\phi}]$$

$$\textcolor{green}{s} = \frac{\alpha}{\beta}$$

output $\sigma = (\textcolor{green}{s}, \textcolor{green}{R})$

Rewriting ECDSA with ECDSA Tuples

[Lindell Nof Ranellucci 18] [Abram Nof Orlandi Scholl Shlomovits 22]

ECDSASign($\textcolor{green}{sk}, m$) :

$$[\textcolor{green}{k}] \leftarrow \mathbb{Z}_q$$

$$\textcolor{green}{R} = [\textcolor{green}{k}] \cdot G$$

$$e = H(m)$$

$$[\textcolor{red}{\phi}] \leftarrow \mathbb{Z}_q$$

$$\alpha = (e + [\textcolor{green}{sk}] \cdot r_x) [\textcolor{red}{\phi}]$$

Public values

$$\beta = [\textcolor{green}{k}][\textcolor{red}{\phi}]$$

Safe to reveal
 β : because $\textcolor{red}{\phi}$ is OTP
 α : because fixed by β, s

$$\textcolor{green}{s} = \frac{\alpha}{\beta}$$

output $\sigma = (\textcolor{green}{s}, \textcolor{green}{R})$

Rewriting ECDSA with ECDSA Tuples

[Lindell Nof Ranellucci 18] [Abram Nof Orlandi Scholl Shlomovits 22]

ECDSASign(sk, m) :

$$[k] \leftarrow \mathbb{Z}_q$$

$$R = [k] \cdot G$$

$$e = H(m)$$

$$[\phi] \leftarrow \mathbb{Z}_q$$

$$\alpha = (e + [\text{sk}] \cdot r_x) [\phi]$$

Public values

$$\beta = [k][\phi]$$

Secure mult: Only (nonlinear)
combination of secret values

Safe to reveal
 β : because ϕ is OTP
 α : because fixed by β, s

$$s = \frac{\alpha}{\beta}$$

output $\sigma = (s, R)$

Signing from ECDSA Tuples

[Abram Nof Orlandi Scholl Shlomovits 22]

$[sk]$ $[k]$ $[\phi]$ $[\phi k]$ $[\phi sk]$

Signing from ECDSA Tuples

[Abram Nof Orlandi Scholl Shlomovits 22]

$[sk]$ $[k]$ $[\phi]$ $[\phi k]$ $[\phi sk]$

Round 1

Round 2

Establish $R = [k] \cdot G$

Signing from ECDSA Tuples

[Abram Nof Orlandi Scholl Shlomovits 22]

$[sk]$ $[k]$ $[\phi]$ $[\phi k]$ $[\phi sk]$

Round 1

Round 2

Establish $R = [k] \cdot G$

Round 3

Reveal $\alpha = e + r_x[\phi sk]$ and $\beta = [\phi k]$

Output $(R, s = \alpha/\beta)$

ECDSA Tuple Generation

$[sk][k]$

$[\phi]$

$[\phi k]$

$[\phi sk]$

ECDSA Tuple Generation

Input : $[sk][k]$

Sample : $[\phi]$

$[\phi k]$

$[\phi sk]$

ECDSA Tuple Generation

Local

Input : $[sk][k]$
Sample : $[\phi]$

$[\phi k]$

$[\phi sk]$

ECDSA Tuple Generation

Local

Input : $[sk][k]$
Sample : $[\phi]$

$$[\phi k] = \text{MULT}([\phi], [k])$$
$$[\phi sk] = \text{MULT}([\phi], [sk])$$

ECDSA Tuple Generation

Local

Input : $[sk][k]$
Sample : $[\phi]$

Consistency:
straightforward

$$[\phi k] = \text{MULT}([\phi], [k])$$
$$[\phi sk] = \text{MULT}([\phi], [sk])$$

ECDSA Tuple Generation

Local

Input : $[sk][k]$
Sample : $[\phi]$

Consistency:
straightforward

$$[\phi k] = \text{MULT}([\phi], [k])$$
$$[\phi sk] = \text{MULT}([\phi], [sk])$$

Verify:

$$[k] \cdot G = R$$
$$[sk] \cdot G = pk$$

ECDSA Tuple Generation

Local

Input : $[sk][k]$
Sample : $[\phi]$

Consistency:
straightforward

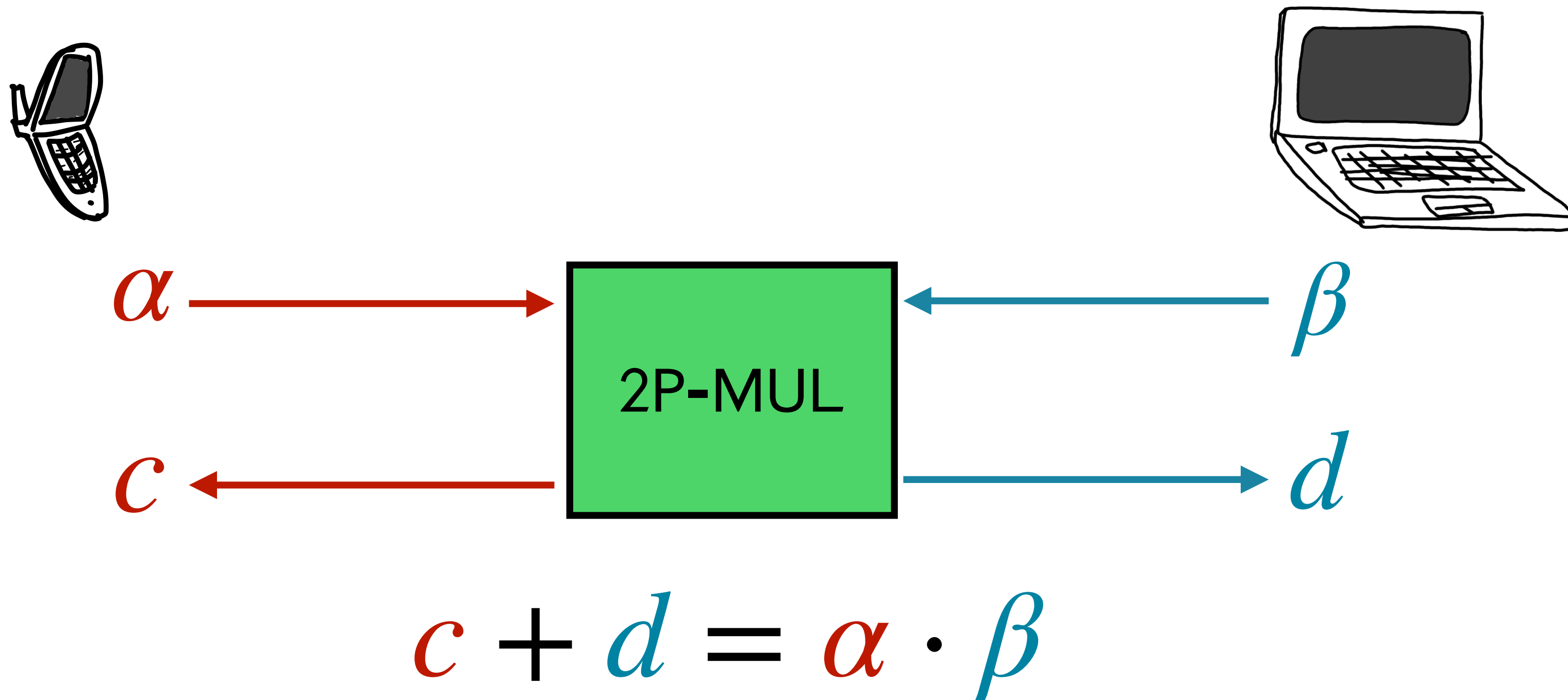
$$[\phi k] = \text{MULT}([\phi], [k])$$
$$[\phi sk] = \text{MULT}([\phi], [sk])$$

Previous works: ZK proofs, MACs, etc.
This work: Simple pairwise check

Verify: $[k] \cdot G = R$
 $[sk] \cdot G = pk$

Secure Two-Party Multiplication

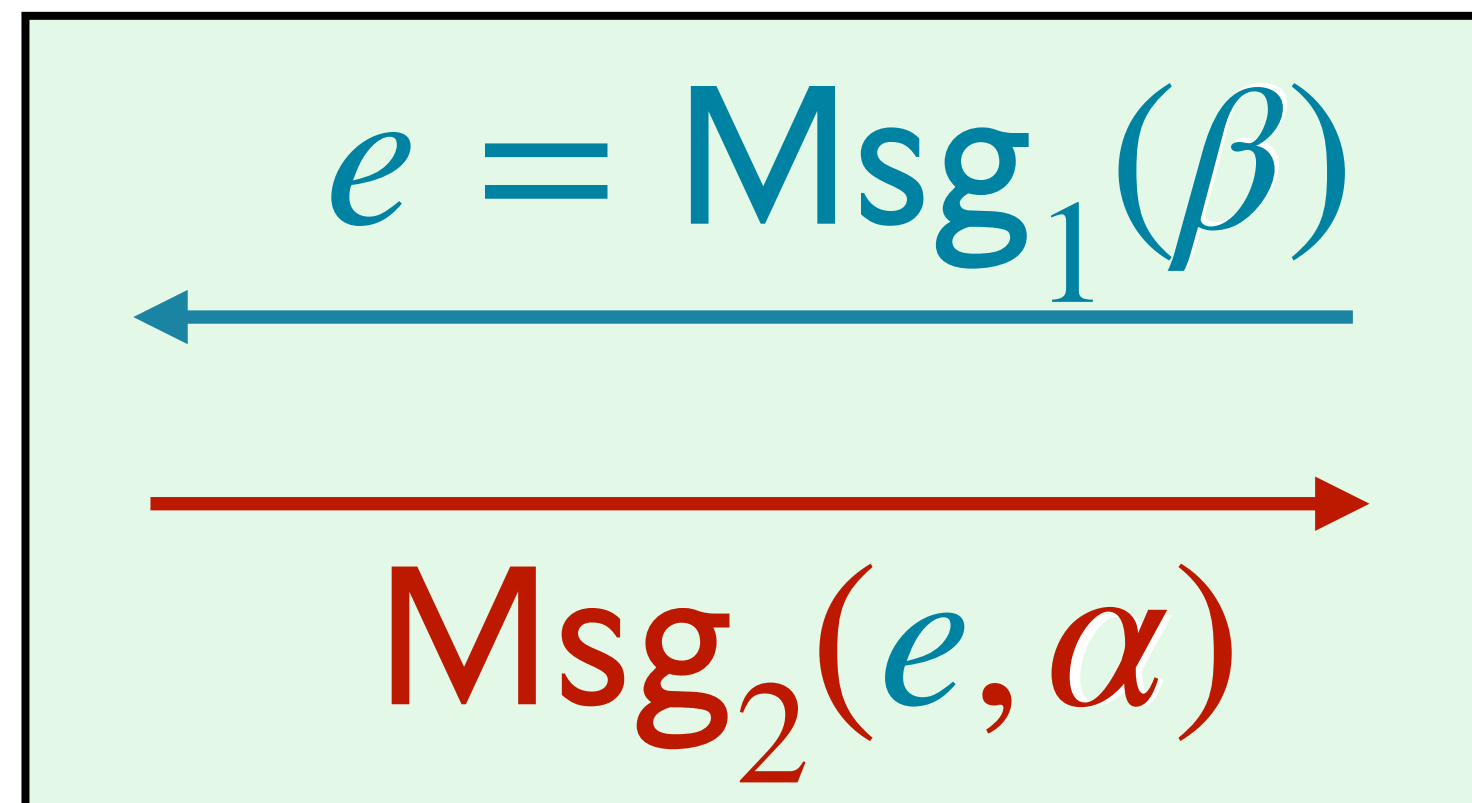
a.k.a. OLE, Mult2Add



Two-Round 2P-MUL



c



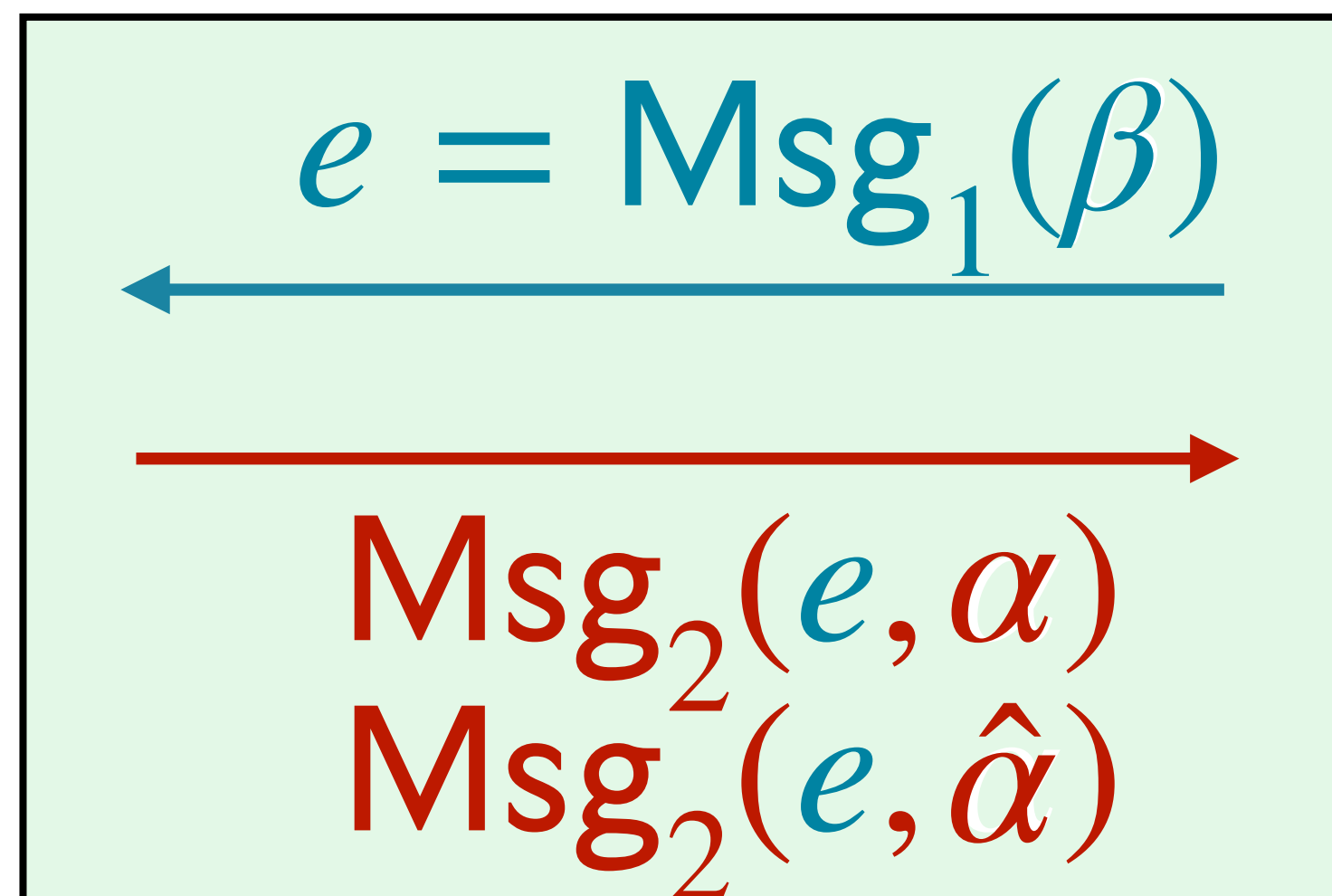
d

$$c + d = \alpha \cdot \beta$$

Two-Round 2P-MUL



c



d

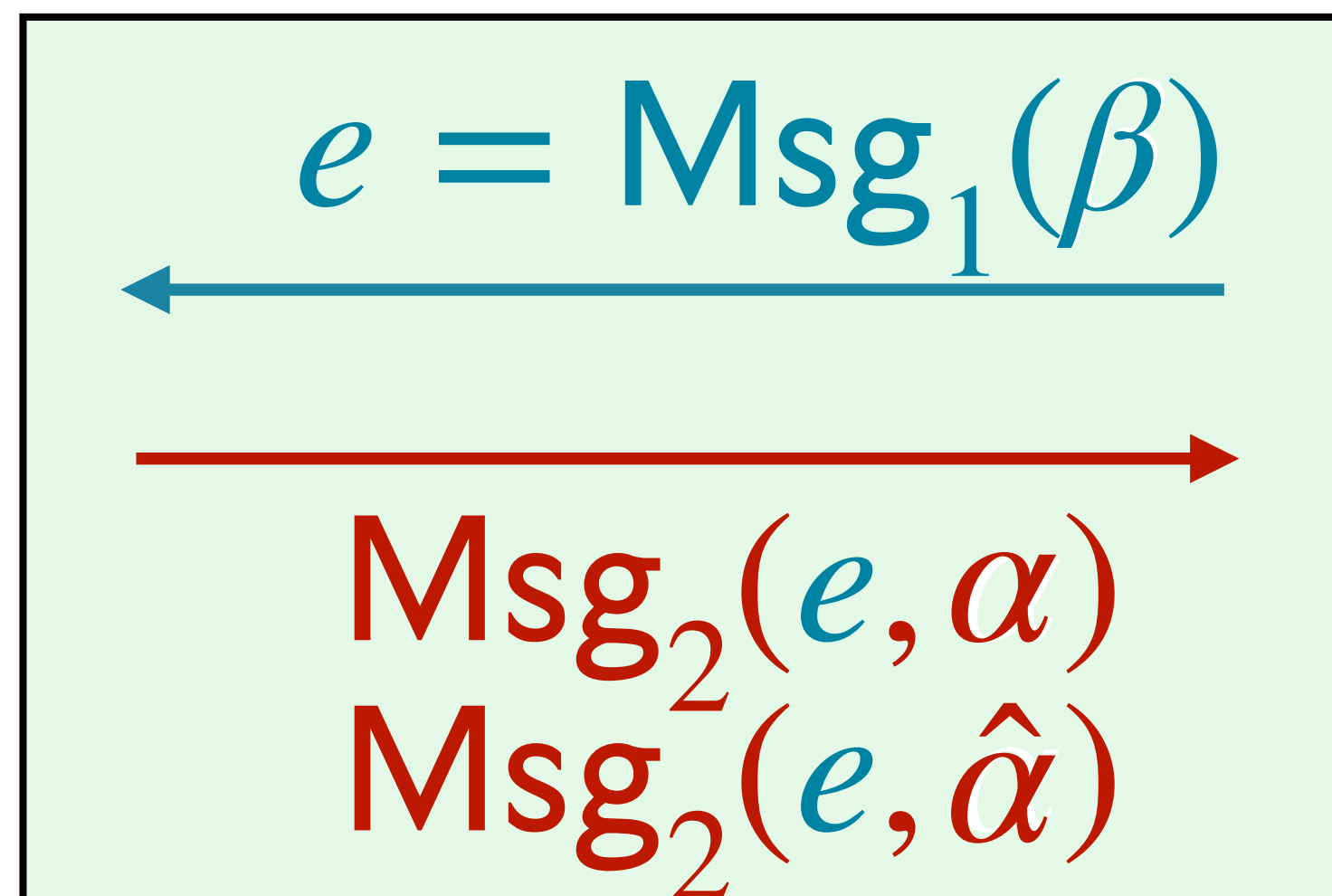
$$c + d = \alpha \cdot \beta$$

$$\hat{c} + \hat{d} = \hat{\alpha} \cdot \beta$$

Two-Round 2P-MUL



c



d

$$c + d = \alpha \cdot \beta$$

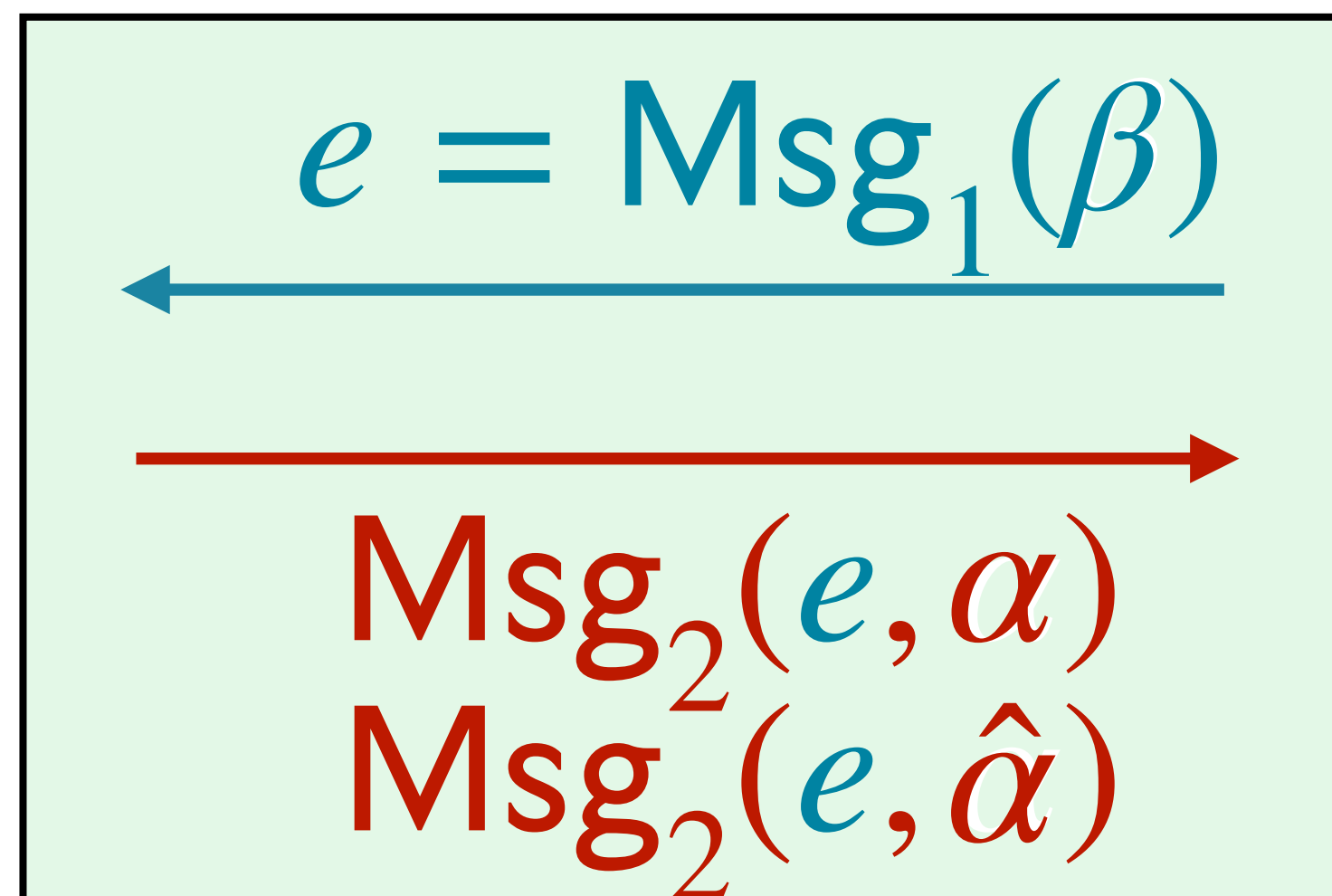
$$\hat{c} + \hat{d} = \hat{\alpha} \cdot \beta$$

Consistency
“for free”

Two-Round 2P-MUL



c



d

a.k.a. Vector OLE
(VOLE)

$$\begin{aligned} c + d &= \alpha \cdot \beta \\ \hat{c} + \hat{d} &= \hat{\alpha} \cdot \beta \end{aligned}$$

Consistency
“for free”

ECDSA Tuple Generation

Input : $[sk][k]$
Sample : $[\phi]$

Consistency:
straightforward



$$[\phi k] = \text{MULT}([\phi], [k])$$
$$[\phi sk] = \text{MULT}([\phi], [sk])$$

ECDSA Tuple Generation

Input : $[sk][k]$
Sample : $[\phi]$

Consistency:
straightforward



$$[\phi k] = \text{MULT}([\phi], [k])$$
$$[\phi sk] = \text{MULT}([\phi], [sk])$$

Verify:

$$[k] \cdot G = R$$
$$[sk] \cdot G = pk$$

ECDSA Tuple Generation

Input : $[sk][k]$
Sample : $[\phi]$

Consistency:
straightforward



ϕ is a MAC on k, sk

Verify MAC in $\mathbb{G}$

Byproduct of 2P-MUL:
BDOZ MACs

Verify in parallel with MUL

$$[\phi k] = \text{MULT}([\phi], [k])$$
$$[\phi sk] = \text{MULT}([\phi], [sk])$$

Verify:

$$[k] \cdot G = R$$
$$[sk] \cdot G = pk$$

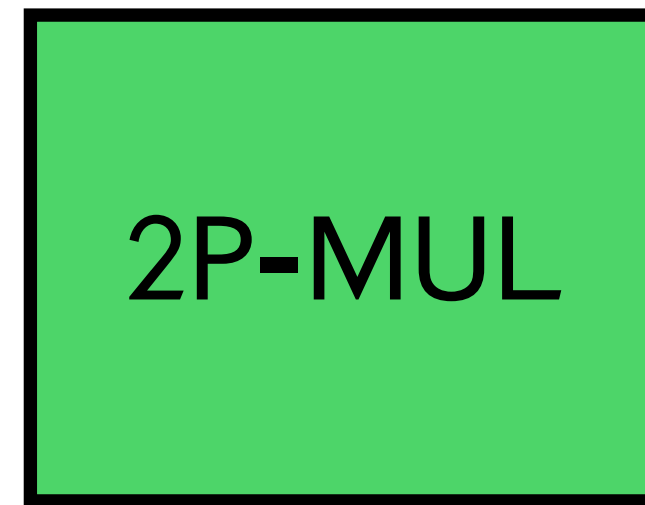
Verifying Consistency w.r.t. $\mathbb{G}$

MULT($[\phi]$, $[k]$)

Simplified :



ϕ



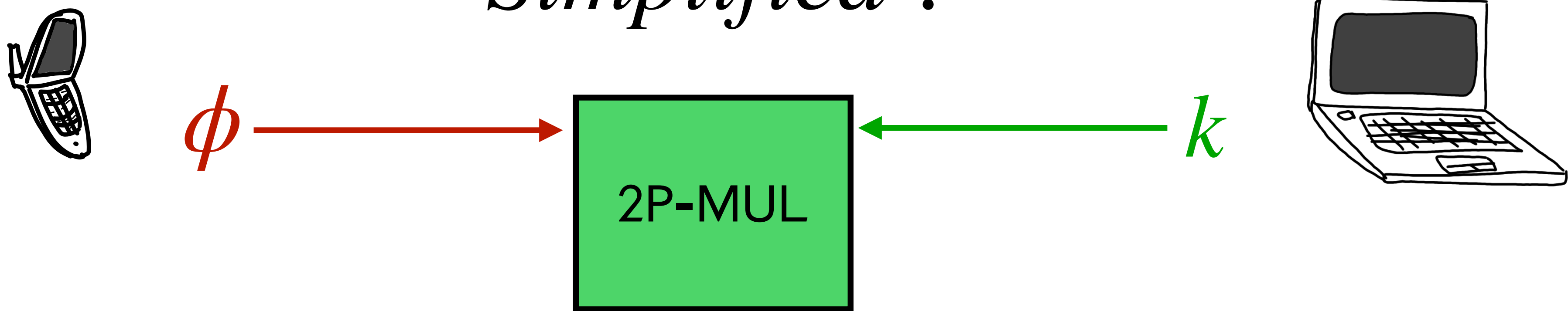
k



Verifying Consistency w.r.t. $\mathbb{G}$

MULT($[\phi]$, $[k]$)

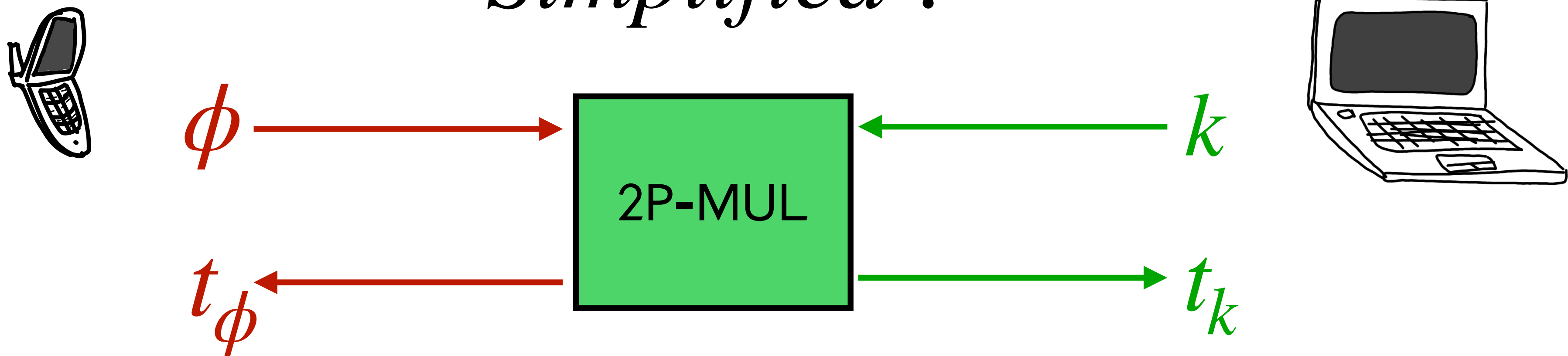
Simplified :



Verifying Consistency w.r.t. $\mathbb{G}$

MULT($[\phi]$, $[k]$)

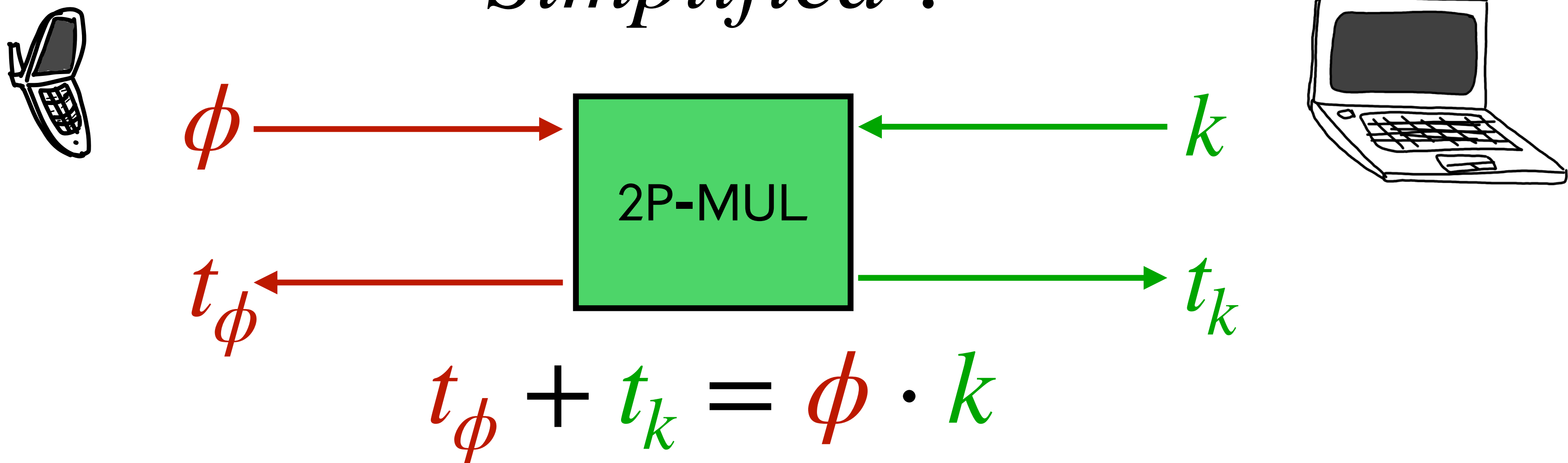
Simplified :



Verifying Consistency w.r.t. $\mathbb{G}$

MULT($[\phi]$, $[k]$)

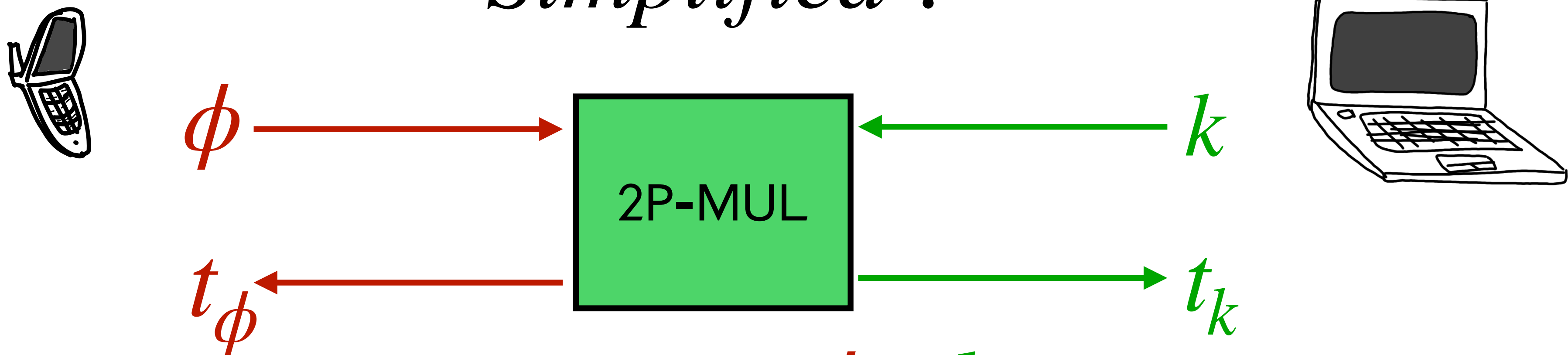
Simplified :



Verifying Consistency w.r.t. $\mathbb{G}$

MULT($[\phi]$, $[k]$)

Simplified :



$$t_\phi + t_k = \phi \cdot k$$

Claim: $[k] \cdot G = R$

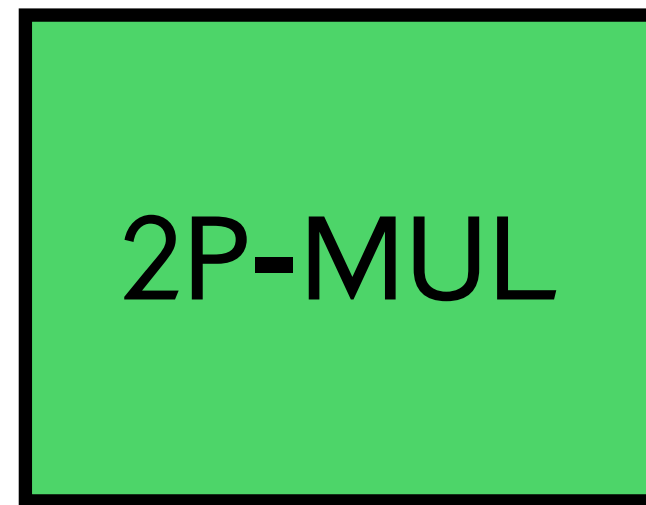
Verifying Consistency w.r.t. $\mathbb{G}$

MULT($[\phi]$, $[k]$)

Simplified :



ϕ



k



t_ϕ



2P-MUL



t_k

$$T_\phi = t_\phi \cdot G$$

$$t_\phi + t_k = \phi \cdot k$$

$$T_k = t_k \cdot G$$

Claim: $[k] \cdot G = R$

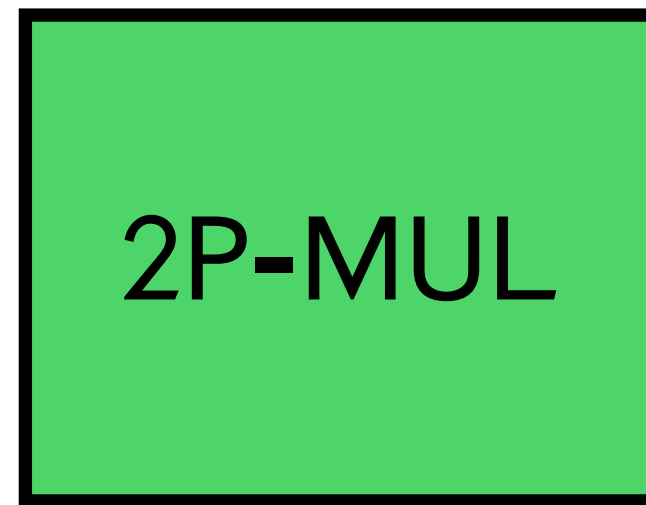
Verifying Consistency w.r.t. $\mathbb{G}$

MULT($[\phi]$, $[k]$)

Simplified :



ϕ



k



t_ϕ



2P-MUL



t_k

$$T_\phi = t_\phi \cdot G$$

$$t_\phi + t_k = \phi \cdot k$$

$$T_k = t_k \cdot G$$

Claim: $[k] \cdot G = R$

$$T_k = \phi R - T_\phi$$

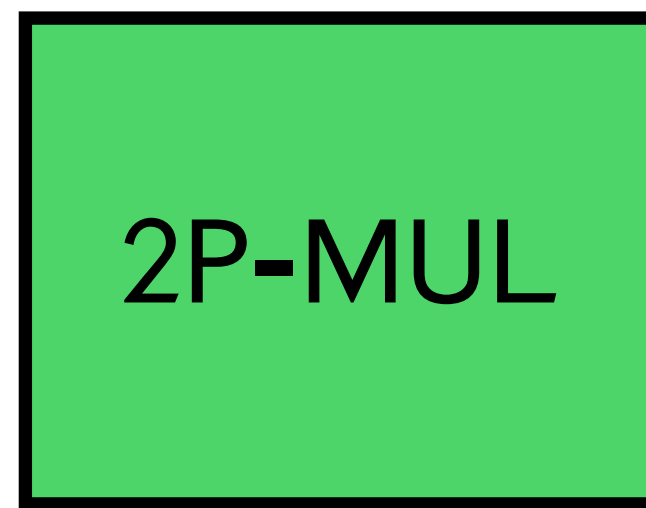
Verifying Consistency w.r.t. $\mathbb{G}$

MULT($[\phi]$, $[k]$)

Simplified :



ϕ



k



t_ϕ



t_k

$$T_\phi = t_\phi \cdot G$$

$$t_\phi + t_k = \phi \cdot k$$

$$T_k = t_k \cdot G$$

Claim: $[k] \cdot G = R$

$$T_k = \phi R - T_\phi$$

T_k



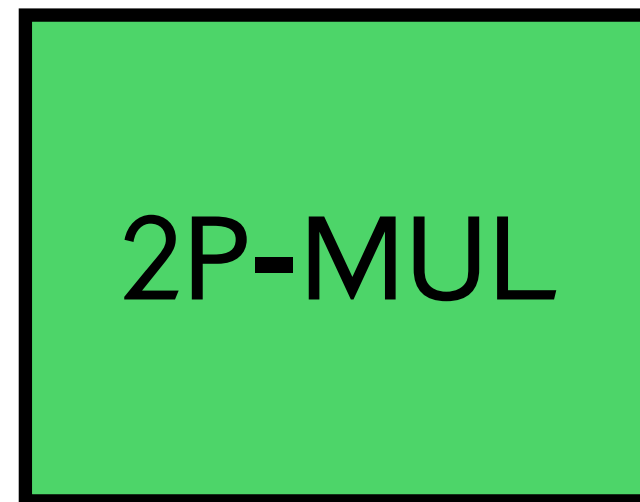
Verifying Consistency w.r.t. $\mathbb{G}$

MULT($[\phi]$, $[k]$)

Simplified :



ϕ



k



t_ϕ



t_k

$$T_\phi = t_\phi \cdot G$$

$$t_\phi + t_k = \phi \cdot k$$

$$T_k = t_k \cdot G$$

Claim: $[k] \cdot G = R$

$$T_k = \phi R - T_\phi$$

Match?

T_k



Verifying Consistency w.r.t. $\mathbb{G}$

MULT($[\phi]$, $[k]$)

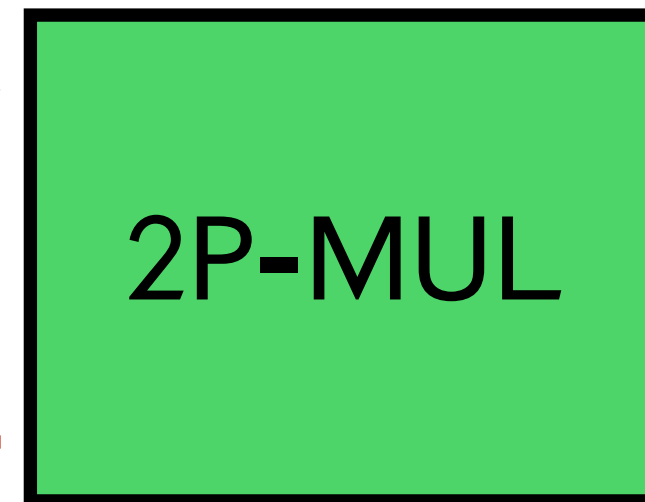
Simplified :



No information
about k

ϕ

t_ϕ



k

t_k



$$T_\phi = t_\phi \cdot G$$

$$t_\phi + t_k = \phi \cdot k$$

$$T_k = t_k \cdot G$$

Claim: $[k] \cdot G = R$

$$T_k = \phi R - T_\phi$$

Match?

T_k

Verifying Consistency w.r.t. $\mathbb{G}$

MULT($[\phi]$, $[k]$)

Simplified :

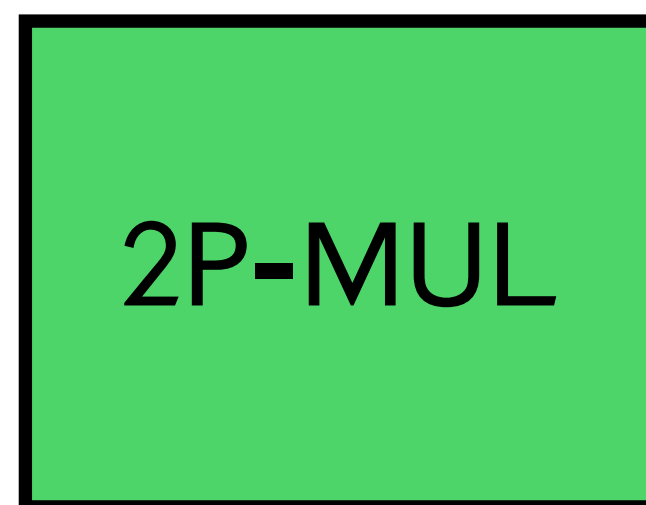


No information
about k

ϕ

t_ϕ

$$T_\phi = t_\phi \cdot G$$



2P-MUL

k

t_k

$$t_\phi + t_k = \phi \cdot k$$

$$T_k = t_k \cdot G$$



Claim: $[k] \cdot G = R$

$$T_k = \phi R - T_\phi$$

Match?

T_k

In case of cheat:
have to guess ϕ
(2^{-256} chance)

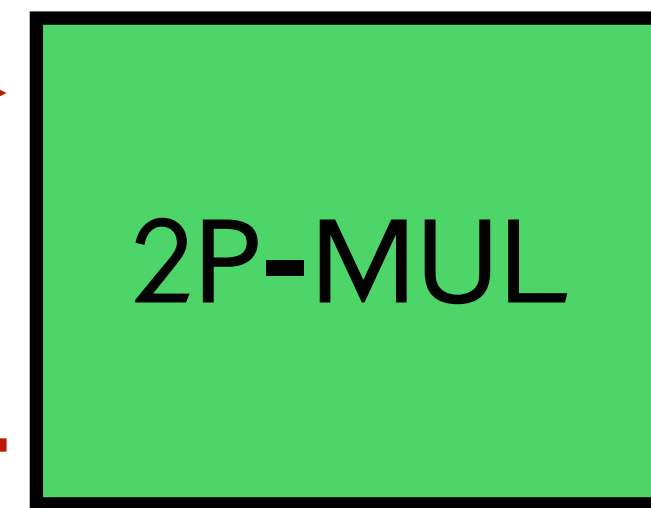
Verifying Consistency w.r.t. $\mathbb{G}$

MULT($[\phi]$, $[k]$)

Simplified :



ϕ



k



t_ϕ



t_k

$$T_\phi = t_\phi \cdot G$$

$$t_\phi + t_k = \phi \cdot k$$

$$T_k = t_k \cdot G$$

Claimed $R^* = \Delta + k \cdot G$

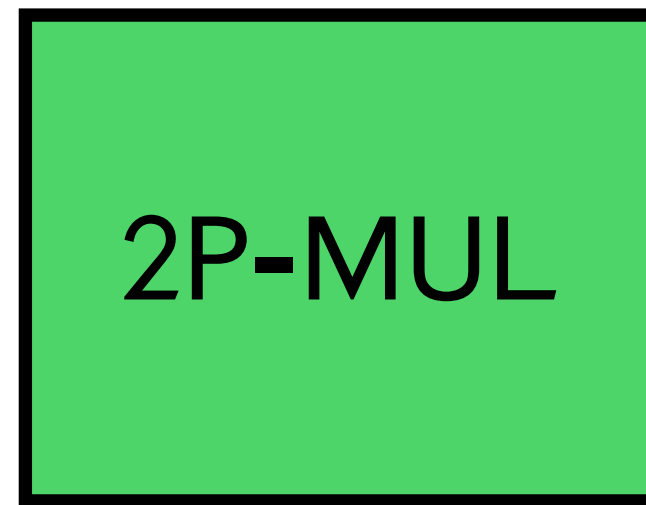
Verifying Consistency w.r.t. $\mathbb{G}$

MULT($[\phi]$, $[k]$)

Simplified :



ϕ



k



t_ϕ



2P-MUL



t_k

$$T_\phi = t_\phi \cdot G$$

$$t_\phi + t_k = \phi \cdot k$$

$$T_k = t_k \cdot G$$

Claimed $R^* = \Delta + k \cdot G$

$$T_k^* = \phi R^* - T_\phi$$

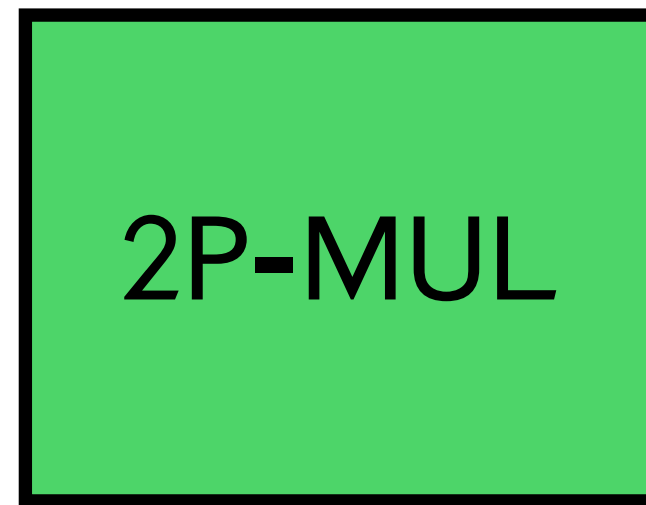
Verifying Consistency w.r.t. $\mathbb{G}$

MULT($[\phi]$, $[k]$)

Simplified :



ϕ



k



t_ϕ



t_k

$$T_\phi = t_\phi \cdot G$$

$$t_\phi + t_k = \phi \cdot k$$

$$T_k = t_k \cdot G$$

Claimed $R^* = \Delta + k \cdot G$

$$\begin{aligned} T_k^* &= \phi R^* - T_\phi \\ &= T_k + \phi \Delta \end{aligned}$$

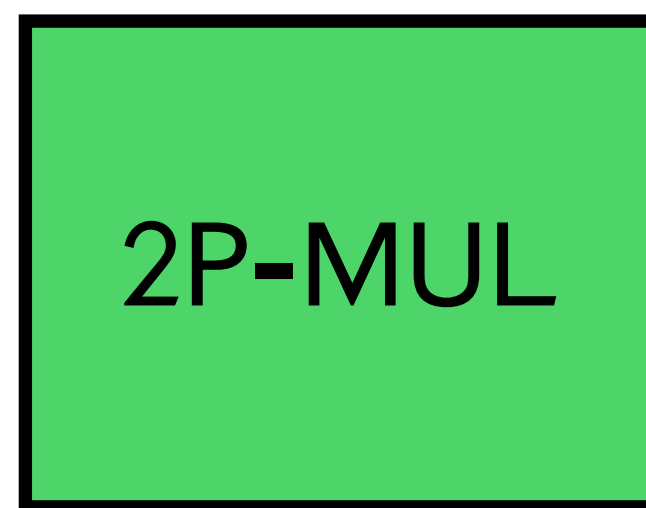
Verifying Consistency w.r.t. $\mathbb{G}$

MULT($[\phi]$, $[k]$)

Simplified :



ϕ



k



t_ϕ



t_k

$$T_\phi = t_\phi \cdot G$$

$$t_\phi + t_k = \phi \cdot k$$

$$T_k = t_k \cdot G$$

$$\text{Claimed } R^* = \Delta + k \cdot G$$

$$\begin{aligned} T_k^* &= \phi R^* - T_\phi \\ &= T_k + \phi \Delta \end{aligned}$$

No information about ϕ
 $\Rightarrow$ distributed uniformly in $\mathbb{Z}_q$

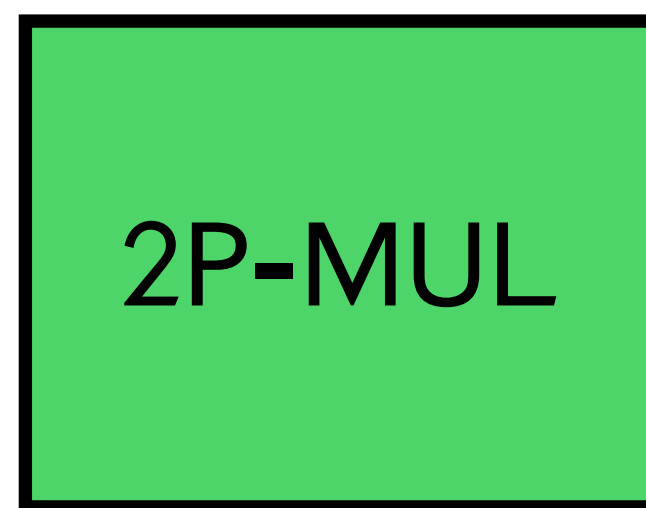
Verifying Consistency w.r.t. $\mathbb{G}$

MULT($[\phi]$, $[k]$)

Simplified :



ϕ



k



t_ϕ



t_k

$$T_\phi = t_\phi \cdot G$$

$$t_\phi + t_k = \phi \cdot k$$

$$T_k = t_k \cdot G$$

$$\text{Claimed } R^* = \Delta + k \cdot G$$

$$\begin{aligned} T_k^* &= \phi R^* - T_\phi \\ &= T_k + \phi \Delta \end{aligned}$$

No information about ϕ
 $\Rightarrow$ distributed uniformly in $\mathbb{Z}_q$

Statistically unlikely to send correct T_k^*

Notes on Consistency Check

- Case 1: Inconsistent k^* —almost certainly fails
- Case 2: Consistent k — nothing about ϕ leaked
 $\Rightarrow \phi$ is a MAC key, but also safe to (re)use in ECDSA tuple
- Very cheap, cost superseded by 2P-MUL
- Exact same structure for $[\phi sk]$ verification with pk
- Actual check: each party validates 2P-MUL inputs
(i.e. shares of k , sk , ϕ) used by every counterparty

3 Round ECDSA Signing

[This work]

Sample $[k]$

Round 1

Establish $R = [k] \cdot G$

Exchange $\text{Commit}(R_i)$

Round 2

Release R

3 Round ECDSA Signing

[This work]

Sample $[k]$ $[\phi]$

Round 1

Round 2

Establish $R = [k] \cdot G$

Exchange $\text{Commit}(R_i)$

Release R

Multiply $[\phi]$ with $[k]$, $[sk]$

MUL message 1

MUL message 2

*Pairwise
consistency check*

$[sk]$ $[k]$ $[\phi]$ $[\phi k]$ $[\phi sk]$

3 Round ECDSA Signing

[This work]

Sample $[k]$ $[\phi]$

Round 1

Establish $R = [k] \cdot G$

Exchange $\text{Commit}(R_i)$

Multiply $[\phi]$ with $[k]$, $[sk]$

MUL message 1

Round 2

Release R

MUL message 2

*Pairwise
consistency check*

$[sk]$ $[k]$ $[\phi]$ $[\phi k]$ $[\phi sk]$

Round 3

Reveal $\alpha = e[\phi] + r_x[\phi sk]$ and $\beta = [\phi k]$

Output $(R, \sigma = \alpha/\beta)$



Intro

How to distribute ECDSA

Tradeoffs

MP-Schnorr
is easy

but not
ECDSA

Evolution of
Techniques

ECDSA
Tuples

Our protocol:
Simple consistency
check

OT vs
AHE

Instantiating Multiplication

- Secure n -party mult can be reduced to $2n$ instances of 2P-MUL
- 2P-MUL inherently requires public key crypto
- Broadly two approaches:
 - Additively Homomorphic Encryption
(low bandwidth, high computation)
 - Oblivious Transfer
(low computation, high bandwidth)

2P-MUL from Additively Homomorphic Encryption

- Additive Homomorphism: $\alpha \cdot \text{Enc}(x) + \text{Enc}(\beta) = \text{Enc}(\alpha x + \beta)$

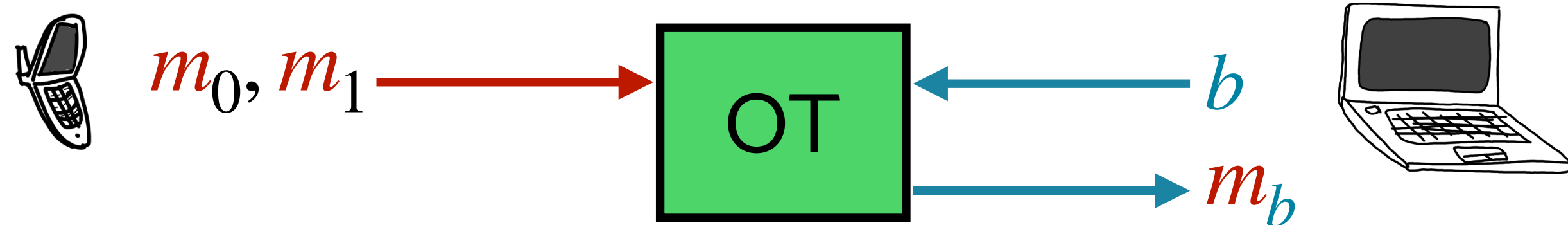
[Gilboa 99]: Conceptually simple protocol for MUL from AHE

[CGGMP 20]: Hardened for active security through ZK proofs

- Instantiations from factoring based cryptography (e.g. [Paillier 99]) and class groups [Castagnos Laguillaumie 15]
- Advantages: Parties exchange (relatively) compact ciphertexts
- Downsides:
 - Ciphertext operations are heavy (2 orders of magnitude slower than EC)
 - Seem to require ZK proofs to prevent misuse

2P-MUL from Oblivious Transfer

- Oblivious Transfer (OT):



[Gilboa 99]: Elegant protocol for MUL from OT

[DKLs 18,19, HMRT 22]: active security by randomized encoding+statistical checks

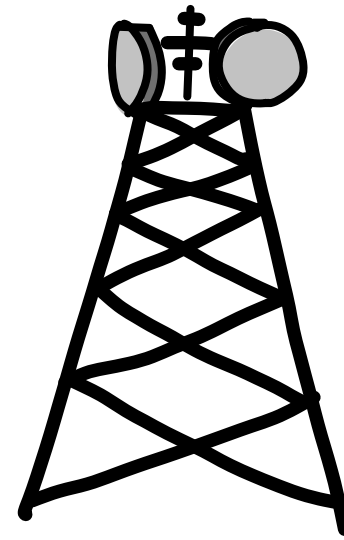
- Instantiable with ECDSA curve (think DH key exchange)
- Advantages: By OT Extension [IKNP03, Roy22] public key operations can be moved to one-time key generation phase, so only hashes when signing (1 order of magnitude slower than single party signing)
- Downsides: ~ 1000 OTs/sig, each transmits two $\mathbb{Z}_q$ elements

2P-MUL: AHE vs OT

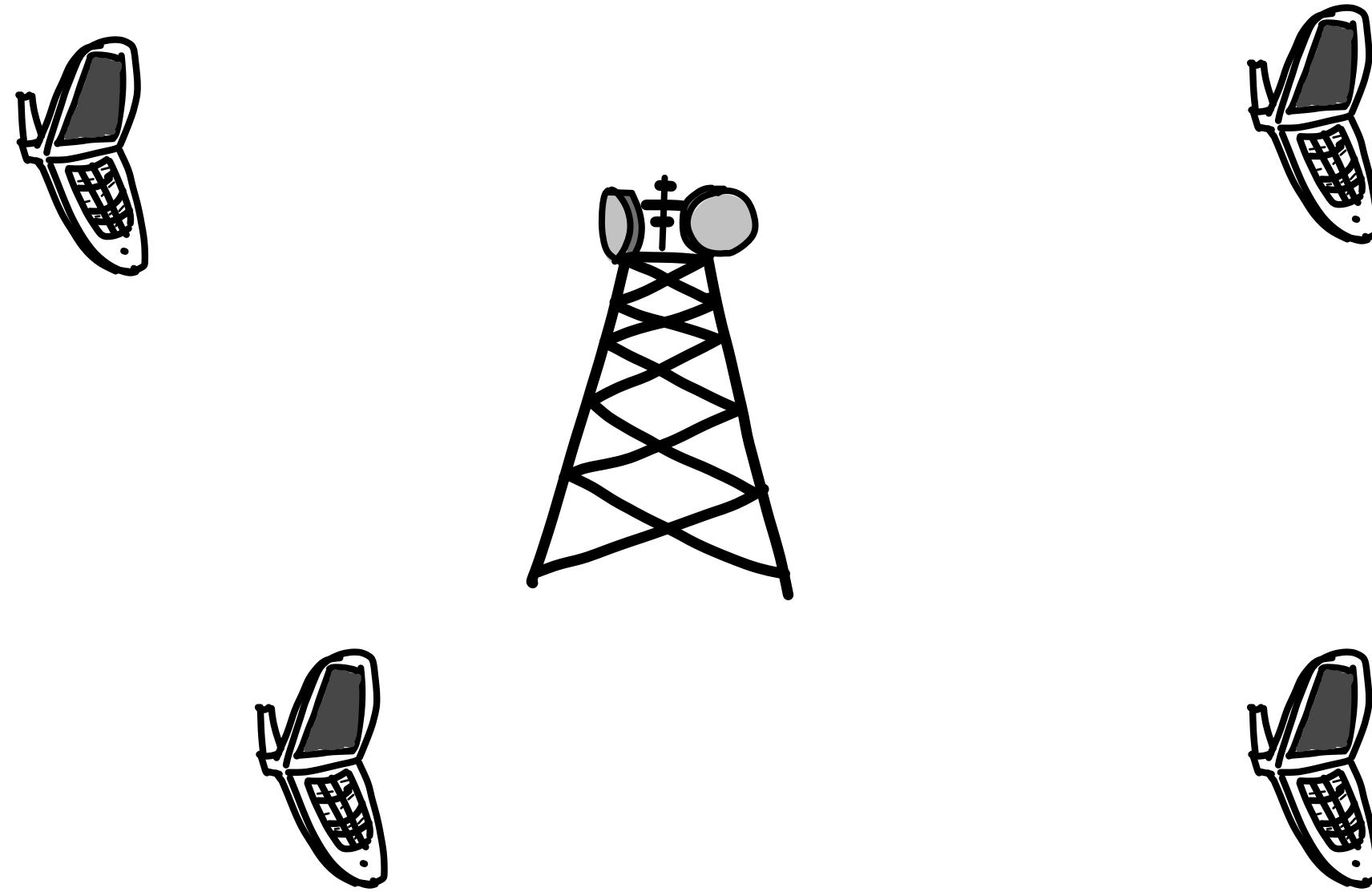
- Tradeoff to make: Computation vs. Bandwidth during signing time
- Rough costs with 256-bit curve, for each additional party (computation aggregated across [Gavenda 21, XAXYC 21, BMP 22]):

	Bandwidth	Computation
OT [DKLs 23]	60 KB	Few milliseconds
Paillier [CGGMP 20]	15 KB	Hundreds of milliseconds
Paillier [GG 18]	7 KB	Hundreds of milliseconds
Class Groups [CCLST20, YCX21]	4.5 KB	> 1 second

Is communication the bottleneck?



Is communication the bottleneck?



- **Mobile applications (human-initiated):**

Is communication the bottleneck?



- **Mobile applications (human-initiated):**

Is communication the bottleneck?



- **Mobile applications (human-initiated):**
 - eg. $t=4$, $\sim 2\text{Mbits}$ transmitted per party

Is communication the bottleneck?



- **Mobile applications (human-initiated):**
 - eg. $t=4$, $\sim 2\text{Mbits}$ transmitted per party
 - Well within LTE envelope for responsiveness

Example 1: Mobile Wallet

Multiplier: OT-based

Parties: 4

Curve: 256-bit

2 Mbits

sent per party

Example 1: Mobile Wallet

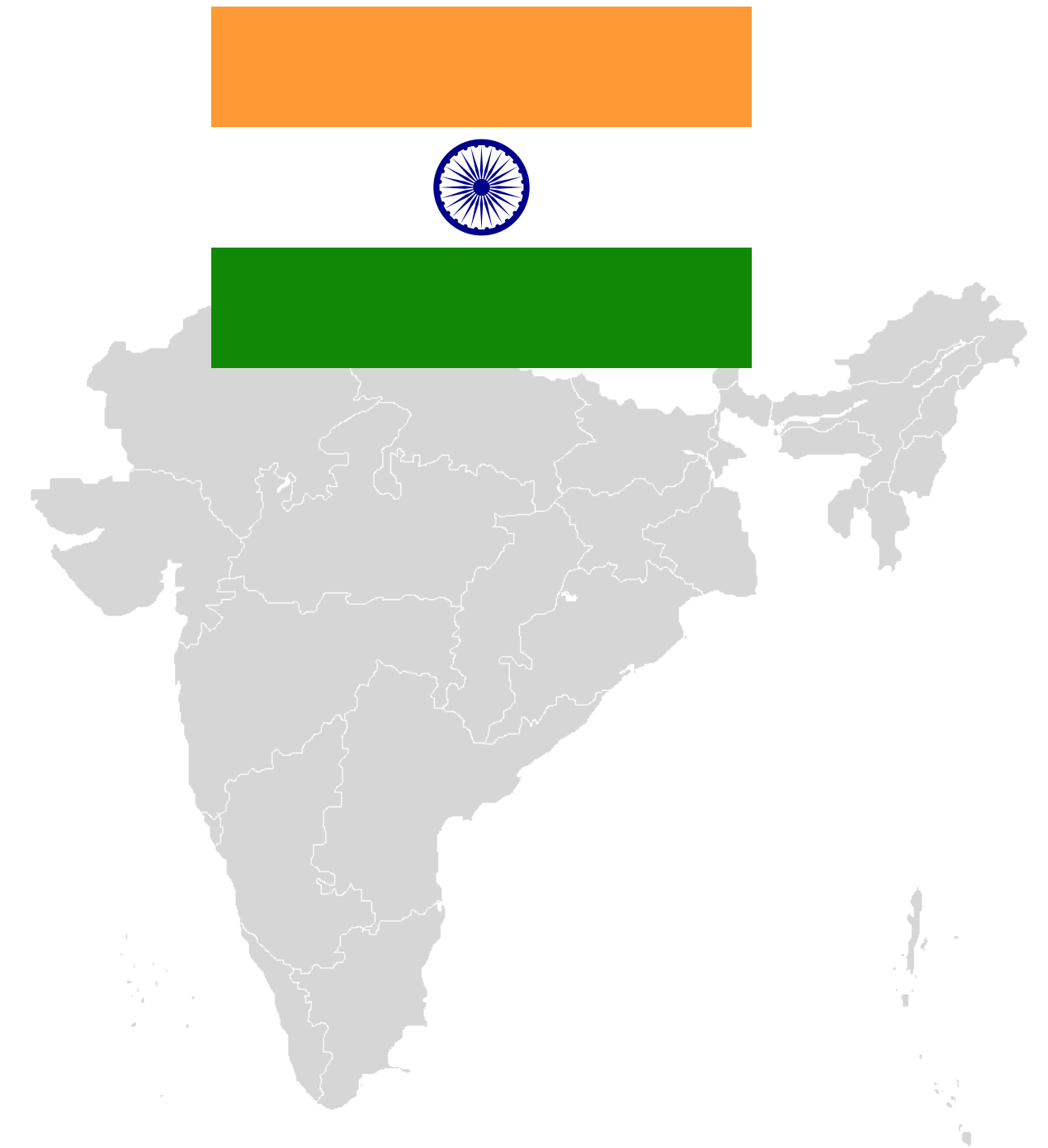
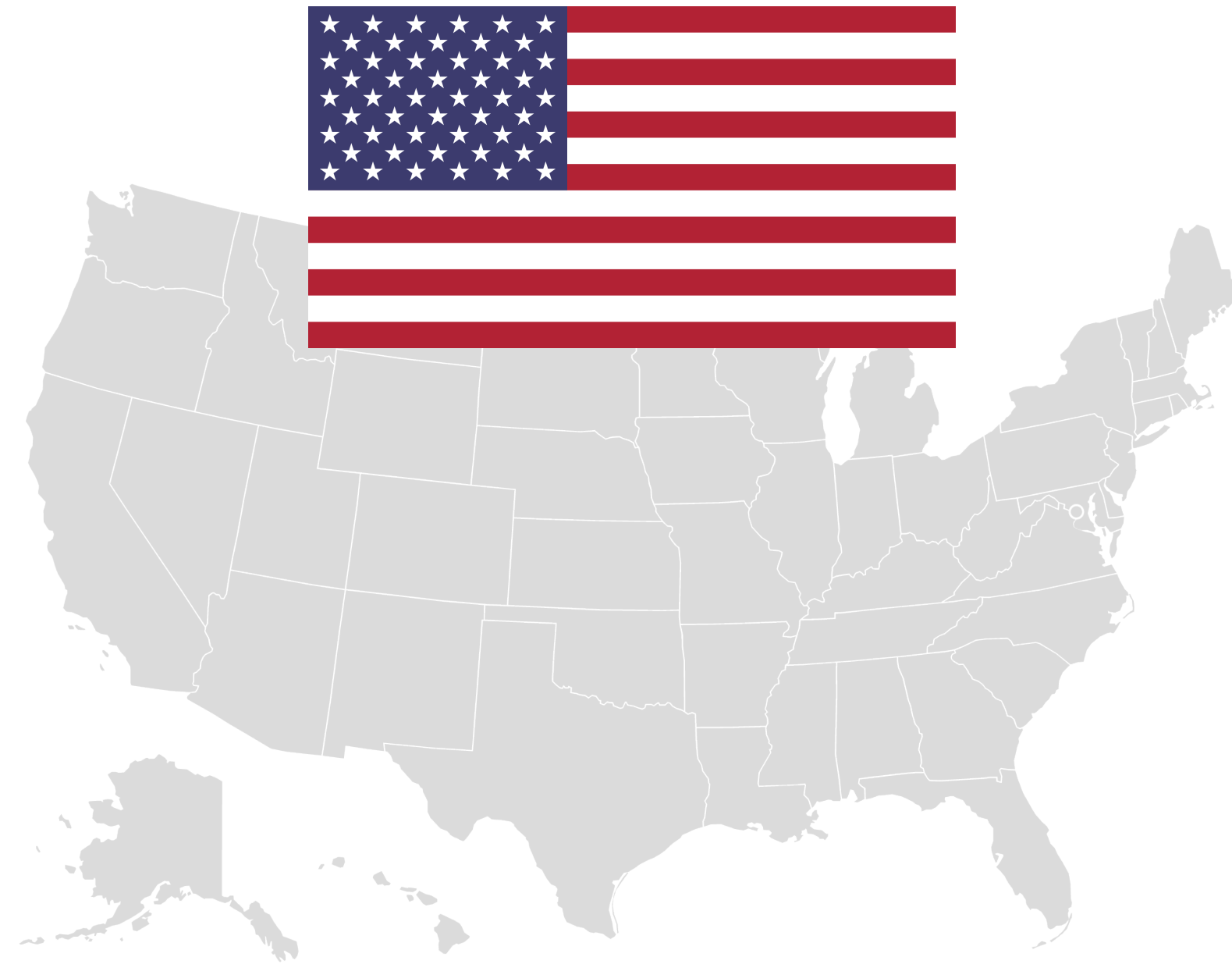
Multiplier: OT-based

Parties: 4

Curve: 256-bit

2 Mbits

sent per party



Example 1: Mobile Wallet

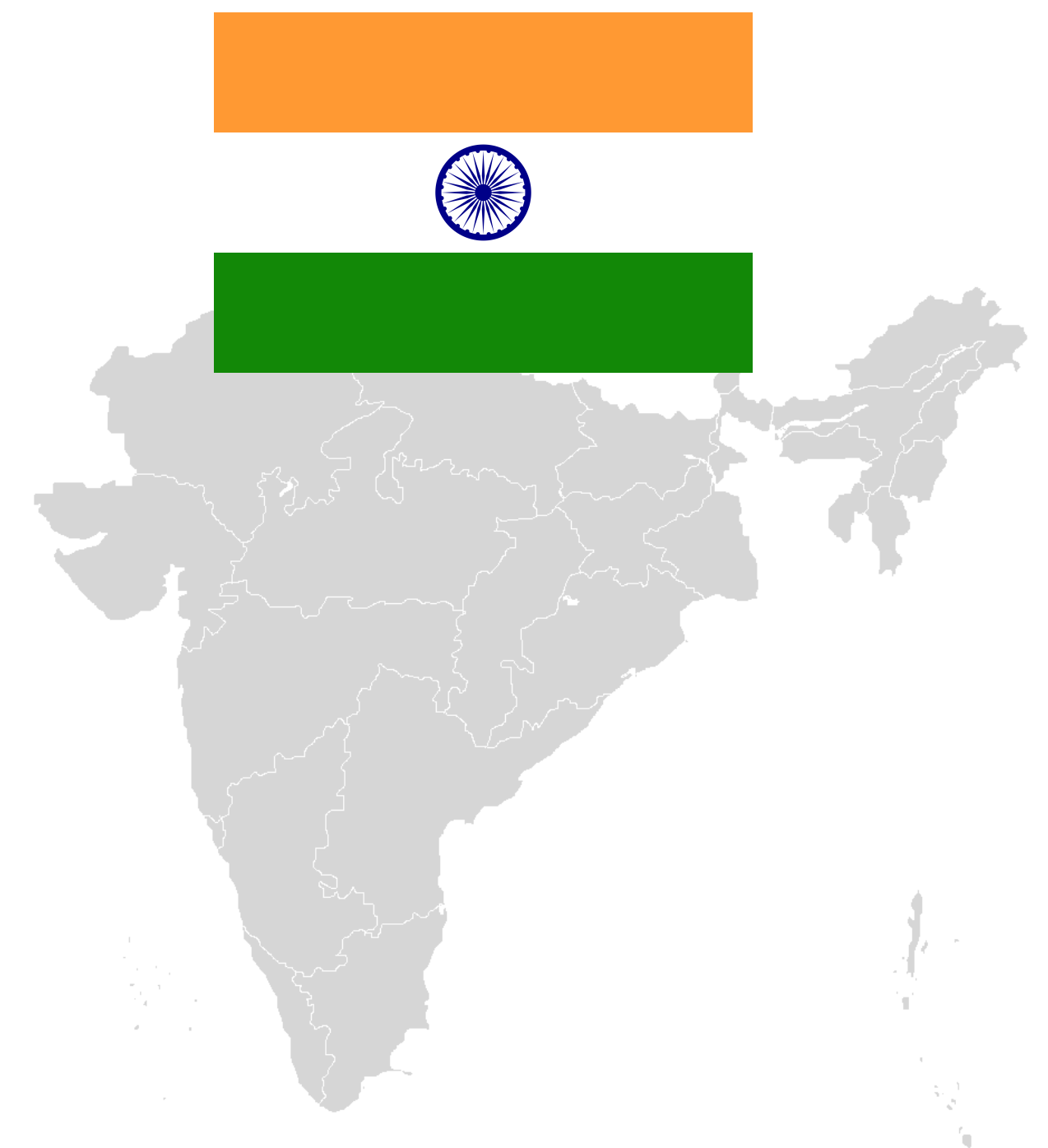
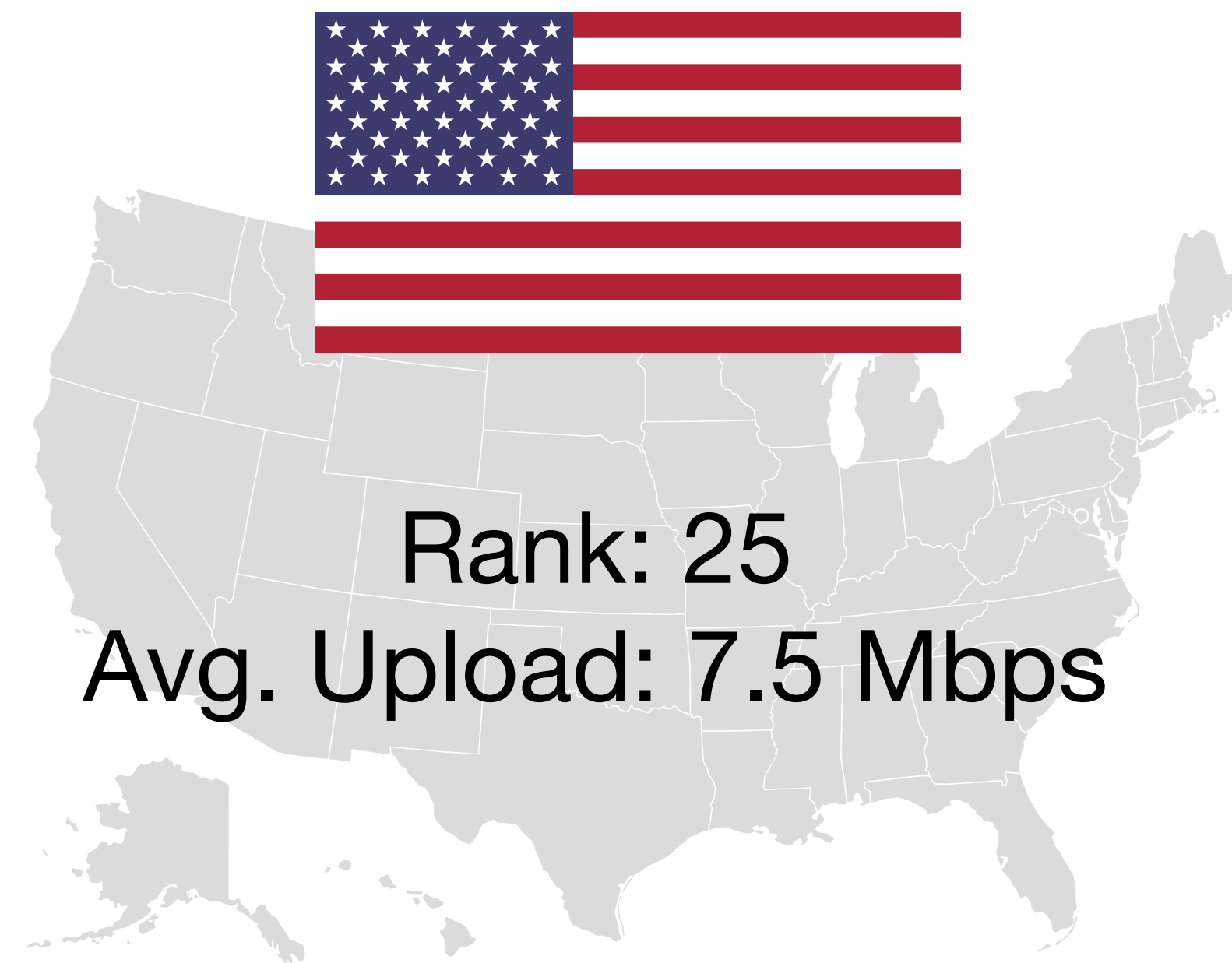
Multiplier: OT-based

Parties: 4

Curve: 256-bit

2 Mbits

sent per party



source: opensignal (2020)

Example 1: Mobile Wallet

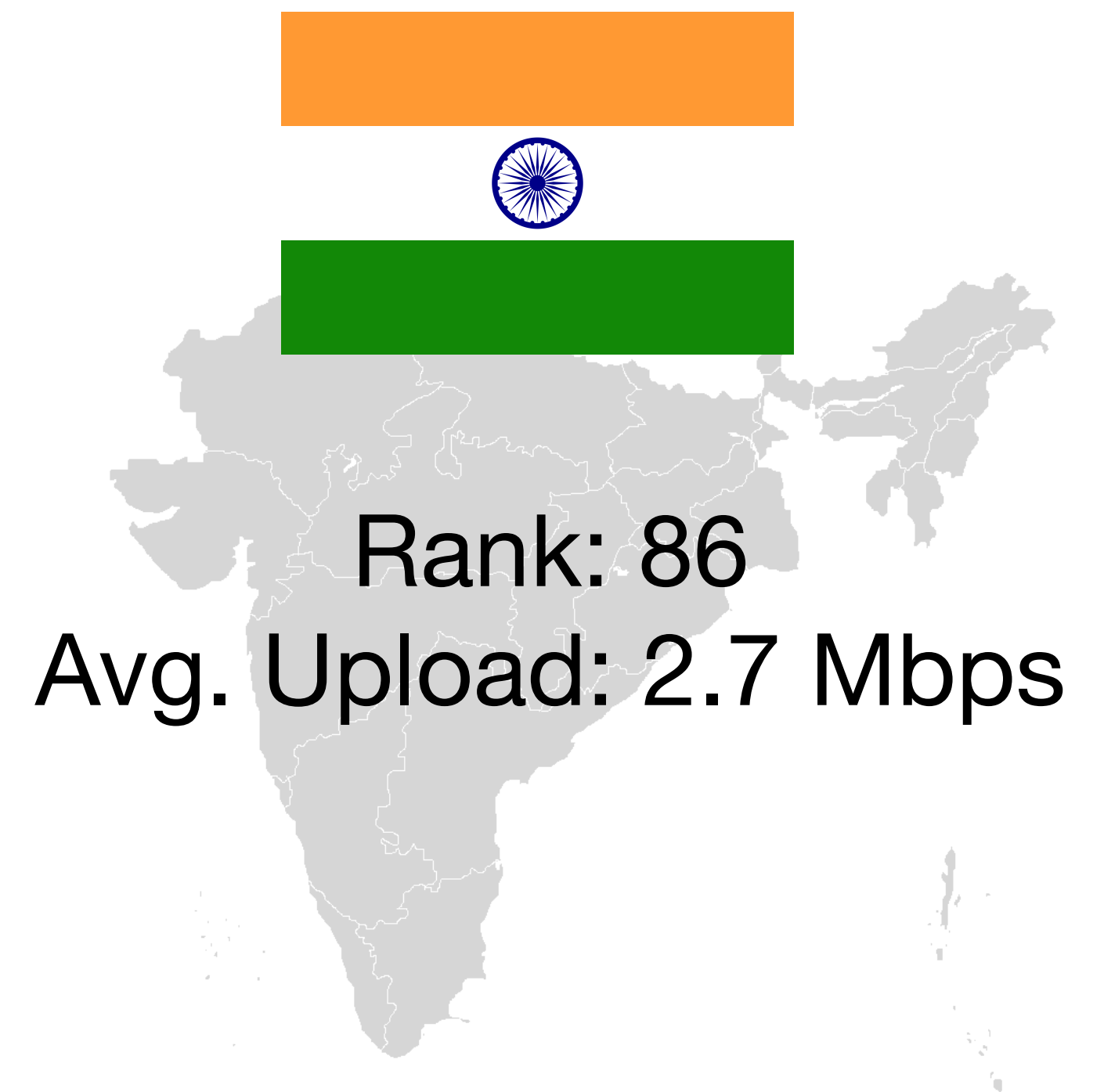
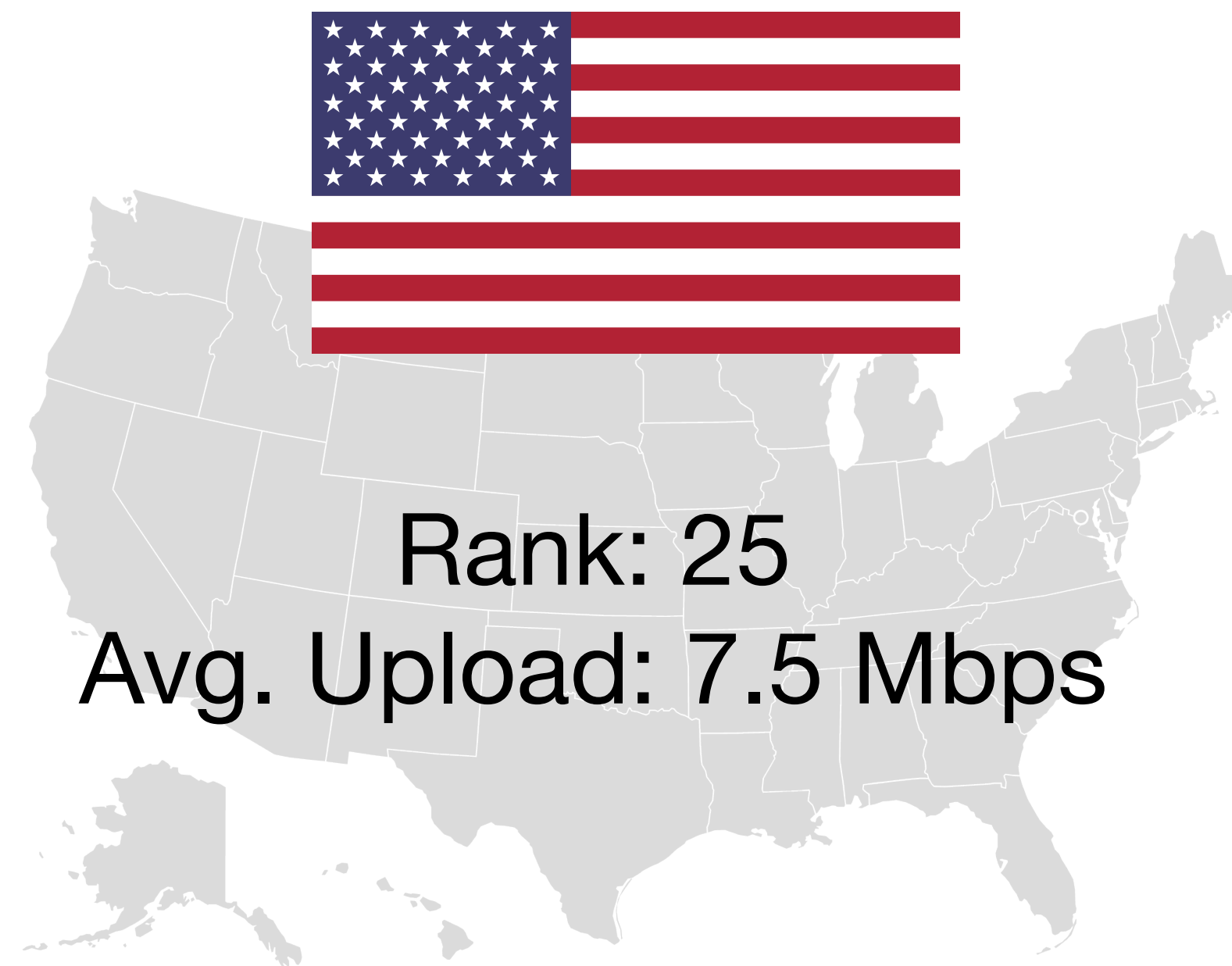
Multiplier: OT-based

Parties: 4

Curve: 256-bit

2 Mbits

sent per party



source: opensignal (2020)

Example 1: Mobile Wallet

Multiplier: OT-based

Parties: 4

Curve: 256-bit

2 Mbits

sent per party



source: opensignal (2020)

Example 1: Mobile Wallet

Multiplier: OT-based

Parties: 4

Curve: 256-bit

2 Mbits

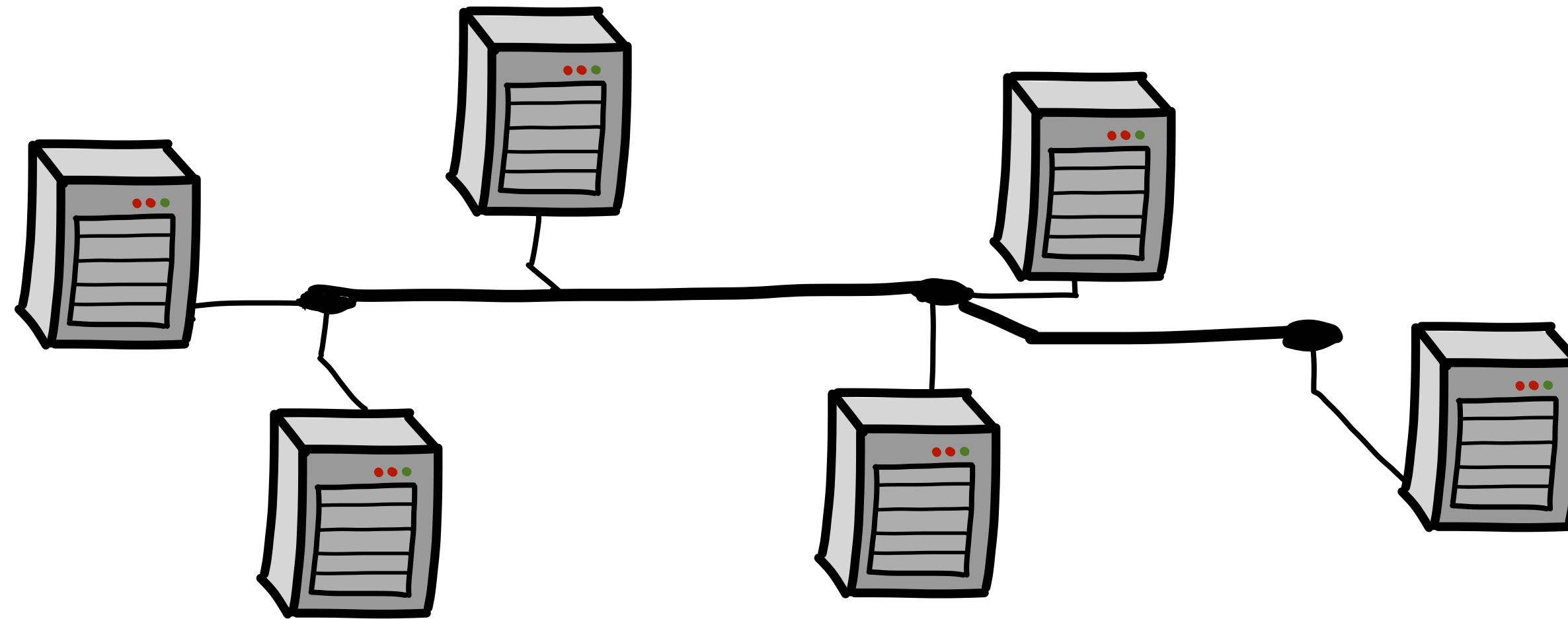
sent per party



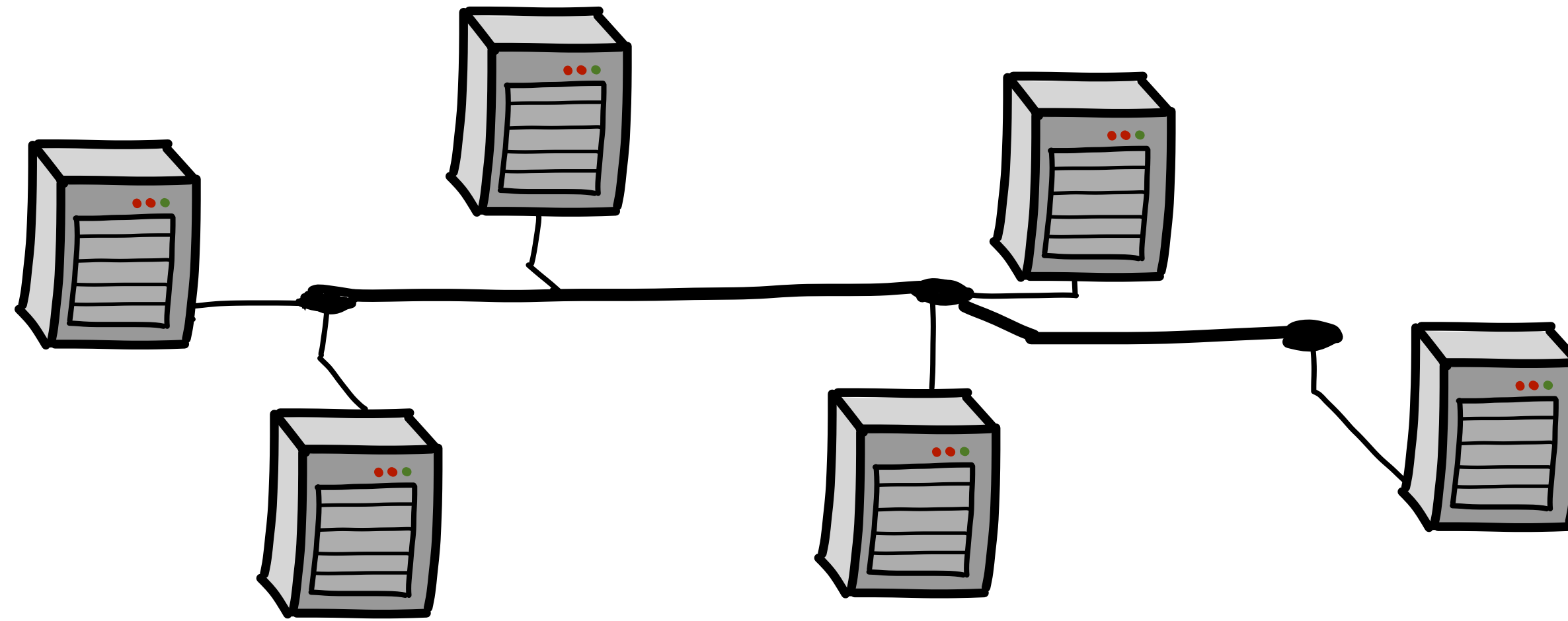
Paillier+ZK takes this long for computation alone
on powerful hardware!

source: opensignal (2020)

Is communication the bottleneck?

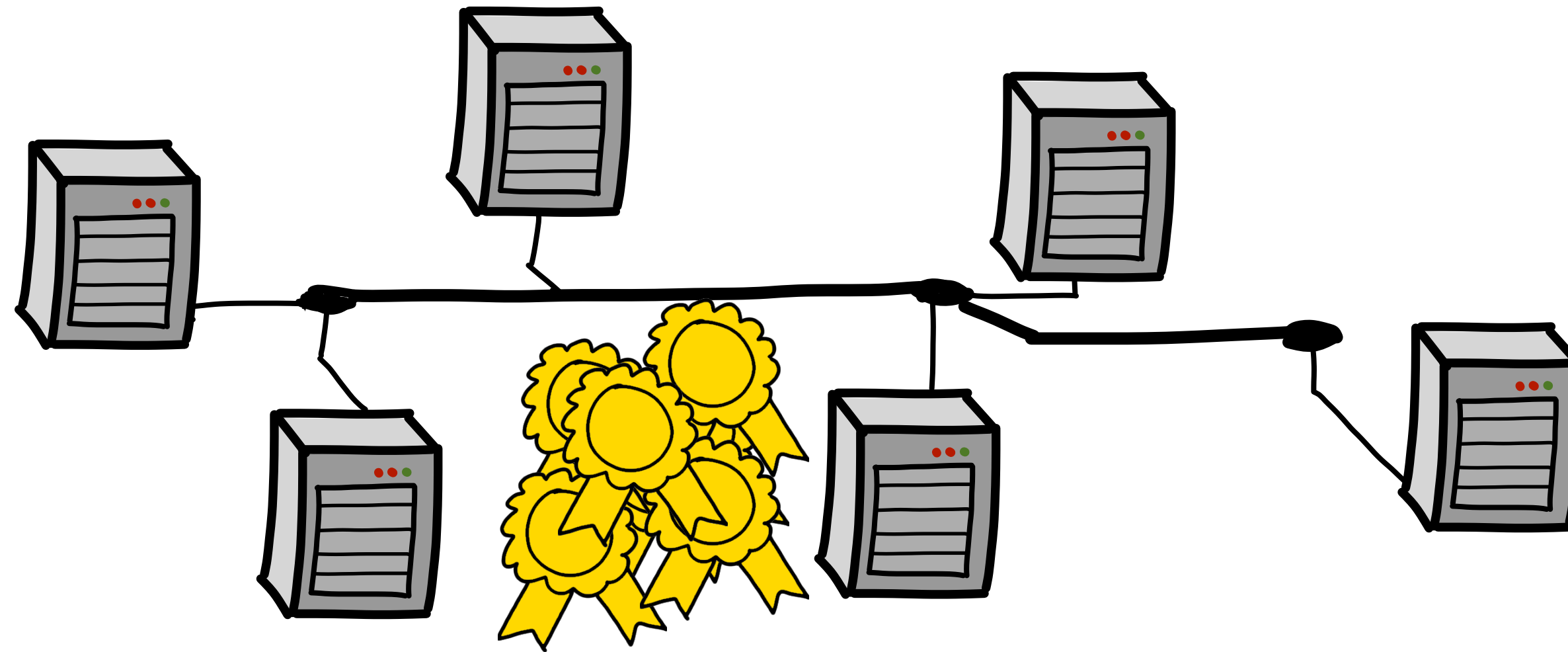


Is communication the bottleneck?



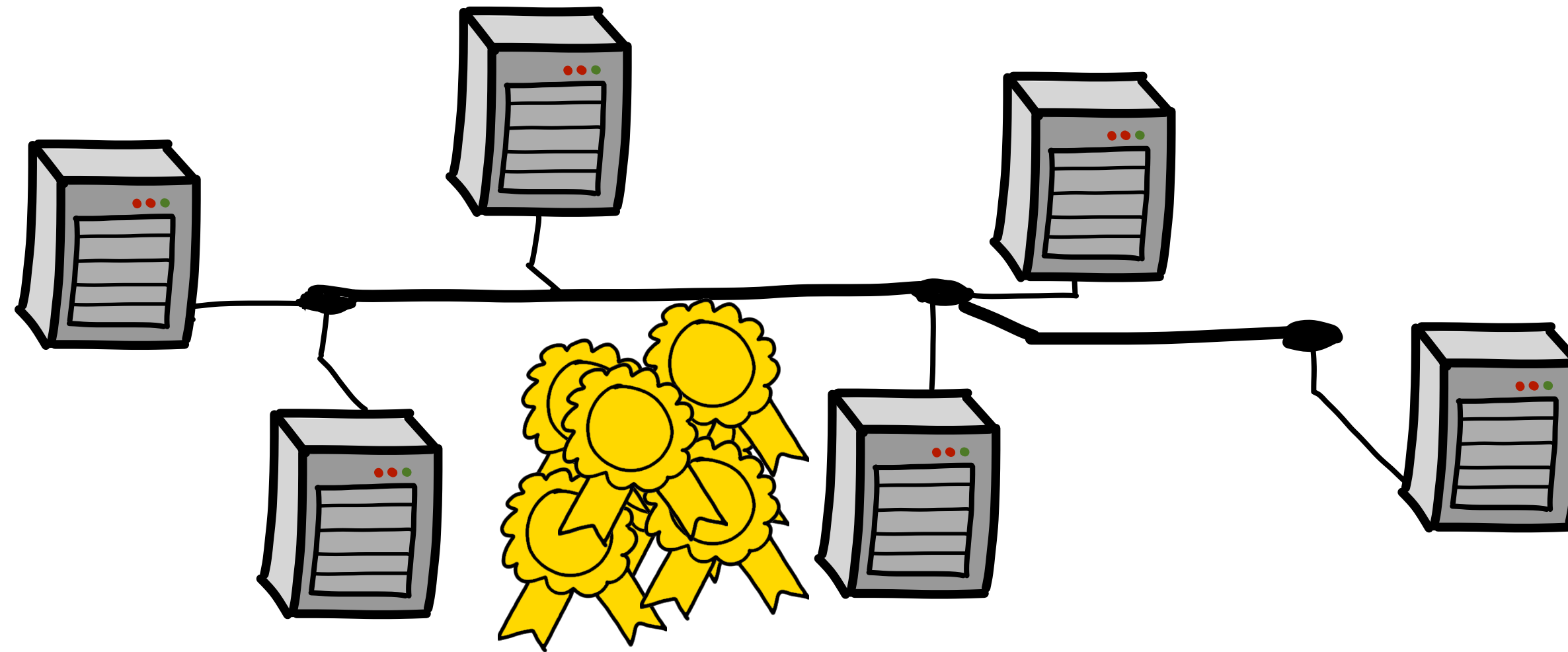
- Large-scale automated distributed signing:

Is communication the bottleneck?



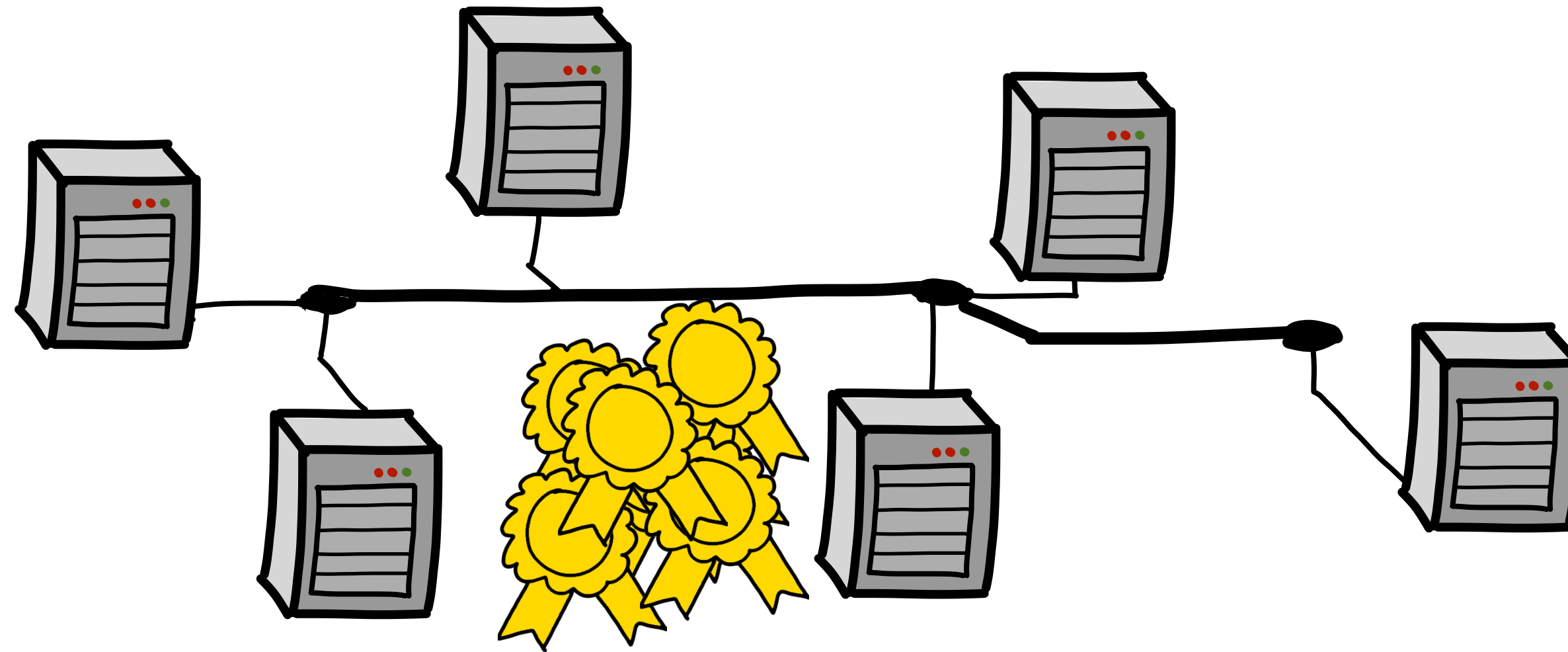
- Large-scale automated distributed signing:

Is communication the bottleneck?



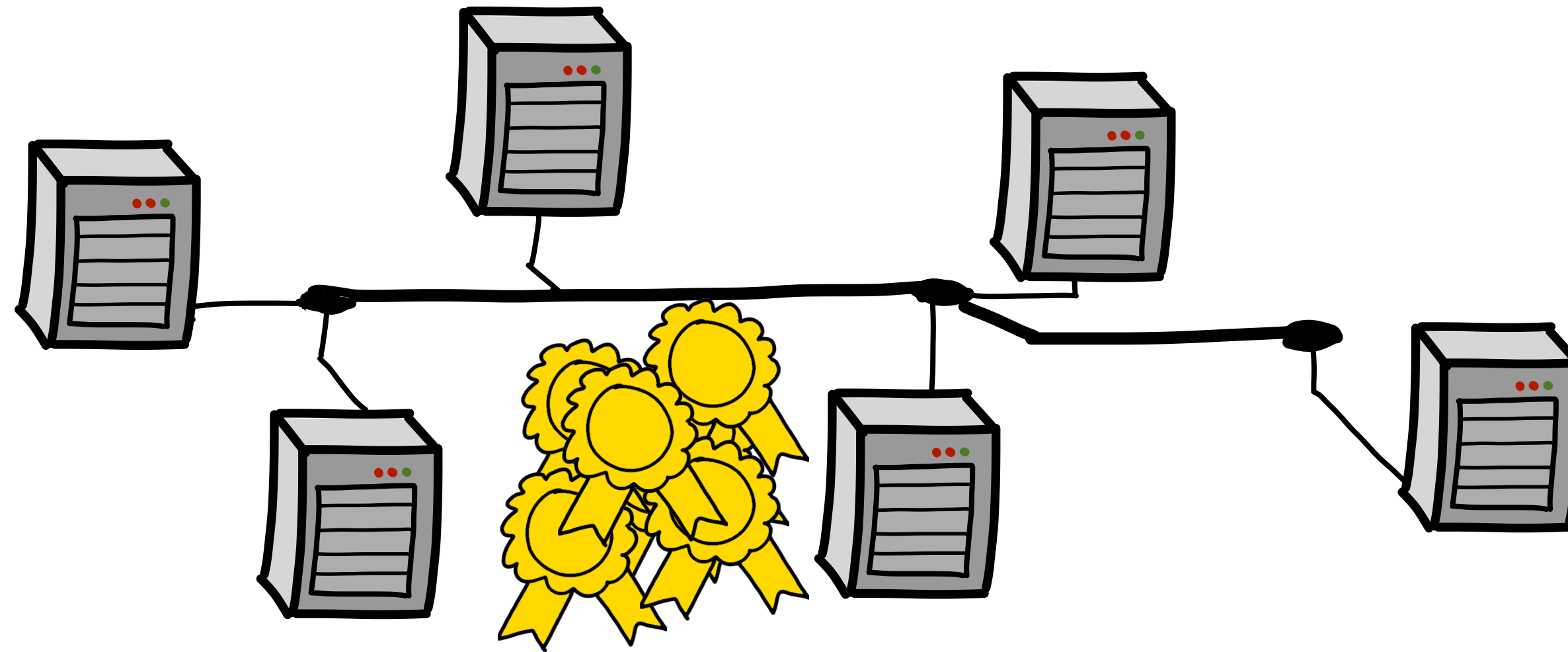
- **Large-scale automated distributed signing:**
 - Threshold 2: $3.8\text{ms/sig} \leq \sim 263 \text{ sig/second}$

Is communication the bottleneck?



- **Large-scale automated distributed signing:**
 - Threshold 2: 3.8ms/sig $\leq$ ~263 sig/second
 - Threshold 20: 31.6ms/sig $\leq$ ~31 sig/second

Is communication the bottleneck?



- **Large-scale automated distributed signing:**
 - Threshold 2: 3.8ms/sig $\leq$ ~263 sig/second
 - Threshold 20: 31.6ms/sig $\leq$ ~31 sig/second
- Neither setting saturates a gigabit connection

Example 2: Datacenter Signing

How much bandwidth to be CPU bound?
(including preprocessing)

2 Parties
~250 sigs/second

256 Parties
~3 sigs/second

using GCP n1-highcpu nodes

Example 2: Datacenter Signing

How much bandwidth to be CPU bound?
(including preprocessing)

2 Parties
~250 sigs/second

Each party sends:
~700 Kbits per sig

256 Parties
~3 sigs/second

Each party sends:
~185 Mbits per sig

using GCP n1-highcpu nodes

Example 2: Datacenter Signing

How much bandwidth to be CPU bound?
(including preprocessing)

2 Parties

~250 sigs/second

Each party sends:
~700 Kbits per sig

Bandwidth required:
~180 Mbps symmetric

256 Parties

~3 sigs/second

Each party sends:
~185 Mbits per sig

Bandwidth required:
~555 Mbps symmetric

using GCP n1-highcpu nodes



Intro

MP-Schnorr
is easy

but not
ECDSA



How to distribute ECDSA

Evolution of
Techniques

ECDSA
Tuples

Our protocol:
Simple consistency
check



Tradeoffs

OT vs
AHE

In Conclusion

- Threshold ECDSA in Three Rounds: Now matches Schnorr
- Enabled by well-chosen correlation + simple new consistency check
- Blackbox use of UC 2-round 2P-MUL
NOTE: OT-based protocols satisfy UC, but AHE is more complicated
- No (explicit) ZK proofs during signing or DKG
⇒ light protocol and straightforward UC analysis

dkls.info

Thanks!

